Probabilistic dynamic semantics

Elements in Semantics

DOI: 10.xxxx/xxxxxxxx (do not change)
First published online: MMM dd YYYY (do not change)

Julian Grove
University of Florida

Aaron Steven White
University of Rochester

Abstract: We introduce the framework of Probabilistic Dynamic Semantics (PDS), which we use to seamlessly integrate dynamic semantic analyses of lexical and discourse phenomena in the Montagovian tradition with probabilistic models of linguistic inference datasets. We show how PDS provides a general standpoint from which to understand uncertainty and context sensitivity in discourse and briefly illustrate applications to gradable adjective semantics and the veridicality inferences of clause-embedding predicates.

Keywords: semantics, probabilistic models

JEL classifications: A12, B34, C56, D78, E90

ISBNs: xxxxxxxxxxxxx(PB) xxxxxxxxxxxxx(OC)
ISSNs: xxxx-xxxx (online) xxxx-xxxx (print)

Contents

1 Introduction

A core feature of dynamic semantic frameworks is their ability to precisely model how a linguistic expression's semantic value changes the context in which it is used (Asher, 1993; Dekker, 1993; J. Groenendijk & Stokhof, 1990; J. A. Groenendijk & Stokhof, 1991; Heim, 1982, 1983; Kamp, 1981; Kamp & Reyle, 1993; Karttunen, 1976, i.a.). The integration of dynamic semantics with Montague semantics has in turn yielded a family of frameworks within which precise statements can be made about the way dynamic semantic values of complex expressions are compositionally derived from the morphemes that constitute them (de Groote, 2006; Muskens, 1996, i.a.). This integration has been deepened even further with the introduction of powerful abstractions from programming language theory that allow a variety of semantic phenomena—indefiniteness, coreference, and quantification, among others—to be modeled in terms of distinct modules that themselves compose with each other in natural ways (Barker, 2002; Barker & Shan, 2014; Bumford & Charlow, 2022, 2025; Charlow, 2014; Shan, 2002, i.a.).

One major strength of these frameworks, which we henceforth refer to under the heading of *Dynamic Montague Semantics* (DMS), is their ability to model a rich array of interpretive ambiguities. For example, from (1), one is quite likely to infer that the magician's assistant is secretly pleased, but not necessarily that the magician is pleased, even though, in principle, it may be that both are, or even that only the magician is.

(1) a. Whenever anyone laughed, the magician scowled and their assistant smirked.

b. They were secretly pleased.

These inferences can be explained by invoking an ambiguity in what *they* can be used to refer to when it is preceded by (1a). In DMS, this ambiguity is commonly explained by positing that pronouns *read* representations of referents from representations of contexts, and that ambiguities ensue due to the availability of multiple readable referent representations (Karttunen, 1976, et seq.). In the case of (1), a DMS analysis might posit (i) that referent representations for a laugher, the magician, the assistant, and all subgroups of the three are added to some representation of (1a)'s context of utterance; (ii) that the pronoun *they* is ambiguous between being singular and plural (or is simply underspecified for number); and (iii) that after the contextual representation is changed by (1a), (a) only the individual referent representations of the magician and the magician's assistant are readable if *they* is interpreted

as having an individual referent; and (b) only the referent representation of the group containing both the magician and the magician's assistant is readable if *they* is interpreted as having a group referent. Ultimately, there are three possible interpretations. Notably, none of these interpretations involve a laugher necessarily being pleased.

Two properties of this explanation are important. First, it understands the interpretation process as compositional. Without compositionality, it is difficult to explain why there is an apparent referential ambiguity in the first place, since it's unclear why ambiguities about the interpretation of pronouns should percolate to ambiguity about the interpretation of entire sentences and discourses. Second, any uncertainty about the inferences to be drawn is explained as a consequence of *summing over* interpretations, the result of which may jointly support only a subset of the inferences licensed by each interpretation on its own. That one does not necessarily infer that the magician—or for that matter, their assistant—was pleased results from three distinct interpretations being summed over: one implying that the magician was pleased, one implying that their assistant was pleased, and a third implying that both were.

DMS provides powerful tools for specifying the range of semantic values a linguistic expression *can* have—and thus the inferences it must and may support. But at the same time, it does not typically provide a way of specifying the likelihood that an expression *will* have a particular semantic value in a particular context. For example, it does not model the likelihood that *they* will refer to an individual versus a group in (1b). As a consequence, it does not make predictions about the likelihood that a language comprehender will draw a particular inference.

A natural place to look when approaching this challenge is the probabilistic semantics and pragmatics literature (see Erk, 2022, for a recent review). Here, semantic values can take a variety of forms, such as probabilistic programs (Goodman & Lassiter, 2015) or graphical models (Erk & Herbelot, 2023), and can naturally state the likelihood of an inference in terms of the likelihood of particular interpretations of an expression. The main downside of these approaches is that they do not (currently) allow one to take up the modular approach to theory development that makes DMS so powerful. For instance, if one wants to combine a particular analysis of indefiniteness, coreference, and/or quantification with a probabilistic semantics, one must integrate them "by hand"; it is not simply a matter of defining a module that can be plugged into an existing theory or framework.

The aim of this text is to introduce a framework—*Probabilistic Dynamic Semantics* (PDS)—that supports this sort of plugability as a means to seamlessly synthesize insights from both the DMS literature and the probabilistic semantics

and pragmatics literature. The main idea behind the framework is to view probabilistic reasoning as arising from a module that can be added to a semantic theory without fundamentally changing its structure. PDS has three important features:

- *Compositionality*: analyses of complex expressions incorporating probabilistic uncertainty are derived from analyses of simpler expressions incorporating probabilistic uncertainty, using the type of composition scheme prevalent in DMS.
- *Modularity*: traditional semantic analyses may be *lifted* into the probabilistic setting without tampering with their core structure.
- *Formal evaluability*: in addition to being applicable to the traditional sources of data that inform the development of semantic theory (i.e., informal inference and acceptability judgments), analyses couched within PDS can be quantitatively compared with one another by evaluating their relative fits to experimental data using standard model comparison metrics.

The first and second features are underwritten by the fact that the probabilistic reasoning components of PDS are developed using the same sorts of powerful abstractions from programming language theory that modern DMS approaches use to ensure modularity. Because of this, we can ensure that these probabilistic reasoning components may be well integrated into existing DMS frameworks. At the same time, these two features characterize some existing probabilistic semantic frameworks to varying extents. Bernardy, Blanck, Chatzikyriakidis, and Maskharashvili (2022), for example, introduce a dependently typed language which can be used to carry out probabilistic reasoning, and Grove and Bernardy (2023) present similar ideas in a modular format using λ-calculus and continuations.

The third feature—formal evaluability—is unique among DMS frameworks. PDS supports this feature by regarding every semantic representation as leading a double life. Semantic representations always have a discrete structure familiar from DMS. This discrete structure determines the range of interpretations—and thereby, the range of inferences—supported by a particular discourse. But PDS representations can always be *evaluated* to a probabilistic model in a way that retains important aspects of the discrete structure while also supporting fine-grained weightings of the inferences which that structure determines.

The methodological benefit afforded by this duality is that one may develop and test semantic analyses using quite fine-grained aspects of the distributions of judgments in formally collected experiments, including, e.g., variability in how comprehenders approach a particular task. This capability is itself supported by PDS's modularity; the way that PDS evaluates discrete structures into

probabilistic models allows these models to be composed with further processes linking distributions on inferences with distributions on judgments about those inferences. PDS makes these *linking models* explicit by encoding them as functions from answers to a question under discussion (QUD; Ginzburg, 1996; Roberts, 2012) onto possible responses associated with some data collection instrument (e.g., a Likert scale, slider, etc.). Because they are defined explicitly—in type theory, no less—linking models can be stated and studied in their own right, independent of the particular semantic analysis under investigation.

In the next section, we give an overview of our proposals before formalizing them in Section 3. We illustrate a number of example analyses using the framework and the probabilistic models they give rise to in Section 4.

Before we dive in, we should stress that our aim in this text is to lay out the formal details of the framework of PDS, as well as to lay out some existing and potential applications of it to formal experimental data in a way that is relatively abstract and programmatic. We take this tack for expository reasons: by pulling back a little, we can characterize the more general research program that we believe PDS represents.[1]

2 High-level sketch

DMS sometimes models utterance meanings as maps from discourse states into sets of discourse states:[2] states tend to keep track of which anaphoric possibilities are available for pronouns, while *sets* of such states model the non-determinism associated with the meanings of indefinite expressions such as *a magician* (J. A. Groenendijk & Stokhof, 1991; Muskens, 1996, i.a.). PDS inherits this functional view of utterances; but following much work in the probabilistic semantics and pragmatics literature (Bergen, Levy, & Goodman, 2016; Frank & Goodman, 2012; Lassiter, 2011; Lassiter & Goodman, 2017; van Benthem, Gerbrandy, & Kooi, 2009; Zeevat, 2013, i.a.), it translates this idea into a probabilistic setting: in PDS, utterances denote maps from discourse states to *probability distributions* over discourse states. Thus in comparison to DMS, PDS introduces a weighting on discourse states, allowing one to model preferences for certain resolutions of ambiguity over others.

[1] For an example application of the framework, see Grove and White (2024).

[2] In its distributive implementations, that is (for a discussion of distributive vs. non-distributive variants of dynamic semantics, see, e.g., Charlow, 2019).

2.1 Probability distributions as monadic values

In and of itself, the move to layer a probabilistic regime on top of a traditional dynamic semantics is not entirely new (see, e.g., Erk & Herbelot, 2023). More novel is that we implement this view of probabilistic uncertainty by regarding probability distributions as *monadic values* that inhabit types arising from a *probability monad* (see Bernardy, Blanck, Chatzikyriakidis, Lappin, & Maskharashvili, 2019; Giorgolo & Asudeh, 2014; Grove & Bernardy, 2023). We formalize this view in Section 3.2; but the gist is that viewing probability distributions this way allows PDS to map linguistic expressions of a particular type to probability distributions over objects of that type so that the usual compositional structure of semantic analyses is retained. This in turn allows PDS to compose probabilistic analyses with other DMS analyses of phenomena traditionally approached within dynamic semantics, such as anaphora. Meanwhile, it allows one to define explicit *linking models* that map probability distributions over discourse states to probability distributions over judgments recorded using some response instrument.[3]

Crucial for PDS is that the functorial structure arising from the monadic characterization of probability distributions allows them to be *stacked*: not only can you sample ordinary values from probability distributions, but you can sample entire probability distributions from them. Because the types derived from a probability monad may be inhabited by distributions *over* distributions, this stacking can be as deep as necessary to model the sorts of uncertainty of interest to the analyst.

2.2 Two kinds of uncertainty

In the current text, we argue that at least two levels of stacking are necessary in order to appropriately model two kinds of interpretive uncertainty, respectively, which we refer to as *resolved* (or *type-level*) *uncertainty* and *unresolved* (or *token-level*) *uncertainty*. Resolved uncertainty is the kind of uncertainty which relates to lexical, structural, or semantic (e.g., scopal) ambiguity. For example, a polysemous word gives rise to resolved uncertainty. Based on the content of its direct object, *ran* in (2) seems likely to take on its locomotion sense, though it is plausible that it also has a management sense, i.e., if Jo is understood to be

[3]This capability is often discussed in the experimental linguistics literature under the heading of *linking hypotheses* or *linking assumptions* (see Phillips, Gaston, Huang, & Muller, 2021). For our purposes, we define linking models to be statistical models that relate a PDS analysis (which determines a probability distribution over the inferences supported by a linguistic expression) to comprehenders' judgments, as recorded using a particular instrument.

the race's organizer.

(2) Jo ran a race.

In contrast, unresolved uncertainty is that which is associated with an expression in view of some *fixed* meaning it has. Vague adjectives give rise to unresolved uncertainty, as witnessed by the vague inferences they support: the heights of entities of which *tall* can be truthfully predicated remains uncertain on any given use of (3), even while the adjective's meaning plausibly does not always vary across such uses.

(3) Jo is tall.

In general, we conceptualize unresolved uncertainty as reflecting the uncertainty that one has about a given inference at a particular point in some discourse after having fixed the meanings of the linguistic expressions.

Put slightly differently, one typically has resolved uncertainty because of one's knowledge about the meanings of expressions *qua* expressions. Sometimes *run* means this; sometimes it means that. Thus, any analysis of the uncertainty about the meaning of *run* should capture that it is uncertainty about *types* of utterance act. In contrast, unresolved uncertainty encompasses any semantic uncertainty that remains, having fixed the type of the utterance act—it is uncertainty that inheres in the inferences themselves.[4]

To capture this idea, our approach regards these types of uncertainty as interacting with each other in a restricted fashion, indeed one which takes advantage of the fact that distributions may be stacked. Because resolved uncertainty must be *resolved* in order for one to draw semantically licensed inferences from uses of particular expressions, we take resolved parameters to be *fixed* in the computation of unresolved uncertainty. As we describe in Section 4, this rigid connection among sources of uncertainty is a natural consequence of structuring probabilistic reasoning in terms of stacked probability distributions. We show how PDS can capture the distinction between levels of uncertainty in Section 4.

2.3 Discourse states

We follow a common DMS practice by regarding discourse states as tuples of parameters. We depart slightly from the usual assumption that these tuples

[4]See Beaver (1999, 2001), which describe an analogous bifurcation of orders of pragmatic reasoning in the representation of the common ground.

are homogenous lists by treating them as potentially arbitrarily complex, i.e., *heterogeneous* (though, see Bumford & Charlow, 2022; Grove, 2019). As such, they could be structured according to a variety of models sometimes employed in formal pragmatics. For example, we will define one parameter of this list to be a representation of the Stalnakerian common ground (or more aptly, the "context set": Stalnaker, 1978, et seq.) and another parameter to be the QUD stack (Ginzburg, 1996, et seq.), or *table* (Farkas & Bruce, 2010).

We represent common grounds as probability distributions over indices that encode information both about how the world is and about the values of certain kinds of linguistic parameters. The "possible world" part of an index represents facts about, e.g., individuals' heights, while the "linguistic parameter" part encodes certain facts about lexical meaning, e.g., the identity of the height threshold conveyed by a vague adjective such as *tall* (see Kennedy, 2007; Kennedy & McNally, 2005; Lassiter, 2011, i.a.).

Utterances—and more broadly, discourses—map tuples of parameters onto probability distributions over new tuples of parameters. Moreover, complex linguistic acts may be *sequenced*. To model the effect of multiple linguistic acts on an ongoing discourse, we employ the sequencing operation (*bind*) native to the probability monad. As a result, compositionality of interpretation obtains in PDS from the level of individual morphemes all the way up to the level of complex exchanges. For example, a discourse may consist in (i) making an assertion, which (perhaps) modifies the common ground; (ii) asking a question, which pushes a QUD to the top of the QUD stack; or (iii) a sequence of these. Regardless, we require the functions encoding discourses to return probabilistic values, in order to capture their inherent uncertainty.

2.4 Linking models

A linking model takes a discourse as conceived above, together with an initial probability distribution over discourse states, and links them to a distribution over responses to the current QUD (i.e., the one at the top of the QUD stack). The possible responses to the QUD, moreover, are determined by a data collection instrument—this could be a Likert scale, a slider scale, or something else. In the context of fitting a statistical model to an inference dataset consisting of responses on the testing instrument, this distribution is fixed by a *likelihood function*, which is chosen from a family of functions constrained by the nature of the values encoded by the instrument—e.g., a Bernoulli distribution for instruments that produce binary values; a categorical distribution for instruments that produce unordered, multivalued discrete responses; a linked logit distribution

for instruments that produce ordered, multivalued discrete responses; and so on.

3 Formalizing the framework

We now fill in the formal details of the above sketch. First, in Section 3.1, we introduce the basic aspects of our grammar formalism, which is combinatory categorial grammar (CCG; Steedman, 1996, 2000). Then, in Section 3.2, we equip grammatical operations with probabilistic effects and show how probabilistic uncertainty in discourse can be modeled in terms of a probabilistic update operation on discourse states. Throughout this section and the remainder of the text, we provide both formal definitions and code snippets that illustrate the implementation of these definitions in the Haskell programming language.

3.1 CCG

CCG is a highly lexicalized grammar formalism, in which expressions inhabit *categories* that encode their selectional and distributional properties. A noun phrase such as *a race*, for example, may be given the type np, while a determiner—something which in English occurs to the left of a noun in order to form a noun phrase—may be given the type np/n. (Thus the forward direction of the slash indicates that a noun should occur to the *right* of the determiner.)

We use CCG in our presentation of PDS to support the development of *semantic grammar fragments*, which produce analyses of a given collection of semantic phenomena. Having a grammar fragment—e.g., one which generates the stimuli about which inference judgments are experimentally collected in a linguistic dataset—allows one to provide an unbroken chain connecting the semantic analysis of a given phenomenon to a probabilistic model of judgments about expressions exhibiting the phenomenon. CCG is a particularly useful formalism for the development of semantic grammar fragments because it assumes a tight connection between syntactic categories and semantic types, which in turn supports a succinct expression of the fragment. Another benefit is that it is likely to be sufficiently expressive to capture most (if not all) of the kinds of syntactic dependencies found in natural languages (Joshi 1985; Vijay-Shanker and Weir 1994, et seq.; cf. Kobele 2006). There is nothing inherent to PDS that requires the use of CCG, and fragments using alternative syntactic formalisms—such as, e.g., Minimalist Grammars (Stabler, 1997)—would also be possible.

We employ the set of categories of (4), which we define in Haskell in Figure 1.

```haskell
data Cat = Base String -- atomic types
         | Cat :/: Cat -- right slashes
         | Cat :\: Cat -- left slashes
  deriving (Eq)

(//), (\\) :: Cat -> Cat -> Cat
a // b = a :/: b
a \\ b = a :\: b

np, n, s :: Cat
np = Base "np"
n  = Base "n"
s  = Base "s"
```

Figure 1 CCG categories in Haskell.

(4)

$$\texttt{Base} ::= np \mid n \mid s \mid \ldots$$
$$\texttt{Cat} ::= \texttt{Base} \mid \texttt{Cat/Cat} \mid \texttt{Cat\Cat}$$

Thus we have the usual base categories for noun phrases (np), nouns (n), and sentences (s). We leave open what other base categories we might need, allowing any string of characters to represent such a category, in principle. We will later require a category $q_{deg.}$ of degree questions, when we analyze the semantics of *how likely* questions, for example. Further, given the set `Base` of base categories, the full set `Cat` of categories is the smallest superset of `Base` such that $b/a, b\backslash a \in \texttt{Cat}$ just in case $a, b \in \texttt{Cat}$.

Any complex category in `Cat` features slashes, which indicate on which side an expression of category takes its argument. Thus an expression of type b/a (for some two types a and b) occurs with an expression of type a on its right in order to form an expression of type b, while an expression of type $b\backslash a$ occurs with an expression of type a on its *left* in order to form an expression of type b. We adopt the convention of notating categories without parentheses when possible, under the assumption that they are left-associative; i.e., $a \mid_1 b \mid_2 c \stackrel{\text{def}}{=} (a \mid_1 b) \mid_2 a$ (where $\mid_1$ and $\mid_2$ are either forward or backward slashes). In Haskell, we can simply have `infixl :/:` and `infixl :\:` to ensure left-associativity. Thus for example, the type $s\backslash(np/np)$ continues to be

$$\frac{}{\Gamma, x : \alpha \vdash x : \alpha}\ \mathtt{Ax} \qquad \frac{}{\Gamma \vdash \diamond : \diamond}\ \diamond\mathtt{I}$$

$$\frac{\Gamma, x : \alpha \vdash t : \beta}{\Gamma \vdash \lambda x.t : \alpha \to \beta} \to\mathtt{I} \qquad \frac{\Gamma \vdash t : \alpha \qquad \Gamma \vdash u : \beta}{\Gamma \vdash \langle t, u \rangle : \alpha \times \beta} \times\mathtt{I}$$

$$\frac{\Gamma \vdash t : \alpha \to \beta \qquad \Gamma \vdash u : \alpha}{\Gamma \vdash t(u) : \beta} \to\mathtt{E} \qquad \frac{\Gamma \vdash t : \alpha_1 \times \alpha_2}{\Gamma \vdash \pi_j(t) : \alpha_j} \times\mathtt{E}_j$$

Figure 2 Typed λ-terms.

written as such, while the type $(s\backslash np)/np$ may be shortened to '$s\backslash np/np$'.

To write CCG expressions, we use the notation

$$\begin{array}{c} s \\ \boxed{m} \end{array} \vdash c$$

which is to be read as stating that string s has category c and semantic value m. We assume s to be a string over some alphabet Σ (i.e., $s \in \Sigma^*$), which we regard as a finite set; e.g., the set of "morphemes of English". In Haskell, we represent such a string as an expression of type `[String]`.

Meanwhile, we assume m to be a typed λ-term. We leave somewhat open what types of λ-terms may be used to define semantic values, but we adopt at least the typing rules in Figure 2. Assuming that all semantic values are closed terms, we therefore have abstractions ($\lambda x.t$), applications ($t(u)$), and n-ary tuples ($\langle t_1, \cdots, t_n \rangle$), along with the empty tuple $\diamond$. We additionally assume that λ-terms can feature constants, drawn from some countable set. In practice, we allow constants to be formed out of either strings or real numbers and provide the following Haskell definition:

```
type Constant = Either String Double
```

As for the semantic types themselves, we assume that there are the atomic types e, t, and r—though we allow atomic types to be identified with arbitrary strings in principle (see Figure 3). As usual, e is the type of entities; t is the type of truth values T and F; and r is the type of real numbers. We don't make any use of real numbers in this section, but they become central in Section 3.2.1.

In Haskell, we encode terms via a data type `Term`; types, via a data type `Type`; and typed terms, via yet another data type `Typed` (see Figure 3). Thus the implementation itself adopts an *extrinsic* (or *Curry-style*) approach to typing: the languages of terms and types are defined individually, and typing rules

```haskell
type Name = String

data Term = Var Name        -- variables
          | Con Constant    -- constants
          | Lam Name Term   -- abstractions
          | App Term Term   -- applications
          | TT              -- the 0-tuple
          | Pair Term Term  -- pairing
          | Pi1 Term        -- first projection
          | Pi2 Term        -- second projection

data Type = Atom String    -- atomic types
          | Type :-> Type  -- arrows
          | Unit           -- the unit type
          | Type :* Type   -- products
          | TyVar Name     -- type variables
  deriving (Eq)

data Typed = Typed { termOf :: Term
                   , typeOf :: Maybe Type
                   } deriving (Eq)
```

Figure 3 Terms, types, and typed terms in Haskell.

are in turn defined to reign in the set of terms to those which are well typed.[5] Moreover, just as with complex categories, we adopt the convention of notating complex semantic types without parentheses when possible. Unlike categories, we assume semantic types are right-associative.[6] To ensure this convention in Haskell, we can write `infixr :->` and `infixr :*`.

In CCG, expressions are combined to form new expressions using application rules, as well as composition (**B**) rules and (often) substitution (**S**) rules. The string *every linguist* can be derived by *right* application from expressions for the strings *every* and *linguist*, for example.

[5]Furthermore, relations on λ-terms, such as β- and η-conversion, are defined for the full un-typed language. This approach stands in juxtaposition to an *instrinsic* (or *Church-style*) approach, whereon terms are defined to carry their types by definition, and such relations are defined only on typed terms.

[6]These conventions mirror each other in the sense that the input type of a function type is assumed to be atomic unless otherwise specified by the use of parentheses.

$$\text{(5)}\quad \frac{\overset{\textit{every}}{\boxed{\lambda p, q.\forall y.p(y) \rightarrow q(y)}} \vdash s/(s\backslash np)/n \qquad \overset{\textit{linguist}}{\boxed{\mathsf{ling}}} \vdash n}{\underset{\textit{every linguist}}{\boxed{\lambda q.\forall y.\mathsf{ling}(y) \rightarrow q(y)}} \vdash s/(s\backslash np)} >$$

The resulting expression has the category of a quantifier; in this case, it takes on its right an expression which takes a noun phrase on its left to form a sentence, and it forms a sentence with that expression. This type—$s/(s\backslash np)$—is mirrored by the type of the λ-term which is the expression's semantic value: $(e \rightarrow t) \rightarrow t$. Indeed, the two are related by a *type homomorphism*; i.e., a map from categories to semantic types that preserves certain structure—here, the structure of categories formed via slashes ($/$ and $\backslash$), which get turned into semantic types formed via arrows ($\rightarrow$). We may further codify the behavior of this homomorphism on base categories (treating only np, n, and s for now).

$$\text{(6)}\quad \begin{aligned} [\![np]\!] &= e \\ [\![n]\!] &= e \rightarrow t \\ [\![s]\!] &= t \end{aligned}$$

Indeed, the CCG derivation given in (5) tacitly assumes that noun phrases denote entities, that nouns denote functions from entities to truth values, and that sentences denote truth values.

Importantly, `Type`s may be assigned to `Term`s automatically in Haskell by defining *signatures*—i.e., maps from constants to types:

```
type Sig = Constant -> Maybe Type
```

Signatures map constants onto `Maybe Type`: constants that receive a type from a signature are assigned `Just t` for some `Type t` by that signature. Not every constant will receive a type from a given signature, in which case it is mapped to `Nothing`. Once we know the signature of a set of constants, we can assign a type to any λ-term that features any constants from that set automatically, by inferring its *principal type*—that is, its most general possible type, with type variables used to represent any unknowns.

Thus for example, we can provide a signature for the constant `Left "ling"` that makes it a function of type `e :-> t`:

```
lingSig :: Sig
lingSig = \case
```

```
ty :: Sig -> Term -> Typed
ty tau te = Typed te theType
    where theType :: Maybe Type
          theType = evalStateT
                      (computeType tau te)
                      (tyVars, start, empty)

          tyVars :: TyVars
          tyVars = "" : map show ints >>= \i ->
                   map (:i) ['α'..'ω']

          ints :: [Integer]
          ints = 1 : map succ ints

          start :: VarTyping
          start = const Nothing

          empty :: Constr
          empty = []
```

Figure 4 A function which, given a signature, turns an untyped λ-term into a typed λ-term (so long as this is possible).

```
  Left "ling" -> Just (e :-> t)
  -           -> Nothing
```

Further, we can now *ask* for the type of a term featuring this constant using the function `ty`, defined in Figure 4. This function, when provided a signature and a term, returns a value of type `Typed` (i.e., a typed λ-term). For example, in GHCI:

```
ghci> ty lingSig (lam x (x @@ (Con (Left "ling"))))
λx.x(ling) : ((e → t) → β) → β
```

Here, the type variable variable β is inferred as the result of the relevant term.

Crucially, given this setup, *every* CCG rule is analogous to the application rules in that it preserves the structure of syntactic categories in the types of semantic values via the type homomorphism. For another example, the rightward composition rule can be used to combine *every linguist* with *saw*.

$$\frac{\overset{s_1}{\boxed{m_1}} \vdash c/b \qquad \overset{s_2}{\boxed{m_2}} \vdash b \mid_n a_n \cdots \mid_1 a_1}{\overset{s_1\ s_2}{\boxed{\lambda x_1,\ldots,x_n.m_1(m_2(x_1)\cdots(x_n))}} \vdash c \mid_n a_n \cdots \mid_1 a_1}\ >\mathbf{B}_n$$

$$\frac{\overset{s_1}{\boxed{m_1}} \vdash b \mid_n a_n \cdots \mid_1 a_1 \qquad \overset{s_1}{\boxed{m_2}} \vdash c\backslash b}{\overset{s_1\ s_2}{\boxed{\lambda x_1,\ldots,x_n.m_2(m_1(x_1)\cdots(x_n))}} \vdash c \mid_n a_n \cdots \mid_1 a_1}\ <\mathbf{B}_n$$

$$\frac{\overset{s_1}{\boxed{m_1}} \vdash c/b \mid_n a_n \cdots \mid_1 a_1 \qquad \overset{s_2}{\boxed{m_2}} \vdash b \mid_n a_n \cdots \mid_1 a_1}{\overset{s_1\ s_2}{\boxed{\lambda x_1,\ldots,x_n.m_1(x_1)\cdots(x_n)(m_2(x_1)\cdots(x_n))}} \vdash c \mid_n a_n \cdots \mid_1 a_1}\ >\mathbf{S}_n$$

$$\frac{\overset{s_1}{\boxed{m_1}} \vdash b \mid_n a_n \cdots \mid_1 a_1 \qquad \overset{s_2}{\boxed{m_2}} \vdash c\backslash b \mid_n a_n \cdots \mid_1 a_1 \cdots \mid_1 a_1}{\overset{s_1\ s_2}{\boxed{\lambda x_1,\ldots,x_n.m_2(x_1)\cdots(x_n)(m_1(x_1)\cdots(x_n))}} \vdash c \mid_n a_n \cdots \mid_1 a_1}\ <\mathbf{S}_n$$

Figure 5 CCG rule schemata.

$$\frac{\overset{\textit{every linguist}}{\boxed{\lambda q.\forall y.\mathsf{ling}(y) \rightarrow q(y)}} \vdash s/(s\backslash np) \qquad \overset{\textit{saw}}{\boxed{\mathsf{see}}} \vdash s\backslash np/np}{\overset{\textit{every linguist saw}}{\boxed{\lambda x.\forall y.\mathsf{ling}(y) \rightarrow \mathsf{see}(x)(y)}} \vdash s/np}\ >\mathbf{B} \tag{7}$$

Here, the resulting type—s/np—is mapped to $[\![np]\!] \rightarrow [\![s]\!] = e \rightarrow t$, which is precisely the type of the resulting semantic value.

We provide the full set of CCG rule schemata in Figure 5. These feature both composition (**B**) and substitution (**S**) rules. Following standard practice in CCG, the rule schemata define n^{th} order instances of the composition and substitution rules. Intuitively, n is the number of arguments which need to be "gotten out of the way" when applying rules. Thus for instance, $\mathbf{B}_0$ and $\mathbf{S}_0$ coincide and

$$\frac{}{\Gamma, x:\alpha \vdash x:\alpha}\,\texttt{Ax} \qquad \frac{}{\Gamma \vdash \diamond : \diamond}\,\diamond\texttt{I}$$

$$\frac{\Gamma, x:\alpha \vdash t:\beta}{\Gamma \vdash \lambda x.t : \alpha \to \beta}\,{\to}\texttt{I} \qquad \frac{\Gamma \vdash t:\alpha \qquad \Gamma \vdash u:\beta}{\Gamma \vdash \langle t,u\rangle : \alpha \times \beta}\,{\times}\texttt{I}$$

$$\frac{\Gamma \vdash t:\alpha \to \beta \qquad \Gamma \vdash u:\alpha}{\Gamma \vdash t(u):\beta}\,{\to}\texttt{E} \qquad \frac{\Gamma \vdash t:\alpha_1 \times \alpha_2}{\Gamma \vdash \pi_j(t):\alpha_j}\,{\times}\texttt{E}_j$$

$$\frac{\Gamma \vdash t:\alpha}{\Gamma \vdash \textcircled{t} : \mathrm{P}\alpha}\,\texttt{Return} \qquad \frac{\Gamma \vdash t:\mathrm{P}\alpha \qquad \Gamma, x:\alpha \vdash u:\mathrm{P}\beta}{\Gamma \vdash \begin{pmatrix} x \sim t \\ u \end{pmatrix} : \mathrm{P}\beta}\,\texttt{Bind}$$

Figure 6 Typed probabilistic programs.

correspond to the standard application rules.

$$\frac{\overset{s_1}{\boxed{m_1}} \vdash c/b \qquad \overset{s_2}{\boxed{m_2}} \vdash b}{\overset{s_1\ s_2}{\boxed{m_1(m_2)}} \vdash c}\,> \qquad \frac{\overset{s_1}{\boxed{m_1}} \vdash b \qquad \overset{s_2}{\boxed{m_2}} \vdash c\backslash b}{\overset{s_1\ s_2}{\boxed{m_2(m_1)}} \vdash c}\,<$$

Likewise, the standard **B** rules may be identified with the $\mathbf{B}_1$ rules, and similarly for the **S** rules.

3.2 Probabilistic dynamism

With the preliminaries now in place, we turn to the formalization of the properly probabilistic and dynamic components of PDS.

3.2.1 Augmenting the type system

The type system presented in Section 3.1 included types for entities (e), truth values (t), and real numbers (r), along with function and product types defined over those. Here, we follow Grove and Bernardy (2023), who show how a semantics incorporating Bayesian reasoning can be encoded using a λ-calculus with such a standard type system. However, we eschew the continuation-based presentation in Grove and Bernardy 2023 by incorporating a new type

constructor (P) for probabilistic programs; this leads to a somewhat more abstract presentation, though one which can be reasoned about with equal ease.[7]

Types of the form Pα are inhabited by *probabilistic programs* that represent probability distributions over values of type α. For example, a program of type Pt represents a probability distribution over truth values (i.e., a Bernoulli distribution); a program of type Pe represents a probability distribution over entities (e.g., a categorical distribution); and a program of type P$(e \to t)$ represents a probability distribution over functions from entities to truth values. The typing rules for probabilistic programs are provided in full in Figure 6.

Note the two rules `Return` and `Bind`, which are new. `Return` says that any term t encoding a value of type α can be lifted into a program representing a trivial probability distribution $\textcircled{t}$ of type Pα which assigns all of its mass to t; that is, if one samples a value from this distribution, that value is always t. Meanwhile, `Bind` allows a probabilistic program u of type Pβ which depends on a variable x of type α to receive a value for that variable from the probabilistic program t of type Pα. Thus the α-shaped hole inside u is filled by effectively *sampling* a value from t. Indeed, one can understand the notation '$\binom{x \sim t}{u}$', as saying, "sample x from t and continue with u."

Figure 7 shows how these new constructs and their associated types are encoded in Haskell. Note that here, `Let x t u` encodes the bind statement just mentioned. Thus when `t` is assigned type `P a` and `u` is assigned type `P b`, `Let x t u` will be assigned type `P b`. Also important to note is the novel encoding of arbitrary type constructors; in general, a Haskell expression such as `TyCon "T" [a1, ..., an]` will be understood to encode some novel type T$a_1, \ldots, a_n$. Such bespoke type constructors will be useful for defining functions on types, as we will see in Section 3.2.5, when we define a type to represent stacks of QUDs.

The probability monad

A very useful fact about the type constructor P is that it is defined to be a *monad*. For P to be a monad means that together with the two constructors return

[7] There is a substantial (and growing) literature in linguistic semantics and cognitive science that uses probabilistic programs, broadly construed. Early work, including the first version of Goodman et al. 2016 (which now relies on WebPPL (Goodman & Stuhlmüller, 2014)) carried out probabilistic computation in the Lisp library Church (Goodman, Mansinghka, Roy, Bonawitz, & Tenenbaum, 2008). Importantly, such systems are distinguished from the approach of this text in that they tend to rely on imperative constructs (e.g., for sampling and Bayesian update) to carry out probabilistic reasoning.

```
type Name = String

data Term = Var Name      -- variables
    | Con Constant        -- constants
    | Lam Name Term       -- abstractions
    | App Term Term       -- applications
    | TT                  -- the 0-tuple
    | Pair Term Term      -- pairing
    | Pi1 Term            -- first projection
    | Pi2 Term            -- second projection
    | Return Term         -- trivial distributions
    | Let Name Term Term -- sampling/effects

data Type = Atom String          -- atomic types
          | Type :-> Type        -- arrows
          | Unit                 -- the unit type
          | Type :* Type         -- products
          | P Type               -- probabilistic types
          | TyCon String [Type] -- type constructors
          | TyVar Name           -- type variables
  deriving (Eq)

data Typed = Typed { termOf :: Term
                   , typeOf :: Maybe Type
                   } deriving (Eq)
```

Figure 7 Probabilistic programs, probabilistic types, and typed probabilistic programs in Haskell.

$$\underset{\textit{Left identity}}{\begin{matrix} x \sim \boxed{v} \\ k \end{matrix} = k[v/x]} \qquad \underset{\textit{Right identity}}{\begin{matrix} x \sim m \\ \boxed{x} \end{matrix} = m} \qquad \underset{\textit{Associativity}}{\begin{matrix} y \sim \begin{pmatrix} x \sim m \\ n \end{pmatrix} \\ o \end{matrix} = \begin{matrix} x \sim m \\ y \sim n \\ o \end{matrix}}$$

Figure 8 The monad laws.

and bind (see Figure 6), it must satisfy the three monad laws in Figure 8.[8] These laws provide part of the operational interpretation of probabilistic programs—in particular, so that they together constitute a language for reasoning about probability distributions. Left identity says that sampling a value (x) from the trivial distribution $\textcircled{v}$ always results in v; as a result, one could instead continue with v in place of x, without bothering to go through the detour of sampling it. Right identity says that sampling a value (x) from a program and merely returning it (as $\textcircled{x}$) has no effect on the distribution that x was sampled from; sampling a value and immediately returning it thus also constitutes a detour. Finally, Associativity allows one to *linearize* complex probabilistic programs containing hierarchical structure; thus rather than sampling y from a complex program containing a bind statement, one can pull the bind out (so to speak) and instead sample y from its continuation (n).

Monads, which Shan (2002) introduced into the linguistic semantics literature, have allowed researchers to use very general compositional techniques to study classic dynamic semantic phenomena such as anaphora and/or indefiniteness (Charlow, 2014, 2020a, 2020b; Giorgolo & Unger, 2009), as well as conventional implicature (Giorgolo & Asudeh, 2012), intensionality (Charlow, 2020a; Elliott, 2022), and other phenomena. What monads provide in each case—and, in particular, in the case of probabilistic programs—is a general interface consisting of a collection of typed operators that obey the laws in Figure 8. In part because the monadic interface is so general, one can use it to see and study these phenomena in a way that is *modular*. A monadic analysis allows the

[8] Going from left to right, one can view these equalities as giving rise to *reduction rules* (Benton, Bierman, & Paiva, 1998). In particular, Left identity may be viewed as β-reduction ($\Rightarrow_\beta$), Right identity, as η-reduction ($\Rightarrow_\eta$), and Associativity, meanwhile, as a commuting conversion ($\Rightarrow_c$).

Because we take the constructors for probabilistic programs to be primitives of the language, such reduction rules are not redundant with those of the ambient λ-calculus. Moreover, it is useful to have both Left identity and Associativity, since they allow probabilistic programs to be reduced to a normal form in which any return statement only occurs either at the end of a program or inside the argument of a function. Consider the following reduction, for instance, in which the returned u, otherwise trapped, is eliminated in favor of a direct substitution:

$$
\begin{array}{ll}
 & y \sim \left(\begin{array}{l} x \sim t \\ \textcircled{u} \end{array}\right) \\
 & v \\
 & \\
 & x \sim t \\
\Rightarrow_c & y \sim \textcircled{u} \\
 & v \\
 & \\
 & x \sim t \\
\Rightarrow_\beta & v[u/y]
\end{array}
$$

See Figure 9 for a Haskell implementation of $\Rightarrow_\beta$ and $\Rightarrow_c$.

"Fregean bread and butter" operations like functional application (in the words of Charlow (2014)) to be seen as the compositional core of the analysis, while the phenomenon under investigation (e.g., indefiniteness) can be viewed as being piped through the analysis by the operators. For an introduction to these ideas—which regard natural language semantic analyses as based primarily on functors (of which monads are an instance)—see Bumford and Charlow 2025.

Our first probabilistic programs

It's useful to go through some simple examples of probabilistic programs that feature the return and bind operators, in order to provide a sense of how they work. To start, suppose we have some categorical distribution, $\mathsf{mammal} : \mathrm{P}e$, on mammals. We can represent a distribution on *mammals' mothers* as in (8a)

(8) a. $x \sim \mathsf{mammal}$
$\boxed{\mathsf{mother}(x)}$

b.
```
let' x mammal (
  Return (mother @@ x)
)
```

Here, a random entity $x : e$ is *sampled* from $\mathsf{mammal} : \mathrm{P}e$ using bind, and then $\mathsf{mother}(x) : e$ is returned, as indicated by the orange box. Since return turns things of type α into probabilistic programs of type $\mathrm{P}\alpha$, the resulting probabilistic program is of type $\mathrm{P}e$. Furthermore, assuming that the probability distribution mammal only has support on (i.e., assigns non-zero probability to) the entities which are mammals, the distribution which results will only have support on the entities which are the mothers of entities which are mammals.

(8b) contains the Haskell rendition of the same program (see Figure 10 for definitions of the abbreviations involved). Here we use `let'` in place of the '$\sim$' encoding bind, and `Return` in place of the orange box encoding return.

Making observations

Our probabilistic language also comes with a function observe for *making observations* inside of a program (see Bernardy et al., 2022; Grove & Bernardy, 2023).[9]

[9]We define observe here as a primitive of the language of probabilistic programs (i.e., a constant). In their continuation-based treatment, Grove and Bernardy (2023) implement observe so that it has

```haskell
{-# LANGUAGE LambdaCase #-}

betaNormal :: Term -> Term
betaNormal = \case
  -- variables and constants (already in β-normal form):
  v@(Var _) -> v
  c@(Con _) -> c
  -- abstractions and applications:
  Lam v t   -> Lam v (betaNormal t)
  App t u   -> case betaNormal t of
                 -- β-reduction:
                 Lam v t' -> betaNormal (subst v u t')
                 t'       -> App t' (betaNormal u)
  -- tuples:
  TT  -> TT
  Pair t u  -> Pair (betaNormal t) (betaNormal u)
  Pi1 t     -> case betaNormal t of
                 -- β-reduction
                 Pair x _ -> x
                 t'       -> Pi1 t'
  Pi2 t     -> case betaNormal t of
                 -- β-reduction:
                 Pair _ y -> y
                 t'       -> Pi2 t'
  -- probabilistic programs:
  Return t  -> Return (betaNormal t)
  Let v t u -> case betaNormal t of
                 -- Left identity (β-reduction):
                 Return t'  -> betaNormal (subst v t' u)
                 -- Associativity (commuting conversion):
                 Let w t' x -> betaNormal (
                   Let fr t'
                    (Let v (subst w (Var fr) x) u)
                   )
                   where fr:esh = fresh [u, x]
                 t'         -> Let v t' (betaNormal u)
```

Figure 9 β-reduction in Haskell, with a take on the monad laws.

```
u, v, w, x, y, z :: Term
u = Var "u"
v = Var "v"
w = Var "w"
x = Var "x"
y = Var "y"
z = Var "z"

lam' :: Term -> Term -> Term
lam' (Var n) t = Lam n t

let' :: Term -> Term -> Term -> Term
let' (Var n) t u = Let n t u

(@@), (&) :: Term -> Term -> Term
t @@ u = App t u
t & u  = Pair t u

observe :: Term -> Term -> Term
observe t u = Let "_" (Con (Left "observe") @@ t) u

mammal, mother, dog :: Term
mammal = Con (Left "mammal")
mother = Con (Left "mother")
dog    = Con (Left "dog")

bernoulli :: Double -> Term
bernoulli x = Con (Left "bernoulli") @@ Con (Right x)
```

Figure 10 Some Haskell abbreviations for probabilistic programs.

(9) observe : $t \to \mathrm{P}\diamond$

observe takes a truth value and either keeps or throws out the distribution represented by the expression which follows it, depending on whether this truth value is T or F. For instance, we may instead constrain our "mother" program to describe a distribution over only dogs' mothers.

(10) a. $x \sim$ mammal
observe(dog(x))
mother(x)

b.
```
let' x mammal (
  observe (dog @@ x) (
    Return (mother @@ x)
  )
)
```

This distribution assigns a probability of 0 to any entity which is not the mother of some dog. As for the mothers of dogs, it assigns them a probability mass in proportion to whatever the dogs were assigned by the original mammal program.

Let's look at one more example, (11a), to help pump intuitions a little more. This example encodes a Bernoulli distribution via the constant Bernoulli : $r \to \mathrm{P}t$. The idea is that, e.g., Bernoulli(0.5) encodes a probability distribution over the values T and F, with $True$ being returned with probability 0.5 and F being returned with probability $1 - 0.5 = 0.5$.

(11) a. $x \sim$ Bernoulli(0.5)
observe(x)
(x)

b.
```
let' x (bernoulli 0.5) (
  observe x (
    Return x
  )
)
```

the conditioning behavior described only informally here. One could (if they wanted to) interpret the current system into one that uses continuations, so that observe has the behavior needed.

This program is a little odd. It samples a truth value from $\mathsf{Bernoulli}(0.5)$—which is thus T or F with probability 0.5—and then it *observes* this truth value. As a result, the truth value only makes it past the observe statement in order to be returned if it is T. The entire program may thus be simplified to $\boxed{\mathsf{T}}$.

3.2.2 The common ground

Here, we make good on the assumption, mentioned in Section 2.3, that common grounds amount to probability distributions over indices of some kind. In general, we assume that common grounds are probabilistic programs of type $\mathsf{P}\iota$, for some type ι which we regard as polymorphic. What we require, however, is that given an index $i : \iota$, we may assume the existence of functions such as, e.g., $\mathsf{height} : \iota \to e \to r$ and $\mathsf{d}_{tall} : \iota \to r$, as might be implicated in the meaning of the adjective *tall*.

(12) $[\![tall]\!] = \lambda x, i.\mathsf{height}(x) \geq \mathsf{d}_{tall}(i)$

The idea is that height, when given an index i, gives back a function $\mathsf{height}(i)$ from entities to real numbers representing those entities' heights (at that index). Meanwhile, $\mathsf{d}_{tall}(i)$, a real number, is a degree corresponding to the adjective's standard. Thus the meaning of *tall* is a function of type $e \to \iota \to t$ that takes entities onto *propositions*, i.e., sets of indices of type ι.

More generally, we assume that all semantic values associated with verbs and predicates are sensitive to indices in this way. To implement this assumption instead of the extensional typing scheme introduced in Section 3.1, we assign sentences the intensional type of propositions, as in (13). Adjectives such as *tall* then end up receiving the same type as nouns.

(13) $[\![np]\!] = e$
$[\![n]\!] = e \to \iota \to t$
$[\![ap]\!] = e \to \iota \to t$
$[\![s]\!] = \iota \to t$

Crucially, we assume that functions such as height and d_{tall} come in pairs, as in (14) and (15).

(14) a. $\mathsf{height} : \iota \to e \to r$

b. $\mathsf{upd\_height} : (e \to r) \to \iota \to \iota$

```
{-# LANGUAGE LambdaCase #-}
{-# LANGUAGE PatternSynonyms #-}

type DeltaRule = Term -> Maybe Term

pattern LkUp :: String -> Term -> Term
pattern LkUp c i = Con (Left c) `App` i

pattern Upd :: String -> Term -> Term -> Term
pattern Upd c v i =
  Con (Left ('u' : 'p' : 'd' : '_' : c)) `App` v `App` i

indices :: DeltaRule
indices = \case
  LkUp c (Upd  c' v _)  | c' == c -> Just v
  LkUp c (Upd  c' _ i)  | c' /= c -> Just (LkUp c i)
  Upd  c (LkUp c' i') i | i' == i
                       &&  c' == c -> Just i
  Upd  c v (Upd c' _ i) | c' == c -> Just (Upd c v i)
  _                                -> Nothing
```

Figure 11 δ-rules for indices.

(15) a. $\mathsf{d}_{tall} : \iota \to r$

b. $\mathsf{upd\_d}_{tall} : r \to \iota \to \iota$

Given values $v : e \to r$ and $x : r$, as well as an index $i : \iota$, $\mathsf{upd\_height}(v)(i)$ and $\mathsf{upd\_d}_{tall}(x)(i)$ are both new indices (of type ι) containing values that height and d_{tall} (respectively) may in turn retrieve.

Thus as a result, we expect the constants in (14) and (15) to satisfy the equalities in (16) and (17), respectively.

(16)

$$\mathsf{height}(\mathsf{upd\_height}(v)(i)) = v$$
$$\mathsf{upd\_height}(\mathsf{height}(i))(i) = i$$
$$\mathsf{upd\_height}(v)(\mathsf{upd\_height}(w)(i)) = \mathsf{upd\_height}(v)(i)$$

(17)

$$\mathsf{d}_{tall}(\mathsf{upd\_d}_{tall}(x)(i)) = x$$
$$\mathsf{upd\_d}_{tall}(\mathsf{d}_{tall}(i))(i) = i$$
$$\mathsf{upd\_d}_{tall}(x)(\mathsf{upd\_d}_{tall}(y)(i)) = \mathsf{upd\_d}_{tall}(x)(i)$$

Each of the first equalities ensures that updating an index with a value for the relevant constant and then retrieving a value from that index using that constant results in the value with which the index was updated. Meanwhile, each of the second equalities ensures that superfluous updates have no effect. And each of the third equalities ensures that updating twice results in the second update overwriting the first update.

Furthermore, because upd_height and $\mathsf{upd\_d}_{tall}$ may each update indices which have *already* been updated with the other, we also require the two equalities in (18).

(18)

$$\mathsf{height}(\mathsf{upd\_d}_{tall}(x)(i)) = \mathsf{height}(i)$$
$$\mathsf{d}_{tall}(\mathsf{upd\_height}(v)(i)) = \mathsf{d}_{tall}(i)$$

That is, height and d_{tall} need to be able to *look past* update operations pertaining to other constants; likewise for any other such intensional constants.

Figure 11 implements these ideas in Haskell by introducing the central notion of a δ-rule. The type alias `DeltaRule` is defined as `Term -> Maybe Term`; a δ-rule is thus a function taking terms onto new terms—but crucially, in a way that encodes equalities such as the ones discussed here, by viewing them as *reductions*.

For example, one line of the δ-rule `indices` defined in Figure 11 is

```
LkUp c (Upd c' _ i) | c' /= c -> Just (LkUp c i)
```

Indeed, this line encodes the equalities in (18), which require intensional constants to skip past updates which don't concern them. Throughout the rest of this text, we will make crucial use of these kinds of rules in order to encode the axioms (and, often, theorems) that allow complex probabilistic programs to be reduced to a kind of normal form that can be easily rendered into a probabilistic model. To ensure that such rules take effect, we also generalize our definition of β-reduction so that it renders terms not just into β-normal forms, but into $\beta\delta$-normal forms (see Figure 12). For example, the function

`betaDeltaNormal indices` takes any `Term` into β-normal form, while also ensuring that any occurrences of sub-terms satisfying the left-hand side of the `indices` rule are reduced.

An example common ground

(19a) provides an example of a possible common ground; i.e., a probabilistic program of type $\mathsf{P}\iota$. In words, this common ground encodes a distribution over indices that have been updated with values for the constants height and d_{tall}. The possible values for height can be seen as arising from a distribution over functions of type $e \to r$ which take any given entity onto some height value; this height value itself takes on a normal distribution with mean 160cm and standard deviation 6cm, truncated to positive values to encode that height is positive.[10] Meanwhile, the possible values for the standard threshold d_{tall} take on a normal distribution with mean 175cm and standard deviation 3cm, again truncated to positive values to encode the assumption that heights are positive. (19b) provides the same common ground, encoded in Haskell.

(19) a. $h_{Jo} \sim \mathcal{N}(160, 6)\,\mathsf{T}[0, \infty]$
$\theta \sim \mathcal{N}(175, 3)\,\mathsf{T}[0, \infty]$
$\mathsf{upd\_d}_{tall}(\theta)(\mathsf{upd\_height}(\lambda x.h_{Jo})(@))$

[10] Adopting a convention from the STAN programming language, we use the notation $\mathsf{T}[a, b]$ to indicate truncation to the interval $[a, b]$. Intuitively, truncating a distribution to an interval simply distributes the probability mass that would have been assigned to values outside of this interval over the values inside of this interval.

```haskell
{-# LANGUAGE LambdaCase #-}

betaDeltaNormal :: DeltaRule -> Term -> Term
betaDeltaNormal delta = continue . \case
  -- variables and constants (already in β-normal form):
  v@(Var _) -> v
  c@(Con _) -> c
  -- abstractions and applications:
  Lam v t   -> Lam v (bdnd t)
  App t u   -> case bdnd t of
                 -- β-reduction:
                 Lam v t' -> bdnd (subst v u t')
                 t'       -> App t' (bdnd u)
  TT        -> TT
  Pair t u  -> Pair (bdnd t) (bdnd u)
  Pi1 t     -> case bdnd t of
                 -- β-reduction:
                 Pair x _ -> x
                 t'       -> Pi1 t'
  Pi2 t     -> case bdnd t of
                 -- β-reduction:
                 Pair _ y -> y
                 t'       -> Pi2 t'
  -- probabilistic programs:
  Return t  -> Return (bdnd t)
  Let v t u -> case bdnd t of
                 -- Left identity (β-reduction):
                 Return t'  -> bdnd (subst v t' u)
                 -- Associativity (commuting conversion):
                 Let w t' x -> bdnd (
                   Let fr t'
                    (Let v (subst w (Var fr) x) u)
                   )
                   where fr:esh = fresh [u, x]
                 t'         -> Let v t' (bdnd u)
  where continue, bdnd :: Term -> Term
        -- δ-reduce a term in β-normal form:
        continue t = case fmap bdnd (delta t) of
                       Just t' -> t'
                       Nothing -> t
        bdnd       = betaDeltaNormal delta
```

Figure 12 $\beta\delta$-reduction in Haskell (still encoding the monad laws). Note that δ-rules can only "see" terms that are already in β-normal form. The definition of β-reduction of Figure 9 can now be recovered here as `betaNormal = betaDeltaNormal (\_ -> Nothing)`, i.e., in terms of a δ-rule which does `Nothing`.

b.
```
let' x (truncate (normal 160 6) 0 (1 / 0) ) (
  let' y (truncate (normal 175 3) 0 (1 / 0) ) (
    Return (
      Upd "d_tall" y (
        Upd "height" (lam' z x) _0
      )
    )
  )
)
  where truncate :: Term -> Term -> Term -> Term
        truncate m x y =
          Con (Left "Truncate") @@ (m & x & y)

        normal :: Term -> Term -> Term
        normal x y =
          Con (Left "Normal") @@ (x & y)
```

One aspect of this encoding of a common ground which may appear odd is that while the functions providing values for height are of the correct type $(e \rightarrow r)$, the entity argument of these functions is effectively useless: such functions map entities to a fixed value (albeit one taking on a probability distribution), regardless of what these entities *are*. Indeed, this example is somewhat degenerate, in that it is useful only when there is only one entity under consideration. If we expand our entities-under-consideration to include Bo, as well as Jo, we may incorporate another probability distribution representing Bo's height, as in (20).

(20) $h_{Jo} \sim \mathcal{N}(160, 6)\,\mathsf{T}[0, \infty]$
$h_{Bo} \sim \mathcal{N}(173, 7)\,\mathsf{T}[0, \infty]$
$\theta \sim \mathcal{N}(175, 3)\,\mathsf{T}[0, \infty]$
$\boxed{\mathsf{upd\_d}_{tall}(\theta)(\mathsf{upd\_height}(\lambda x.\mathsf{if\_then\_else}(x = \mathsf{j}, h_{Jo}, h_{Bo}))(@))}$

In this expanded common ground, the distribution over values for height is effectively a (truncated) multivariate normal distribution: one dimension encodes uncertainty about Jo's height, while the other encodes uncertainty about Bo's height. Once applied to either j or b, the function providing the meaning of height will give back either h_{Jo} or h_{Bo}, respectively, once the *if then else* statement is evaluated. Importantly, the functional type of height, in this case, is no longer useless. Generally, such a function will encode n branches in a

common ground keeping track of the heights of n entities (thus requiring, say, $n - 1$ *if then else* statements).

3.2.3 Expressions and discourses

Discourse states constitute another central ingredient of the framework. We follow a tradition in formal pragmatics that regards discourse states as consisting of collections of *metalinguistic parameters*; these may include, e.g., the common ground and the QUD, along with other conversationally relevant features of discourse (e.g., representations of the entities to which pronouns can refer, the available antecedents for ellipsis, etc.). One can view a discourse state as akin to the context state of Farkas and Bruce (2010), though the types of parameters are for current purposes open ended.

Meanwhile, we regard an expression's probabilistic semantic value as a function of type $\sigma \rightarrow \mathrm{P}(\alpha \times \sigma')$, where σ and σ' are types and α is the expression's ordinary semantic type. We abbreviate such types as '$\mathbb{P}^{\sigma}_{\sigma'}\alpha$'. Thus given an input state $s : \sigma$, the semantic value of an expression produces a probability distribution over pairs of ordinary semantic values of type α and possible output states $s' : \sigma'$. An expression of category np, for instance, now has the *probabilistic* type $\mathbb{P}^{\sigma}_{\sigma'}(\llbracket np \rrbracket) = \mathbb{P}^{\sigma}_{\sigma'}e$.

Building on this view of expressions, we regard an *ongoing discourse* as a function of type $\mathbb{P}^{\sigma}_{\sigma'}\diamond$. The effects that both expressions and discourses have are thus *stateful-probabilistic*: they map input states to probability distributions over output states. Discourses differ from expressions, however, in that the value discourses compute is trivial: it is invariably the empty tuple $\diamond$, as dictated by its type. Hence, while expressions produce both stateful-probabilistic effects *and* values, discourses have *only* effects, i.e., they merely update the state.

3.2.4 Semantic composition

The setup we have introduced allows for the possibility that the state parameter σ *changes* in the course of evaluating an expression's probabilistic semantic value. Such a value may map an input state $s : \sigma$ onto a probability distribution over outputs states of type σ' (where $\sigma' \neq \sigma$). This flexibility is needed in order to capture the changing nature of certain components of the discourse state. For instance, the QUD stack may have arbitrarily many questions on it, and these questions may themselves have different types: they may be degree questions, individual questions, etc. Thus whenever an utterance functions to add a question to the QUD stack, the input state's type will not match the output

Left identity

$$\mathsf{do}_{p,p,q}\, x \leftarrow \boxed{v}_p \\ \quad k \quad = \quad k[v/x]$$

Right identity

$$\mathsf{do}_{p,q,q}\, x \leftarrow m \\ \quad \boxed{x}_q \quad = \quad m$$

Associativity

$$\mathsf{do}_{p,r,s}\, y \leftarrow \left(\begin{array}{c}\mathsf{do}_{p,q,r}\, x \leftarrow m \\ n\end{array}\right) \\ \quad o \quad = \quad \mathsf{do}_{p,q,s}\, x \leftarrow m \\ \qquad\qquad \left(\begin{array}{c}\mathsf{do}_{q,r,s}\, y \leftarrow n \\ o\end{array}\right)$$

Figure 13 The parameterized monad laws.

state's type.

To countenance this type-level flexibility, we view the types $\mathbb{P}^{\sigma}_{\sigma'}\alpha$ as arising from a *parameterized* monad, where the parameters are the types of possible states.[11] Parameterized monads are associated with their own definitions of (parameterized) return and bind. To increase clarity, while distinguishing the notations for parameterized and vanilla monads, we present the bind statements of a parameterized monad $\mathbb{P}$ using Haskell's `do`-notation.

(21) Given a collection $\mathcal{S}$ of parameters, a parameterized monad is a map $\mathbb{P}$ from triples consisting of two parameters and a type onto types (i.e., given parameters $p, q \in \mathcal{S}$ and a type α, $\mathbb{P}^p_q\alpha$ is some new type), equipped with two operators satisfying the parameterized monad laws (Figure 13).

$$\boxed{(\cdot)}_p \quad : \quad \alpha \to \mathbb{P}^p_p\alpha \qquad \text{('return')}$$

$$\mathsf{do}_{p,q,r}\, x \leftarrow (\underline{\cdot_1}) \\ \quad (\underline{\cdot_2})(x) \quad : \quad \mathbb{P}^p_q\alpha \to (\alpha \to \mathbb{P}^q_r\beta) \to \mathbb{P}^p_r\beta \qquad \text{('bind')}$$

The particular parameterized monad we employ is defined in (22), where the relevant collection of parameters is just the set of semantic types. Thus we don't constraint the types of states from the outset; they may in principle be *anything*.

[11] See Atkey (2009) on the parameterized State monad and parameterized monads more generally. The current parameterized monad can be viewed as applying a parameterized State monad *transformer* to the underlying probability monad `P`; see Liang, Hudak, and Jones (1995) on monad transformers.

(22)

$$\begin{aligned}
\mathbb{P}^{\sigma}_{\sigma'}\alpha &= \sigma \to \mathsf{P}(\alpha \times \sigma') \\
\boxed{v}_{\sigma} &= \lambda s.\boxed{\langle v, s\rangle} \\
\mathsf{do}_{\sigma,\sigma',\sigma''}\, x \leftarrow m \\ \quad k(x) &= \lambda s.\begin{pmatrix}\langle x, s'\rangle \sim m(s) \\ k(x)(s')\end{pmatrix}
\end{aligned}$$

The do-notation in these definitions should be read as saying, "first bind the variable x to the program m, and then do $k(x)$". Specifically, the return operator takes any value onto a program which reads in a state and then gives back a *degenerate* probability distribution over that value paired up with the state it read in. It therefore provides a stateful analog of what the vanilla probabilistic return operator does—only now, it also manages state-threading in addition to ordinary values. The bind operator, meanwhile, sequences a stateful-probabilistic program m with a continuation k whose result is stateful-probabilistic. It does this by reading in a state, applying m to it and sampling a pair of a value and new state, and then feeding each to k in turn.

We follow a couple of simplifying notational conventions throughout this text. First, we will generally leave the parameters annotating operators implicit, writing '$\boxed{x}$' instead of '$\boxed{x}_p$' and 'do' instead of '$\mathsf{do}_{p,q,r}$'. Second, we will follow Haskell's convention of representing multiple uses of bind in a row

$$\begin{aligned}
&\mathsf{do}\, x \leftarrow m \\
&\quad \mathsf{do}\, y \leftarrow n \\
&\qquad k(x, y)
\end{aligned}$$

by consolidating them into a single do-block,

$$\begin{aligned}
&\mathsf{do}\, x \leftarrow m \\
&\quad\; y \leftarrow n \\
&\quad\; k(x, y)
\end{aligned}$$

or by separating them on a single line by a semicolon to save space.

$$\mathsf{do}\,\{\, x \leftarrow m;\; y \leftarrow n;\; k(x, y)\,\}$$

The updated probabilistic, dynamic CCG rule schemata are provided in Figure 14. These schemata mimic the ones defined in Figure 5 of Section 3.1, except that semantic values are considered to be of type $\mathbb{P}^{\sigma}_{\sigma'}\alpha$ (for some two state types σ and σ') now, rather than of type α. Thus rather than apply the CCG operations to semantic values directly—as we did in Section 3.1—we must bind these semantic values to variables of type α, using our do-notation, and apply

the operations to the bound variables. Meanwhile, the type homomorphism from syntactic types to semantic types is notably no longer present, since the probabilistic dynamic type associated with a given expression is related to its ordinary semantic type by allowing the parameters σ and σ' to be potentially arbitrary state types.[12] The upshot is that an expression's syntactic type continues to determine its ordinary semantic type, though its probabilistic dynamic effects are independent.

3.2.5 Lexical knowledge

Using these tools, we can model different forms of uncertainty as arising from the probabilistic knowledge we have about particular lexical items.

Consider again the sentence in (2).

(2) Jo ran a race.

We may analyze *Jo* as having the lexical entry in (23), where j is a constant of type e.

(23) $\boxed{\begin{array}{l}\textit{Jo}\\ \texttt{do background}_{Jo}\\ \quad\boxed{\texttt{j}}\end{array}} \vdash np$

[12] We can reconstruct something like a type homomorphism by defining $[\![\cdot]\!]^{\mathbb{P}} : \texttt{Cat} \to 2^{\mathcal{T}}$ to take syntactic types onto equivalence classes of semantic types (rather than types per se). Then we would have, for example, $[\![np]\!]^{\mathbb{P}} = \{\mathbb{P}^{\sigma}_{\sigma'} e \mid \sigma, \sigma' \in \mathcal{T}\}$. Further, in general, we can define:

$$\begin{aligned}
[\![a]\!]^{\mathbb{P}} &= \{\mathbb{P}^{\sigma}_{\sigma'}[\![a]\!] \mid \sigma, \sigma' \in \mathcal{T}\} && \text{(if } a \text{ is a base category)}\\
[\![c/b]\!]^{\mathbb{P}} &= [\![c\backslash b]\!]^{\mathbb{P}}\\
&= [\![b]\!]^{\mathbb{P}} \Rightarrow [\![c]\!]^{\mathbb{P}}\\
&\stackrel{\text{def}}{=} \{\mathbb{P}^{\sigma}_{\sigma'}(x \to y) \mid \exists \sigma'' \in \mathcal{T} : \mathbb{P}^{\sigma}_{\sigma''} x \in [\![b]\!]^{\mathbb{P}} \wedge \mathbb{P}^{\sigma''}_{\sigma'} y \in [\![c]\!]^{\mathbb{P}}\}
\end{aligned}$$

Thus for example:

$$\begin{aligned}
[\![np]\!]^{\mathbb{P}} &= \{\mathbb{P}^{\sigma}_{\sigma'} e \mid \sigma, \sigma' \in \mathcal{T}\}\\
[\![np/n]\!]^{\mathbb{P}} &= [\![n]\!]^{\mathbb{P}} \Rightarrow [\![np]\!]^{\mathbb{P}}\\
&= \{\mathbb{P}^{\sigma}_{\sigma'}(x \to y) \mid \exists \sigma'' \in \mathcal{T} : \mathbb{P}^{\sigma}_{\sigma''} x \in [\![n]\!]^{\mathbb{P}} \wedge \mathbb{P}^{\sigma''}_{\sigma'} y \in [\![np]\!]^{\mathbb{P}}\}\\
&= \{\mathbb{P}^{\sigma}_{\sigma'}(e \to \iota \to t) \mid \sigma, \sigma' \in \mathcal{T}\}
\end{aligned}$$

$$\frac{\overset{s_1}{\boxed{M_1}} \vdash c/b \qquad \overset{s_2}{\boxed{M_2}} \vdash b \mid_n a_n \cdots \mid_1 a_1}{\overset{s_1\ s_2}{\boxed{\begin{array}{l}\mathsf{do}\, m_1 \leftarrow M_1; m_2 \leftarrow M_2 \\ \quad \boxed{\lambda x_1, \ldots, x_n.m_1(m_2(x_1)\cdots(x_n))}\end{array}}} \vdash c \mid_n a_n \cdots \mid_1 a_1}\ >\mathbf{B}_n$$

$$\frac{\overset{s_1}{\boxed{m_1}} \vdash b \mid_n a_n \cdots \mid_1 a_1 \qquad \overset{s_1}{\boxed{m_2}} \vdash c\backslash b}{\overset{s_1\ s_2}{\boxed{\begin{array}{l}\mathsf{do}\, m_1 \leftarrow M_1; m_2 \leftarrow M_2 \\ \quad \boxed{\lambda x_1, \ldots, x_n.m_2(m_1(x_1)\cdots(x_n))}\end{array}}} \vdash c \mid_n a_n \cdots \mid_1 a_1}\ <\mathbf{B}_n$$

$$\frac{\overset{s_1}{\boxed{M_1}} \vdash b \mid_n a_n \cdots \mid_1 a_1 \qquad \overset{s_2}{\boxed{M_2}} \vdash c\backslash b \mid_n a_n \cdots \mid_1 a_1 \cdots \mid_1 a_1}{\overset{s_1\ s_2}{\boxed{\begin{array}{l}\mathsf{do}\, m_1 \leftarrow M_1; m_2 \leftarrow M_2 \\ \quad \boxed{\lambda x_1, \ldots, x_n.m_1(x_1)\cdots(x_n)(m_2(x_1)\cdots(x_n))}\end{array}}} \vdash c \mid_n a_n \cdots \mid_1 a_1}\ <\mathbf{S}_n$$

$$\frac{\overset{s_1}{\boxed{M_1}} \vdash c/b \mid_n a_n \cdots \mid_1 a_1 \qquad \overset{s_2}{\boxed{M_2}} \vdash b \mid_n a_n \cdots \mid_1 a_1}{\overset{s_1\ s_2}{\boxed{\begin{array}{l}\mathsf{do}\, m_1 \leftarrow M_1; m_2 \leftarrow M_2 \\ \quad \boxed{\lambda x_1, \ldots, x_n.m_2(x_1)\cdots(x_n)(m_1(x_1)\cdots(x_n))}\end{array}}} \vdash c \mid_n a_n \cdots \mid_1 a_1}\ >\mathbf{S}_n$$

Figure 14 Probabilistic CCG rule schemata.

A couple of important pieces of notation are used here. First, $\texttt{background}_{Jo}$ is a program of type $\mathbb{P}^{\sigma}_{\sigma'}\diamond$ which encodes our background knowledge about the lexical item *Jo*. This knowledge may be quite wide ranging. For instance, it might update an aspect of the state which encodes a phonetic representation of the utterances made in a discourse so far; or, it might add information to the common ground useful for later anaphora, i.e., that Jo is animate.

To flesh these ideas out, it will be useful to have operations that allow us to (probabilisticaly) read and write the values of stateful parameters. Given some state $s : \sigma$, we may wish to view one of its components in the course of defining a program. For example, we may define a constant $\mathsf{CG} : \sigma \to \mathsf{P}\iota$ that represents grabbing the common ground from some state; thus $\mathsf{CG}(s) : \mathsf{P}\iota$ is the common ground of the state s. Then to view the current common ground in some ongoing probabilistic program, we may define the following shorthand:

$$\texttt{view}_{\mathsf{CG}} : \mathbb{P}^{\sigma}_{\sigma}(\mathsf{P}\iota)$$
$$\texttt{view}_{\mathsf{CG}} = \lambda s.\boxed{\langle \mathsf{CG}(s), s\rangle}$$

We can then define programs that involve viewing the common ground at arbitrary points; schematically:

$$\begin{array}{l}\texttt{do}\cdots\\ \quad cg \leftarrow \texttt{view}_{\mathsf{CG}}\\ \quad\cdots\end{array}$$

In turn, we may wish to set a *new* common ground which modifies the old one in some way. Let us then introduce a constant $\mathsf{upd\_CG} : \mathsf{P}\iota \to \sigma \to \sigma$ which, given a state and a common ground, produces a new state that represents the act of *overwriting* the state's common ground with a new one. To do this inside of an ongoing probabilistic program, we define a corresponding shorthand, $\texttt{set}_{\mathsf{CG}}$, analogous to $\texttt{view}_{\mathsf{CG}}$:

$$\texttt{set}_{\mathsf{CG}} : \mathsf{P}\iota \to \mathbb{P}^{\sigma}_{\sigma}\diamond$$
$$\texttt{set}_{\mathsf{CG}} = \lambda cg, s.\boxed{\langle \diamond, \mathsf{upd\_cg}(s)\rangle}$$

For example, we can say a little more now about the program $\texttt{background}_{Jo}$ that we had used in defining the lexical entry for *Jo*. Let's say, for example, that part of the meaning of this lexical item is to update the common ground with the information that Jo is animate. Then we might fill out this meaning as in (24).

(24)
$$\overset{Jo}{\boxed{\begin{array}{l}\mathsf{do}\ cg \leftarrow \mathsf{view}_{\mathsf{CG}} \\ \quad \mathsf{set}_{\mathsf{CG}} \left(\begin{array}{l} i \sim cg \\ \mathsf{observe}(\mathsf{animate}(i)(\mathsf{j})) \\ \textcircled{i} \end{array} \right) \\ \quad \boxed{\mathsf{j}} \end{array}}} \vdash np$$

According to this lexical entry, the probabilistic dynamic meaning of *Jo* is to update the common ground with the proposition according to which Jo is animate and then to return j. Such a meaning can be seen as assigning *Jo* something like a conventional implicature: in addition to returning the entity which is the ordinary denotation of the name, it ensures an update to the common ground—one which isn't piped through an assertion, but arises from the lexical meaning of the name.

We can implement this rendition of lexical meaning, on which expressions of category c are mapped to semantic types $\mathbb{P}^{\sigma}_{\sigma'}[\![c]\!]$, in Haskell by first providing definitions of the parameterized monad operators. Return and bind can be implemented as functions that manipulate λ-terms:

```
-- return:
purePP :: Term -> Term
purePP t = lam fr (Return (t & fr))
  where fr:esh = map var $ fresh [t]

-- bind:
(>>>=) :: Term -> Term -> Term
t >>>= u = lam fr (let' e (t @@ fr) (u @@ Pi1 e @@ Pi2 e))
  where fr:e:sh = map Var $ fresh [t, u]
```

These definitions follow (22), though the presentation of bind appears somewhat different. For example we refer to the return operator as `purePP` to indicate that it produces a pure probabilistic program; i.e., one without probabilistic or stateful effects. In general, a bind statement such as $\begin{array}{l}\mathsf{do}\ x \leftarrow m \\ k\end{array}$ will be rendered in Haskell as `m >>>= lam x k`.

Definitions of `view` and `set` are also forthcoming:

```
view, set :: String -> Term
view c = lam s (Return ((SCon c @@ s) & s))
set  c = lam x (lam s (Return (TT & Upd c x s)))
```

These definitions rely on some `PatternSynonym` abbreviations; specifically:

```
pattern SCon :: String -> Term
pattern SCon x = Con (Left x)

pattern Upd :: String -> Term -> Term -> Term
pattern Upd c v s = SCon ('u' : 'p' : 'd' : '_' : c)
                      `App` v `App` s
```

Further, to sequence instances of `set` —whose value type is `Unit` —with other programs in Haskell, we can define a variant of bind that ignores the bound variable:

```
-- sequence:
(>>>) :: Term -> Term -> Term
m >>> n = m >>>= (lam fr n)
  where fr:esh = map Var $ fresh [n]
```

Thus $\begin{matrix}\text{do}\, m \\ k\end{matrix}$ can now be rendered in Haskell as `m >>> k`.

Lexica themselves can be defined in Haskell as functions from *words* to *meanings*. If we represent words as strings (and expressions, more generally, as lists of words),

```
type Word = String
type Expr = [Word]
```

then we may define a type synonym for lexica:

```
type Lexicon m = Word -> [m]
```

Thus given a type of lexical representations `m`, `Lexicon m` maps words to *lists* of values of type `m`, thus providing a representation of potential lexical ambiguity. In this text, we take `m` to encode (effectively) a pair of a syntactic category and a meaning, i.e., a typed λ-term:

```
data SynSem = SynSem { syn :: Cat, sem :: Typed }
  deriving (Eq, Show)
```

To encode the lexical entry in (24), for example, we can define a lexicon consisting of just one entry (note here the use of Haskell's `LambdaCase` language extension).

(25)
```
lexJo :: Lexicon SynSem
lexJo = \case
  "Jo" -> [ SynSem {
              syn = np,
              sem =
                view "CG" >>>= lam c (
                  set "CG" @@ (
                    let' i c (
                      observe (animate @@ i @@ jo) (
                        Return i
                      )
                    )
                  ) >>>
                  purePP jo
                )
              } ]
  _    -> []
```

Two new terms are used— `animate` and `jo`; these terms are defined as follows:

```
animate, jo :: Term
animate = SCon "animate"
jo      = SCon "j"
```

These sorts of lexica exemplify how we represent lexical knowledge in PDS. Important to note is that not all lexical knowledge, as we define it here, need pertain to linguistic properties per se; lexical background knowledge may contain whatever information about a lexical item might be relevant to the trajectory of an interpretation as it unfolds. Further, lexical knowledge may modify any aspect of the discourse state. We've illustrated a case in which it modifies the common ground, but it could just as well modify the QUD.[13]

Now that we've illustrated our basic interpretation scheme, we can show how to integrate sentence interpretations into full discourses. Here, we define illocutionary operators that map expressions' semantic values onto discourses

[13]For example, this might be required by certain approaches to projective inferences (Simons, 2007; Simons, Beaver, Roberts, & Tonhauser, 2017; Simons, Tonhauser, Beaver, & Roberts, 2010; Tonhauser, Beaver, & Degen, 2018, i.a.).

consisting of various utterance acts. We consider two such operators: `assert` and `ask`.

Making assertions

Given a sentence whose probabilistic dynamic semantic value is $m : \mathbb{P}^{\sigma}_{\sigma'}(\iota \to t)$—i.e., a probabilistic dynamic *proposition*—we may use `assert` to map m to a discourse of type $\mathbb{P}^{\sigma}_{\sigma'}\diamond$.

(26)

$$\begin{aligned} \texttt{assert} \;&:\; \mathbb{P}^{\sigma}_{\sigma'}(\iota \to t) \to \mathbb{P}^{\sigma}_{\sigma'}\diamond \\ \texttt{assert}(m) \;&=\; \texttt{do}\, \phi \leftarrow m \\ &\qquad\quad cg \leftarrow \texttt{view}_{\mathsf{CG}} \\ &\qquad\quad \texttt{set}_{\mathsf{CG}} \begin{pmatrix} i \sim cg \\ \texttt{observe}(\phi(i)) \\ \textcircled{i} \end{pmatrix} \end{aligned}$$

This definition has a few components. In order: (i) it binds the PDS value m to a proposition ϕ of type $\iota \to t$; (ii) it views the current common ground, as cg; (iii) it sets a new common ground. In particular, it a sets a new common ground which is obtained from cg by: (i) sampling an index i from cg; (ii) observing at that index that the proposition ϕ is true; and (iii) returning i. In other words, the new common ground is obtained from cg by filtering out indices at which ϕ is false, otherwise leaving it untouched. Thus to make an assertion is to update the common ground (Stalnaker, 1978).

The Haskell definition of `assert` can follow (26) directly.

(27)

```
assert :: Term
assert = lam m (
           m >>>= lam p (
             view "CG" >>>= lam c (
               set "CG" @@ (
                 let' i c (
                   observe (p @@ i) (
                     Return i
                   ) ) ) ) ) )
```

For example, let us consider (2) from Section 2.

(2) Jo ran a race.

First, we define a few functions on terms to be featured in the meanings of *ran*, *a*, and *race*.

```
runLoc, runOrg, ite :: Term -> Term -> Term -> Term
runLoc i x y = SCon "run_loc" @@ i @@ x @@ y
runOrg i x y = SCon "run_org" @@ i @@ x @@ y
ite    b x y = SCon "if_then_else" @@ (b & x & y)

and, race :: Term -> Term -> Term
and  x y  = SCon "&" @@ x @@ y
race i x  = SCon "race" @@ i @@ x

ex :: Term -> Term
ex f = SCon "∃" @@ f
```

Indeed, we must also provide the constants featured in these meanings with types, in terms of a signature:

```
joRanSig :: Sig
joRanSig = \case
  Left "run_loc"      -> Just (e :-> e :-> t)
  Left "run_org"      -> Just (e :-> e :-> t)
  Left "if_then_else" -> Just (t :* α :* α)
  Left "&"            -> Just (t :-> t :-> t)
  Left "race"         -> Just (ι :-> e :-> t)
  Left "∃"            -> Just ((α :-> t) : -> t)
  _                   -> Nothing
```

We can then define the lexical entries in (28) to encode syntactic categories and meanings for these expressions. Here we provide both the mathematical notation and its Haskell implementation.

(28) a. *ran*

$$\boxed{\begin{array}{l}\mathsf{do}\ b \leftarrow \mathtt{view}_{\tau_{ran}} \\ \quad \boxed{\begin{array}{l}\lambda x, y, i.\mathsf{if\_then\_else}(\\ \quad b, \\ \quad \mathsf{run}_{loc.}(i)(x)(y), \\ \quad \mathsf{run}_{org.}(i)(x)(y))\end{array}}\end{array}} \vdash s\backslash np/np$$

```
lexRan :: Lexicon SynSem
lexRan = \case
  "ran" -> [ SynSem {
               syn = s \\ np // np,
               sem = ty joRanSig $
                 view "tau_run" >>>= lam x (
                   purePP (
                     lam y (lam z (lam i (ite x
                       (runLoc i y z)
                       (runOrg i y z)
                     ) ) ) )
                 )
               } ]
  _     -> []
```

b. $\overset{a}{\boxed{\lambda p, q, y, i.\exists x : p(x)(i) \wedge q(x)(y)(i)}} \vdash s\backslash np/(s\backslash np/np)/n$

```
lexA :: Lexicon SynSem
lexA = \case
  "a" -> [ SynSem {
             syn =
               (s \\ np) // ((s \\ np) // np)
                 // n,
             sem = ty joRanSig $
               purePP (
                 lam x (lam y (lam z (lam i (
                   ex (lam w (
                     and
                       (x @@ w @@ i)
                       (y @@ w @@ x @@ i)
                   ) )
                 ) ) ) )
             } ]
  _   -> []
```

c. $\overset{race}{\boxed{\lambda x, i.\mathsf{race}(i)(x)}} \vdash s\backslash np/np$

```
lexRace :: Lexicon SynSem
lexRace = \case
  "race" -> [ SynSem {
                syn = s // np
                sem = ty joRanSig $
                  purePP (
                      lam x (lam i (
                        race i x
                      ) ) )
              } ]
  _      -> []
```

We may obtain a lexicon for the whole sentence *Jo ran a race* by defining:

```
lexJo <||> lexRan <||> lexA <||> lexRace
```

Here, `(<||>)` is defined by using Haskell's `Alternative` class:

```
(<||>) :: Alternative m
       => (a -> m b) -> (a -> m b) -> a -> m b
f <||> g = \x -> f x <|> g x
```

Thus in general, the meaning of `(<||>)` will depend on the type of data it is applied to. In the case of lists, as above, it has the same result as list concatenation; i.e., `f <||> g = \x -> f x ++ g x`. We will see other uses of this function in what follows, where it is applied to functions ending in `Maybe` types, rather than list types.

A derivation of (2) is given in Figure 15 in terms of the rule schemata of Figure 14 (note that this derivation only ever uses functional application). To save space, we abbreviate the meanings of lexical items, e.g., defining `jo` as:

$$\texttt{jo} \stackrel{\text{def}}{=} \begin{array}{l} \texttt{do}\, cg \leftarrow \texttt{view}_{\texttt{CG}} \\ \quad \texttt{set}_{\texttt{CG}} \left(\begin{array}{l} i \sim cg \\ \texttt{observe}(\texttt{animate}(i)(\texttt{j})) \\ \boxed{i} \end{array} \right) \\ \boxed{\texttt{j}} \end{array}$$

An *assertion* of (2) can now be seen to give rise to the value in (29).

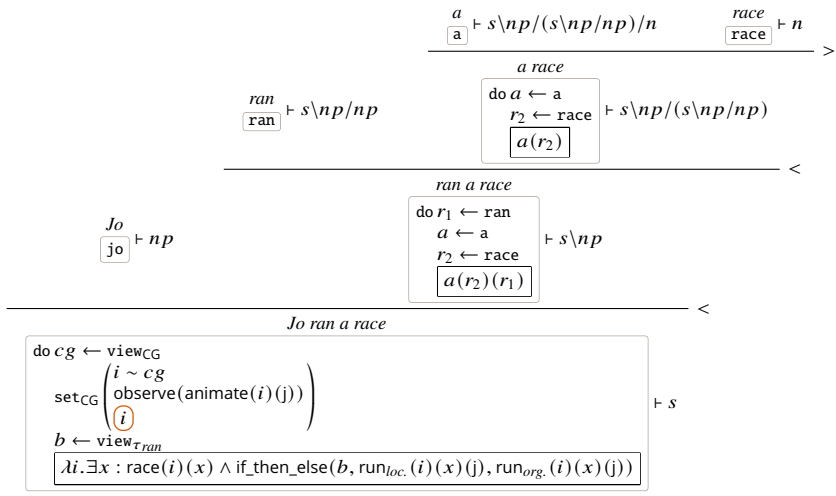


Figure 15 Derivation of *Jo ran a race*.

(29)
$$\begin{array}{l}\mathsf{do}\ cg \leftarrow \mathtt{view}_{\mathsf{CG}}\\ \quad b \leftarrow \mathtt{view}_{\tau_{ran}}\\ \quad \mathtt{set}_{\mathsf{CG}}\left(\begin{array}{l} i \sim cg\\ \mathsf{observe}(\mathsf{animate}(i)(\mathsf{j}))\\ \mathsf{observe}(\exists x : \mathsf{race}(i)(x)\\ \qquad \wedge\ \mathsf{if\_then\_else}(b, \mathsf{run}_{loc.}(i)(x)(\mathsf{j}), \mathsf{run}_{org.}(i)(x)(\mathsf{j})))\\ i\end{array}\right)\end{array}$$

Asking questions

To model asking a question, we follow a categorial tradition that analyzes questions as denoting what amount to sets of true short answer meanings (R. Hausser and Zaefferer (1978); R. R. Hausser (1983); Xiang (2021); cf. J. Groenendijk and Stokhof (1984); Hamblin (1973); Karttunen (1977)). A question meaning is thus of type $\alpha \rightarrow \iota \rightarrow t$ (for some α), where α is the type of the question's short answers.

To provide a syntactic analysis of questions, it's useful to include among our base categories special ones that represent the type of the short answer a particular question requires; e.g., $q_{ind.}$ for individual questions, and $q_{deg.}$ for degree questions. The semantic types assigned to these categories should reflect

these semantic preferences.

(30) $[\![q_{ind.}]\!] = e \to \iota \to t$
$[\![q_{deg.}]\!] = r \to \iota \to t$

Thus degree questions require short answers that have degree (i.e., real number) answers, while individual questions require entity answers.

Our goal is now to analyze the speech act of asking a question, in a way analogous to what we did for assertions. Given an expression whose semantic value is a probabilistic dynamic question meaning

$$m : \mathbb{P}^{\sigma}_{\sigma'}(\alpha \to \iota \to t)$$

we introduce a function $\mathtt{ask}$ that maps m onto a discourse of type $\mathbb{P}^{\sigma}_{\mathtt{Q}\iota\alpha\sigma'}\diamond$. This discourse has the effect of pushing the question onto the stack (Farkas & Bruce, 2010; Ginzburg, 1996).

(31)

$$\begin{aligned} \mathtt{ask} \quad &: \quad \mathbb{P}^{\sigma}_{\sigma'}(\alpha \to \iota \to t) \to \mathbb{P}^{\sigma}_{\mathtt{Q}\iota\alpha\sigma'}\diamond \\ \mathtt{ask}(m) \quad &= \quad \mathtt{do}\, q \leftarrow m \\ &\qquad\quad \mathtt{push}_{\mathsf{QUD}}(q) \end{aligned}$$

In the definition of $\mathtt{ask}(m)$, $q : \alpha \to \iota \to t$ is the question meaning bound to m. $\mathtt{push}_{\mathsf{QUD}}$, further, has the effect of pushing q onto the current state's stack of questions under discussion, and its type is $(\alpha \to \iota \to t) \to \sigma \to \mathtt{Q}\iota\alpha\sigma$. The effect of asking a question is thus to push a new semantic value of type $\alpha \to \iota \to t$—fashioned out of the dynamic semantic value returned by m—onto the QUD stack.

The type-level operator $\mathtt{Q}\iota\alpha\sigma'$ serves to register within the types the fact that a new question meaning with answer type α and index type ι has been pushed onto the stack. A fully general implementation of such operators can be encoded in Haskell, as well (see Figure 16). In the implementation, the type $\mathtt{Q}\iota\alpha\sigma'$ is encoded as `TyCon "Q" [TyVar "ι", TyVar "α", TyVar "σ'"]`.

Here, we say a little more about the program $\mathtt{push}_{\mathsf{QUD}}$. Importantly, we assume the existence of the following two constants in (32) for pushing and popping questions to and from the QUD stack, respectively.

```
data Type = Atom String
          | Type :-> Type
          | Unit
          | Type :* Type
          | P Type
          | TyCon String [Type]
          | TyVar String
  deriving (Eq)
```

Figure 16 The full system of types.

(32) a. $\mathsf{push\_QUD} : (\alpha \to \iota \to t) \times \sigma \to \mathsf{Q}\iota\alpha\sigma$

b. $\mathsf{pop\_QUD} : \sigma \to (\alpha \to \iota \to t) \times \mathsf{popQ}\iota\alpha\sigma$

Further, these constants are expected to interact in the appropriate ways, at both the term level and the type level. At the term level, we have equalities like the one in (33).

(33)

$$\mathsf{pop\_QUD}(\mathsf{push\_QUD}(q, s)) = \langle q, s \rangle$$

Meanwhile, at the type level, we require matching equalities like the one in (34).

(34)

$$\mathsf{popQ}\iota\alpha(\mathsf{Q}\iota\alpha\sigma) = \sigma$$

Indeed, such type equalities ensure that the term equalities are well typed, in the sense that the type of the term on the left-hand side of the equality is the same as the type of the term on the right-hand side.

In terms of the push_QUD and pop_QUD constants, the probabilistic programs $\mathsf{push}_{\mathsf{QUD}}$ and $\mathsf{pop}_{\mathsf{QUD}}$ can be defined:

$$\text{push}_{\text{QUD}} : (\alpha \to \iota \to t) \to \mathbb{P}^{\sigma}_{\mathbb{Q}\iota\alpha\sigma}{}^{\diamond}$$
$$\text{push}_{\text{QUD}}(q) = \lambda s.\boxed{\langle \diamond, \text{push\_QUD}(q, s)\rangle}$$

$$\text{pop}_{\text{QUD}} : \mathbb{P}^{\sigma}_{\text{pop}\mathbb{Q}\iota\alpha\sigma}(\alpha \to \iota \to t)$$
$$\text{pop}_{\text{QUD}} = \lambda s.\boxed{\text{pop\_QUD}(s)}$$

General definitions of these constants—analogous to the `view` and `set` functions—can be encoded in Haskell, as well:

```
push, pop :: String -> Term
push c = lam x (lam s (Return (TT & Push c x s)))
pop  c = lam s (Return (Pop c s))
```

Note that for these definitions to compile, we need the following `PatternSynonyms`:

```
pattern Push :: String -> Term -> Term -> Term
Push c x s = SCon ('p' : 'u' : 's' : 'h' : '_' : c)
               `App` c `App` x `App` s

pattern Pop :: String -> Term -> Term
Pop c s = SCon ('p' : 'o' : 'p' : '_' : c) `App` s
```

This way, pop_{QUD} may be rendered as `pop "QUD"`, and push_{QUD}, as `push "QUD"`.

We can give an example of how to use `ask` using an interpretation for the degree question in (35).

(35) What race did Jo run?

Let's assign the *wh*-word *what*, the auxiliary *did*, and the non-finite verb *run* the lexical entries in (36).

(36) a. $\overset{\textit{what}}{\boxed{\lambda p, q, x, i.p(x)(i) \wedge q(x)(i)}} \vdash q_{ind.}/(s/np)/n$

b. $\overset{\textit{did}}{\boxed{\lambda p.p}} \vdash s/s_{nf}$

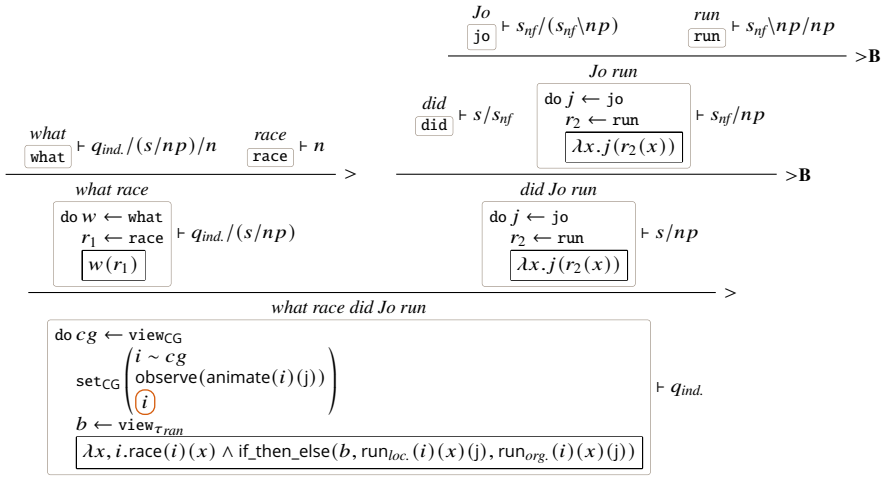


Figure 17 Derivation of *what race did Jo run?*.

c. *run*

do $b \leftarrow \text{view}_{\tau_{run}}$
$\lambda x, y, i.\text{if\_then\_else}($
b,
$\text{run}_{loc.}(i)(x)(y)$,
$\text{run}_{org.}(i)(x)(y))$
⊢ $s_{nf}\backslash np/np$

Then in the derivation, which is given in Figure 17, we assume that *Jo* has the quantifier lexical entry in (37), where it occurs as the subject of a non-finite clause.

(37) *Jo*

do $cg \leftarrow \text{view}_{CG}$
set_{CG} ($i \sim cg$, observe(animate(i)(j)), i)
$\lambda k.k(\text{j})$
⊢ $s_{nf}/(s_{nf}\backslash np)$

If we plug the resulting question meaning into ask, we get the result in (38).

(38) $\texttt{do}\, cg \leftarrow \texttt{view}_{\mathsf{CG}}$
$\texttt{set}_{\mathsf{CG}}\left(\begin{array}{l} i \sim cg \\ \mathsf{observe}(\mathsf{animate}(i)(\mathrm{j})) \\ \boxed{i} \end{array}\right)$
$b \leftarrow \texttt{view}_{\tau_{ran}}$
$\texttt{push}_{\mathsf{QUD}}(\lambda x, i.\mathsf{race}(i)(x)$
$\quad \wedge\ \mathsf{if\_then\_else}(b, \mathsf{run}_{loc.}(i)(x)(\mathrm{j}), \mathsf{run}_{org.}(i)(x)(\mathrm{j})))$

The result is a discourse of type $\mathbb{P}^{\sigma}_{\mathsf{Q}\iota e\sigma}$.

Responding to questions

Given an ongoing discourse, an interlocutor may respond to the QUD at the top of the stack, based on their prior knowledge. To implement this idea, we can assume that a given responder has some background knowledge $bg : \mathsf{P}\sigma$ over *starting* discourse states, which they use—in conjuction with any interim updates to the discourse—to derive a probability distribution over answers to the current QUD. Given this prior and an ongoing discourse m of type $\mathbb{P}^{\sigma}_{\sigma'}$, we aim to produce an answer distribution, of type $\mathsf{P}\alpha$ (for a QUD of type $\alpha \to \iota \to t$), which has the shape in (39).

(39) $s \sim bg$
$\langle \diamond, s' \rangle \sim m(s)$
$i \sim \mathsf{CG}(s')$
$\boxed{\texttt{answer}(\lambda x.\pi_1(\mathsf{pop\_QUD}(s'))(x)(i))}$

Here, $\pi_1(\mathsf{pop\_QUD}(s'))$ is the QUD at the top of s''s QUD stack. That is, because $\mathsf{pop\_QUD}(s')$ is of type $(\alpha \to \iota \to t) \times \mathsf{popQ}\iota\alpha\sigma'$—it is a pair of a question meaning and a new state—we must grab its first component to obtain a question meaning. Meanwhile, the returned value

$$\texttt{answer}(\lambda x.\pi_1(\mathsf{pop\_QUD}(s'))(x)(i))$$

is a protocol for choosing some value of which this QUD is true, given the index i. Thus answers to the QUD are fundamentally based on the common ground: to compute their probability distribution, an index is sampled from the common ground, and the QUD is evaluated at that index. Thus viewed procedurally, this distribution is gotten by: (i) sampling a state from the prior bg and feeding it to the discourse m; (ii) sampling a new state from the probability distribution thus produced; (iii) sampling an index from this state's common ground; and

(iv) feeding this index to the question meaning popped from this state and then computing an answer from the resulting set.

As for the function `answer`, there are in principle any number of ways it could be instantiated. According to its type, it is just a choice function that chooses elements from sets αs:

$$\texttt{answer} : (\alpha \rightarrow t) \rightarrow \alpha$$

One type of question whose answer we might wish to model, for example, is a degree question; e.g., *how likely is it that S* (see Section 4.1). If we model degrees of likelihood as real numbers, then answering such a question can be understood as selecting the maximum degree of which it is true at some index. Thus `answer` can be interpreted as a maximum:

$$\mathsf{max} : (r \rightarrow t) \rightarrow t$$

3.2.6 Linking models

A core feature of PDS is the support it provides for formal evaluability, i.e., the ability to explicitly quantitatively compare models. We can provide this support by explicitly modeling the relationship between a discourse and a particular data collection instrument whose values correspond to answers to the QUD at the top of the discourse's QUD stack. Such an approach, in turn, allows us to derive explicit probabilistic models that can differ either in the assumptions they make about the underlying probabilistic semantics, or in the way that semantics is related to a response instrument. These probabilistic models may, in turn be fit to data collected using that instrument, and crucially, the relative fits of different models derived under the approach can be compared using standard statistical model comparison metrics.

We assume that a given instrument may be modeled as a family f of distributions representing the *likelihood*, which is then fixed by a collection Φ of nuisance parameters unrelated to the discourse of interest. Thus we may define a family of *response functions*, parametric in the particular data collection instrument (i.e., likelihood function). Each such function takes a distribution representing one's background knowledge bg, along with an ongoing discourse m, and it produces a distribution over answers to the current QUD, given the data collection instrument.

(40)

$$\begin{aligned}\texttt{respond}^{f_\Phi:\alpha\to \mathtt{P}\rho}_{ans:(\alpha\to t)\to\alpha} \quad &: \quad \mathtt{P}\sigma \to \mathbb{P}^{\sigma}_{\mathtt{Q}\iota\alpha\sigma'\diamond} \to \mathtt{P}\rho \\ \texttt{respond}^{f_\Phi}_{ans}(bg)(m) \quad &= \quad s \sim bg \\ &\quad\ \langle \diamond, s' \rangle \sim m(s) \\ &\quad\ i \sim \mathsf{CG}(s') \\ &\quad\ f(ans(\lambda x.\pi_1(\mathsf{pop\_QUD}(s'))(x)(i)), \Phi)\end{aligned}$$

For example, suppose we present an experimental participant with (41) and provide them with two radio buttons labeled '*Jo*' and '*Mo*', respectively—i.e., having the constraint that exactly one button must be selected before proceeding.

(41) a. Jo left.

b. Mo stayed.

c. Which person left?

Viewing (41) as a *discourse*, we can assign it an interpretation like (42) (where we abbreviate the probabilistic semantic values of individual sentences).

(42) `do assert(jo_left)`
`assert(mo_stayed)`
`ask(which_person_left)`

Because the data collection instrument consists of two radio buttons, the response function should produce a probability distribution over length-one strings of bits—i.e., a Bernoulli distribution.

(43) $\mathsf{Bernoulli} : r \to \mathtt{P}t$

Given a real number $p \in [0, 1]$, $\mathsf{Bernoulli}(p)$ returns T with probability p and F with probability $1 - p$. Thus a suitable definition of the response distribution would be the one in (44).

(44)

$$
\begin{aligned}
& \texttt{response}_{\iota}^{\lambda x.\mathsf{Bernoulli}(\mathbb{1}(x=\mathsf{j}))}(bg)((42)) \\
= \; & s \sim bg \\
& \langle \diamond, s' \rangle \sim \texttt{assert}(\texttt{jo\_left})(s) \\
& \langle \diamond, s'' \rangle \sim \texttt{assert}(\texttt{mo\_stayed})(s') \\
& \langle \diamond, s''' \rangle \sim \texttt{ask}(\texttt{which\_person\_left})(s'') \\
& i \sim \mathsf{CG}(s''') \\
& \mathsf{Bernoulli}(\mathbb{1}(\iota x : \pi_1(\mathsf{pop\_QUD}(s'''))(x)(i) = \mathsf{j}))
\end{aligned}
$$

Here, $\mathbb{1}$ is an *indicator* function: it maps T to 1 and F to 0.

(45)

$$
\begin{aligned}
\mathbb{1} &: t \to r \\
\mathbb{1}(\mathsf{T}) &= 1 \\
\mathbb{1}(\mathsf{F}) &= 0
\end{aligned}
$$

As a result, the distribution represented in (44) is Bernoulli with parameter either 1 or 0, depending on whether or not j is an element of the set denoted by the QUD at index i.

In addition, we assume the existence of the constant ι.

(46) $\iota : (e \to t) \to e$

Meanwhile, it is also useful to assume that entities are organized into a join semilattice (Link, 1983), and that ι generally picks out the join of the entities in any set to which it is applied. For example, if the set $p : e \to t$ is a singleton, then $\iota x : p(x)$ is its single element.

Now we can say that if the answer to the question *which person left?* is j, then the response is predicted to take a degenerate distribution all of whose mass is assigned to T—i.e., such that the '*Jo*' button is active; whereas if the answer is something else (e.g., m), then the response is predicted to take a degenerate distribution all of whose mass is assigned to F—i.e., such that the other, '*Mo*' button is active.

Let's turn to an example featuring somewhat more uncertainty. Consider the discourse in (47).

(47) a. Jo, Mo, and Bo left.

b. Two of them returned.

c. Who returned?

Now imagine that the data collection instrument consists of three check boxes, labeled '*Jo*', '*Mo*', and '*Bo*'. Here, we relax the constraint that the choices are mutually exclusive and that one box need be selected in order to proceed. In this case, the response function should produce a probability distribution over length-*three* strings of bits; i.e., a trivariate Bernoulli distribution. Such a probability distribution is defined as accepting seven parameters, since it is effectively a categorical distribution over $2^3 = 8$ categories—one for each of the possible values having the type $t \times t \times t$. For example, it might return $\langle \mathsf{T}, \mathsf{F}, \mathsf{F} \rangle$, $\langle \mathsf{T}, \mathsf{T}, \mathsf{F} \rangle$, $\langle \mathsf{T}, \mathsf{T}, \mathsf{T} \rangle$, and so on.

(48) $\mathsf{TriBernoulli} : r^7 \to \mathtt{P}t^3$

As with the univariate Bernoulli distribution, each parameter p_i is a probability in $[0, 1]$, but now with the additional constraint that $\sum_{i=1}^{7} p_i \leq 1$, i.e., the total probability mass assigned by the distribution. Given all seven probabilities, the eighth is implied, equaling $1 - \sum_{i=1}^{7} p_i$. The resulting probability distribution is over a triple of truth values, each T or F as according to whether or not the box labeled '*Jo*', '*Mo*', or '*Bo*' is checked, respectively. Under these assumptions, a suitable definition of the response distribution would be the one in (49) (here, with m understood to represent the discourse in (47)).

(49)

$$
\begin{aligned}
& \mathtt{respond}_{\iota}^{\lambda x.\mathtt{TriBernoulli}(\mathbb{1}(x=\mathsf{j}),\ldots)}(bg)(m) \\
= \; & s \sim bg \\
& \langle \diamond, s' \rangle \sim m(s) \\
& i \sim \mathsf{CG}(s') \\
& \mathtt{TriBern}(\mathbb{1}(\iota x : \pi_1(\mathsf{pop\_QUD}(s'))(x)(i) = \mathsf{j}), \ldots)
\end{aligned}
$$

Again, the response distribution is degenerate—all of its mass is assigned to one of the eight possible combinations of three truth values—but the exact nature of its degeneracy is now determined by seven parameters, each reflecting an element of the join-semilattice generated by j, m, and b.

Crucially, the discourse in (47)—as opposed to the one in (41)—is associated with significant uncertainty about the answer to its question (*who returned?*).

Unless one has strong priors about Jo, Mo, and Bo, any two-person answer seems about as good as any other. One might wonder if this uncertainty is actually modeled by the response distribution schematized in (49); after all, the likelihood itself is always degenerate, and thus not much of a probability distribution!

In fact, the full model represented by (49) may nonetheless encode a significant amount of uncertainty, resulting in any number of distributions over t^3. This uncertainty stems not from the trivariate Bernoulli likelihood, however, but from the index i which is sampled from the common ground: different indices will provide different answers to the QUD; meanwhile, the total uncertainty will be reflected in the relative flatness of the distribution over these answers. That is, unless one has strong priors about the answer, this distribution is likely to be close to uniform.

We include one more example—this time more schematic—to illustrate `respond` in the setting of a continuous likelihood. See also Section 4.3, where this instance of the function is fully worked out for a task modeling continuous slider-scale judgments. Here, we assume that the function *ans* is fixed to max; i.e., the question answered is a degree question (e.g., *how likely is that...*), and its answer is the maximum degree in the set it denotes. Meanwhile, we assume that someone who responds to the question is attempting to target a degree answer on a bounded slider scale, whose values we model as inhabiting the interval $[0, 1]$. Thus we can fix the response distribution to a *truncated normal distribution*. This allows us to model the targeted degree answer as the location (the underlying mean) of this distribution on the scale; at the same time, we assume that any error in choosing the relevant answer can be modeled as (truncated-)normally distributed noise.

(50)

$$
\begin{aligned}
& \texttt{respond}^{\lambda x.\mathcal{N}(x,\sigma)\,\mathsf{T}[0,1]}_{\mathsf{max}}(bg)(m) \\
= \;& s \sim bg \\
& \langle \diamond, s' \rangle \sim m(s) \\
& i \sim \mathsf{CG}(s') \\
& \mathcal{N}(\mathsf{max}(\lambda d.\pi_1(\mathsf{pop\_QUD}(s'))(d)(i)), \sigma)
\end{aligned}
$$

σ, here, represents the standard deviation of the underlying normal distribution; further the '$\mathsf{T}\,[0, 1]$' notation represents the relevant truncation, i.e., to the unit interval.

Thus besides providing probabilistic dynamic semantic analyses of the linguistic expressions which may constitute some set of experimental materials—

together with the structure of the relevant linking model—PDS recognizes the importance of the assumptions one makes about general background knowledge. All three of these kinds of assumptions are crucial, and they must be specified formally before one evaluates a PDS model to a statistical model that can be fit to data. We return to this point in the next section, where we provide a couple of real-world applications of the framework. Here, we illustrate the explanatory capabilities of the technology we have introduced—focusing in particular on its ability to capture fine-grained distinctions among different forms of uncertainty. As part of this illustration, we show in detail how models stated in typed λ-calculus may be compiled directly into models in the Stan programming language.

3.3 More on δ-rules

To conclude this chapter, we further describe a crucial component of our implementation of PDS: the part which allows the framework to compute values derived from the interaction of constants. Here we list several example δ-rules. For clarity of presentation, many of the rules are defined using Haskell's `PatternSynonyms` language extension. It should be fairly clear, in any given case, what the relevant synonyms abbreviate.

Arithmetic

The following δ-rule performs certain arithmetic simplifications involving addition and multiplication.

```
arithmetic :: DeltaRule
arithmetic = \case
  Add t u      -> case t of
                    Zero -> Just u
                    x@(DCon _) -> case u of
                                    Zero
                                      -> Just x
                                    y@(DCon _)
                                      -> Just (x + y)
                                    _ -> Nothing
                    t'         -> case u of
                                    Zero -> Just t'
                                    _    -> Nothing
```

```
  Mult t u     -> case t of
                    Zero       -> Just Zero
                    One        -> Just u
                    x@(DCon _) -> case u of
                                    Zero
                                      -> Just Zero
                                    One
                                      -> Just x
                                    y@(DCon _)
                                      -> Just (x * y)
                    t'         -> case u of
                                    Zero -> Just Zero
                                    One  -> Just t'
                                    _    -> Nothing
  Neg (DCon x) -> Just (dCon (-x))
  _            -> Nothing
```

Tidying up probabilistic programs

The following rule δ-rule gets rid of vacuous bind statements. Specifically, it removes any unused binding, so long as the probabilistic program to which it is bound has no probabilistic effects besides sampling (e.g., it doesn't perform conditioning).

```
cleanUp :: DeltaRule
cleanUp = \case
  Let v m k
    | sampleOnly m && v `notElem` freeVars k -> Just k
  _                                          -> Nothing
```

The predicate `sampleOnly` is defined as follows:

```
sampleOnly :: Term -> Bool
sampleOnly = \case
  Bern _          -> True
  Normal _ _      -> True
  LogitNormal _ _ -> True
  Truncate m _ _  -> sampleOnly m
  _               -> False
```

Thus Bernoulli distributions, normal distributions, and logit-normal distributions have been defined to satisfy the predicate; truncated distributions satisfy the predicate so long as the distributions they truncate do.

The indicator function

The following δ-rule computes the indicator function $\mathbb{1}$.

```
indicator :: DeltaRule
indicator = \case
  Indi Tr -> Just 1
  Indi Fa -> Just 0
  _       -> Nothing
```

States and indices

The following δ-rule computes reductions for constants that read, update, push, or pop values form either indices or states.

```
states :: DeltaRule
states = \case
  LkUp c   (Upd  c' v _) | c' == c -> Just v
  LkUp c   (Upd  c' _ s) | c' /= c -> Just (LkUp c s)
  Upd  c v (Upd  c' _ s) | c' == c -> Just (Upd c v s)
  Pop  c   (Push c' v s) | c' == c -> Just (v & s)
  Pop  c   (Push c' v s) | c' /= c -> Just (v' & s')
    where v', s' :: Term
          v' = Pi1 (Pop c s)
          s' = Push c' v (Pi2 (Pop c s))
  LkUp c   (Push _  _ s)           -> Just (LkUp c s)
  Pop  c   (Upd  c' v s)           -> Just (v' & s')
    where v', s' :: Term
          v' = Pi1 (Pop c s)
          s' = Upd c' v (Pi2 (Pop c s))
  _                                -> Nothing
```

If then else

The following δ-rule computes *if then else* statements.

```
ite :: DeltaRule
ite = \case
  ITE Tr x y -> Just x
  ITE Fa x y -> Just y
  _          -> Nothing
```

Logical operations

The following δ-rule computes certain propositional logic equivalences.

```
logical :: DeltaRule
logical = \case
  And p  Tr -> Just p
  And Tr p  -> Just p
  And Fa _  -> Just Fa
  And _  Fa -> Just Fa
  Or  p  Fa -> Just p
  Or  Fa p  -> Just p
  Or  Tr _  -> Just Tr
  Or  _  Tr -> Just Tr
  _         -> Nothing
```

Computing the max function

The following δ-rule computes the maximum of sets of real numbers (i.e., when this set is expressed with the right form).

```
maxes :: DeltaRule
maxes = \case
   Max (Lam y (GE x (Var y'))) | y' == y -> Just x
   _                                     -> Nothing
```

Making observations

The following δ-rule allows for reasoning about observe statements involving T and F.

```
observations :: DeltaRule
observations = \case
  Let _ (Observe Tr) k -> Just k
  Let _ (Observe Fa) k -> Just Undefined
  _                    -> Nothing
```

Some ways of computing probabilities

The following δ-rule computes the value of the probability operator, given arguments having particular forms.

```
probabilities :: DeltaRule
probabilities :: DeltaRule
probabilities = \case
  Pr (Return Tr)  -> Just 1
  Pr (Return Fa)  -> Just 0
  Pr (Bern x)     -> Just x
  Pr (Disj x t u) -> Just (x * Pr t + (1 - x) * Pr u)
  _               -> Nothing
```

Combining δ-rules

Finally, note that δ-rules can be combined just like signatures and lexica, using the `(<||>)` function. For example, the rules `observations` and `probabilities` can be combined as `observations <||> probabilities`.

4 Implementations of the framework

As we discussed in Section 2, PDS allows one to naturally distinguish different forms of uncertainty, which broadly fall into the categories of *resolved uncertainty* and *unresolved uncertainty*. One of the goals of this section is to demonstrate how it captures this distinction. We discuss the formal mechanisms by which the distinction is captured in Section 4.1, turning to specific case studies in Sections 4.2 and 4.3.

We should note that while we sometimes take particular positions on which phenomena should be associated with which kinds of uncertainty in this text,

such matters are ultimately empirical. Models fit to linguistic inference data may turn out to perform best—as assessed by standard statistical model comparison metrics—when they regard some source of uncertainty in a way contrary to what we conjecture. Crucially, it is in this way that PDS provides a methodology for performing theory comparison—i.e., by implementing statistical models of linguistic datasets—in addition to providing a theoretical framework for probabilistic semantics.

4.1 Explaining sources of uncertainty

The distinction between resolved and unresolved uncertainty is, in general, reflected in the kinds of responses produced for a particular question. If a question interrogate degrees of likelihood, for example, it will elicit *discrete* inferences about sources of uncertainty which are resolved in nature, but it will elicit *gradient* inferences about sources of uncertainty which are resolved in nature. Here is what we mean and why this is so.

Assume that the relevant question has the form in (51).

(51) How likely is it that S?

To assign a semantics to the modal adjective *likely*, we assume that modal adjective phrases have the same semantic type as sentences. By contrast, the type we assign to ordinary adjective phrases has them map entities to propositions (see Section 4.2 for an example). Meanwhile, degree-denoting expressions are interpreted as real numbers.

(52) $\begin{aligned}[\![ap_{mdl}]\!] &= \iota \to t \\ [\![ap]\!] &= e \to \iota \to t \\ [\![d]\!] &= r\end{aligned}$

Likely itself has the category $ap_{mdl}\backslash d/s$: it selects a sentence, along with a degree-denoting argument, in order to form an ap_{mdl}; thus its semantic type is $(\iota \to t) \to r \to \iota \to t$. The full lexical entry we assign it is in (53) (to be modified in Section 4.2).

(53) *likely*

$$\texttt{do}\ cg \leftarrow \texttt{view}_{\texttt{CG}}\ \boxed{\lambda\phi, d, i.\texttt{Pr}\left(\frac{i' \sim cg}{\boxed{p(i')}}\right) \geq d)} \vdash ap_{mdl}\backslash d/s$$

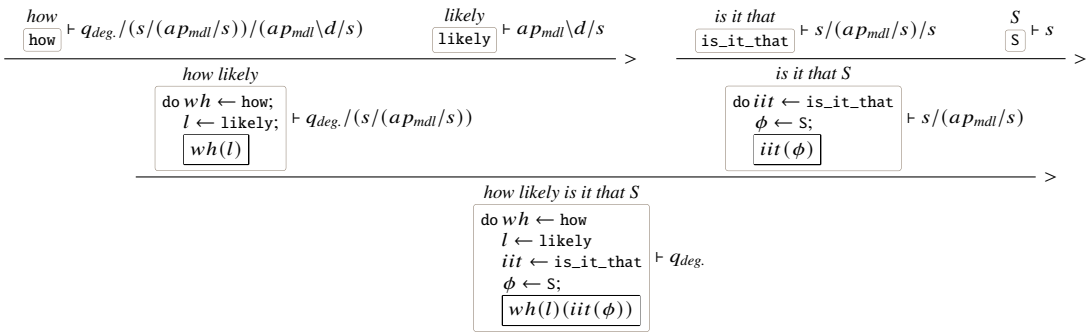


Figure 18 Deriving (51).

According to the semantic part of this lexical entry, *likely* computes the probability that its prejacent is true in the common ground (for analyses in the same vein, see Lassiter, 2017; Swanson, 2016; Yalcin, 2010); it further compares this probability to an abstracted degree threshold ('d'). To compute probabilities, we introduce a constant Pr.

(54) $\mathsf{Pr} : \mathsf{P}t \rightarrow r$

The intention is that a probability is the expected value of the indicator function $\mathbb{1}$ (a fact well-known from probability theory).

The *wh*-word *how* can be given a semantics according to which it feeds its adjectival complement back to the context from which it was extracted, while hanging on to the adjective's degree argument.

(55) *how*: $\lambda f, \phi, i, d.\phi(\lambda p.f(p)(d))$ ⊢ $q_{deg.}/(s/(ap_{mdl}/s))/(ap_{mdl}\backslash d/s)$

For current purposes, we regard the string *is it that* as a constituent and assign it a meaning that merely chases functions around.

(56) *is it that*: $\lambda p, f.f(p)$ ⊢ $s/(ap_{mdl}/s)/s$

A derivation of (51) is provided in Figure 18. Reducing its last term yields the result in (57).

(57) $\texttt{do}\, cg \leftarrow \texttt{view}_{\texttt{CG}}$
$\phi \leftarrow \texttt{S}$
$\boxed{\lambda d, i.Pr\left(\begin{matrix} i' \sim cg \\ \boxed{\phi(i')} \end{matrix}\right) \geq d}$

S here stands in for whatever the probabilistic dynamic meaning of the clause selected by *likely* is. Asking (51) therefore produces the discourse in (58).

(58) $\texttt{do}\, cg \leftarrow \texttt{view}_{\texttt{CG}}$
$\phi \leftarrow \texttt{S}$
$\texttt{set}_{\texttt{QUD}}(\lambda d.Pr\left(\begin{matrix} i \sim cg \\ \boxed{\phi(i)} \end{matrix}\right) \geq d)$

It will become apparent when considering particular examples of prejacents S that the maximum degree d such that

$$Pr\left(\begin{matrix} i \sim cg \\ \boxed{\phi(i)} \end{matrix}\right) \geq d$$

will be either 0 or 1 when S exhibits only resolved uncertainty, while it will potentially be any value in $[0, 1]$ when S exhibits unresolved uncertainty (possibly, in addition to resolved uncertainty). Schematically, this is because if ϕ's value does not in fact depend on i (i.e., if it exhibits no unresolved uncertainty), its probability in the common ground will be either 0 or 1; if ϕ's value does depend on i, this probability will potentially be middling. This point should become clearer as we consider specific examples.

In Sections 4.2 and 4.3, we provide examples of unresolved and resolved uncertainty, respectively. In Section 4.2, we focus on gradable adjectives and the vague inferences they appear to give rise to, and in Section 4.3, we look at the veridicality inferences produced by clause-embedding predicates, with special attention to the treatment they are given in Grove and White 2024. Granted, these two topics provide a mere sample of the phenomena that could potentially fall into either category—however, we believe they give an adequate gist of the framework and how it may be deployed in practice.

4.2 Gradable adjectives and unresolved uncertainty

Adjectives such as *tall*, *wide*, *expensive*, and *happy* are often considered to be *vague*. For example, (59), taken from Kennedy 2007, appears to have somewhat uncertain truth conditions.

(59) The coffee in Rome is expensive.

It is true if the cost of coffee in Rome is as great as some salient threshold for costs—something associated with the adjective *expensive*—but this threshold intuitively remains uncertain when attempting to evaluate whether or not (59) might be true. It may range somewhere from two euros to four euros, for example, but an exact value appears very difficult to pin down.

A hallmark property of vague adjectives like *expensive* is that they exhibit certain unique inference patterns, such as *borderline cases* Kennedy (2007). For example while the Mud Blend ($1.50/lb) might be considered *not expensive*, and Organic Kona ($20/lb) might be considered *expensive*, it's harder to say which category the Swell Start ($9.25/lb) falls into.

Perhaps most famously, vague adjectives—and vague predicates in general—give rise to *sorites* paradoxes. Such paradoxes arise from considering arguments (known as *sorites* arguments) that go as follows.

(60) **Premise 1:** A $10 cup of coffee is expensive.
Premise 2: If an expensive cup of coffee were 1 cent cheaper, it would still be expensive.
Conclusion: Therefore, a free cup of coffee is expensive.

These kinds of inference profiles are notoriously tricky to analyze in terms of the classical notion of truth conditions. Specifically, borderline cases provide instances in which the property denoted by the adjective seems neither to apply nor not to apply to certain entities; but this conflicts with a common theoretical requirement which dictates that truth conditions should bifurcate the space of possible situations into those in which it is true and those in which it is false. Such inference patterns thus at least suggest that traditional model theoretic tools might not be sufficient for studying these kinds of adjectives.

Meanwhile, the sorites paradox is troublesome because it appears to contravene the assumption that inferences should be closed under implication. For example, if we have the following two premises:

(61) a. $10.00 is expensive implies that $9.99 is expensive.
b. $9.99 is expensive implies that $9.98 is expensive.

We should be able to draw the following conclusion:

(62) $10.00 is expensive implies that $9.98 expensive.

And so on, such that we should eventually be able to conclude that $10.00 being expensive (true) implies that $0.00 is expensive (false).

We turn now to the practical challenge of implementing statistical models that test semantic theories against experimental data. The translation from abstract semantic theory to concrete statistical models requires careful attention to how theoretical commitments manifest as computational procedures. This section demonstrates this translation through two models: the first is a relatively simple model of a collection of *norming data*, which elicits judgments about where various objects fall on scales associated with different gradable adjectives; the second is a model of data involving inferences derived from vague gradable adjectives in their positive forms.

4.2.1 Stan as an intermediary

The translation from abstract semantic analyses to concrete statistical models requires an intermediate representation—basically, a language that relates the abstract probabilistic models we state in PDS to the data that we want to fit these models to. Stan has emerged as the *de facto* standard for this role in computational semantics and psycholinguistics (Bürkner, 2017; {Stan Development Team}, 2024). We are going to use it for our implementations, but it's important to note that any language that allows us to express sets of interrelated distributional assumptions would do, in principle.

Stan is a probabilistic programming language designed for statistical inference. Unlike general-purpose programming languages, Stan is specialized for stating distributional assumptions. These are then translated to a C++ backend that performs statistical inference—usually, using a form of Markov Chain Monte Carlo (MCMC) sampling. Stan includes some imperative constructs, but for our purposes, the language provides a means to declare the structure of a probabilistic model, in the sense that we need not pay attention to whatever procedure it implements when actually fitting the model to data.

This method aligns well with our goals: just as a formal semantic analysis is used to declare a representation of semantic content—but not the details of how such content is used in verifying truth or drawing inferences—Stan allows for declaration of probabilistic relationships independent of sampling algorithms. The parallel is not accidental: both frameworks separate the *what* (semantic content, distributional assumptions) from the *how* (verification or processing, inference algorithms). Said another way, our translation to Stan allows us to retain a certain kind of modularity: it provides an interface that hides the gritty details of involved in fitting the models produced by a given semantic theory.

4.2.2 Kernel models

Before diving into the implementation details, it's important to understand what PDS produces and how it relates to the full statistical models we'll develop. PDS outputs what we term a *kernel model*—the semantic core of a complete mixed effects model that corresponds directly to the lexical and compositional semantics. This kernel can in principle be augmented with other components, such as random effects, hierarchical priors, and other statistical machinery, but the current implementation focuses on producing just this semantic kernel.

This distinction is crucial: PDS automates the translation from the compositional semantics to the core statistical model, while leaving room for the analyst to add domain-specific statistical structure. We take this to be a useful separation of concerns, since it allows the semantic theory to remain agnostic about aspects of the statistical model that have nothing to do with the semantic theory itself.

With this understanding of kernel models and Stan's role, we can now examine specific cases. We'll start with a simple one—inferring the degrees associated with different objects on an adjective's scale—before building up to a model of vagueness. We'll be showing prettified versions of what the system actually produces. For each model, we'll also provide the kernel model produced in Haskell, along with the prettified version that includes additional statistical structure.

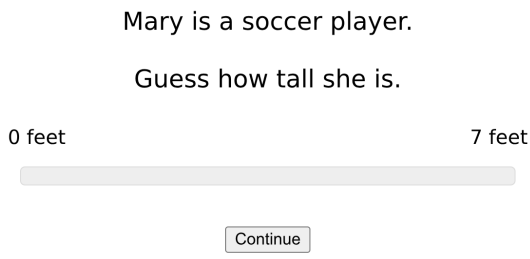
Our first model addresses a fundamental question: how do we infer the "shape" of people's prior beliefs about the degrees that gradable adjectives invoke? The norming experiment that we briefly discuss here provides a clean test case where participants directly report degrees on scales. We'll present the model in a couple of steps. First, we'll provide a realistic model of this data, which one might design as a means for analyzing it. We'll build up the model block-by-block, explaining each line. Then, we'll turn to how we might analyze the discourse representing a trial in the experiment used to collect the data in PDS. In doing so, we'll show which components of the model correspond to the PDS kernel model and which ones are extensions of the model by the analyst.

4.2.3 Understanding the experimental setup

In this task, which we call the *scale-norming* task, we provide experimental participants with prompts like the following one:

participant	adjective	condition	response
1	old	mid	0.51
1	expensive	mid	0.62
1	deep	low	0.22

Figure 19 Example data from the scale-norming experiment.



The idea here is to assess where people believe objects of different kinds fall on the scale associated with some gradable adjective. This information, which we attempt to model, is useful because it allows us to estimate one of the possible sources of uncertainty which are commonly thought to affect the inferences associated with these kinds of adjectives—that which comes from *world knowledge* about the entity itself (in particular where it falls on the adjective's scale). For each adjective that we investigate, the experiment has three conditions—one for each of three different entities—thus providing three different contexts preceding the question prompt. One of these conditions is a "high" condition: we expect the relevant object to he highly on the adjectival scale, compared to the other two conditions. Meanwhile, there is also a "mid" condition and a "low" condition.

Before diving into the Stan code, let's consider how we'll represent the data collected in this experiment, since its format is important for understanding the code. Three example rows of data are provided in Figure 19. Each row represents a single judgment.

- participant: which person made this judgment (participant 1, 2, etc.)
- adjective: the gradable adjective being tested
- condition: whether the judgment is of a high, mid, or low context
- response: the participant's slider response (in [0, 1])

Our simplest model asks the following questions: what degree does each entity

have on the relevant adjectival scale, and how do participants map these degrees to slider responses?

4.2.4 The structure of a Stan program

Every Stan program follows a particular architecture with blocks that appear in a specific order. Each block serves a specific purpose in defining our statistical model. We'll build up our norming model block by block to understand how Stan works.

The data block

Every Stan program begins with a *data* block that tells Stan what information will be provided from the outside world—our experimental observations. Let's build it up piece by piece.

```
data {
  int<lower=1> N_item;        // number of items
  int<lower=1> N_participant; // number of participants
  int<lower=1> N_data;        // number of data points
```

These lines declare basic counts. Their syntax breaks down as follows:

- `int` : this will be an integer (whole number).
- `<lower=1>` : this integer must be at least 1 (no negative counts!).
- `N_item` : the variable name, we'll use this throughout our program.
- `// number of items` : a comment explaining what the line represents.

Why do we need the constraints? Stan uses them to: (i) catch errors early (e.g., if we accidentally pass 0 items, the program will complain); and (ii) optimize its algorithms (e.g., knowing bounds helps the sampler work efficiently).

Next, we handle a subtle but important issue—boundary responses:

```
  int<lower=1> N_0; // number of 0s
  int<lower=1> N_1; // number of 1s
```

Why separate these out? Slider responses of exactly 0 or 1 are "censored"—they might represent even more extreme judgments which the scale can't capture. We'll handle these in a second. For the actual response data:

```
  vector<lower=0, upper=1>[N_data] y; // response in (0, 1)
```

This line declares a vector (like an array) of length `N_data`, in which each element must be strictly between (i.e., not including) 0 and 1.

Finally, we need to map responses to items and participants:

```
  array[N_data] int<lower=1, upper=N_item> item;
  array[N_0] int<lower=1, upper=N_item> item_0;
  array[N_1] int<lower=1, upper=N_item> item_1;
  array[N_data] int<lower=1, upper=N_participant> participant;
  array[N_0] int<lower=1, upper=N_participant> participant_0;
  array[N_1] int<lower=1, upper=N_participant> participant_1;
}
```

These arrays function as lookup tables. If `item[5]` is 3, then the fifth response in our data is about item number 3. This indexing structure connects our flat data file to the hierarchical structure of our experiment.

If we looking back at our data, we can see that when Stan reads it, it will:

- count the number of unique items and bind the result to `N_item` (e.g., 36 if we have 12 adjectives and 3 conditions);
- count the number of unique participants and bind the result to `N_participant`;
- extract any responses between and excluding 0 and 1 and bind the resulting vector to `y` vector; and
- build index arrays mapping each response to its item and participant.

The parameters block: what we want to learn

Having declared our data, we must declare our parameters; i.e., the unknown quantities that we want to infer. This is where semantic theory meets statistical inference:

```
parameters {
  // Fixed effects
  vector[N_item] mu_guess;
```

This line declares a vector of "guesses" (degrees) for each item. Why the label '`mu_guess`'? In statistics, μ ('mu') traditionally denotes a mean or central tendency. These means can be understood as representing our best guess, as researchers, about each item's true degree on its scale—the theoretical degrees that the semantic analysis posits. But crucially, they can also be viewed as

representing *subjects'* uncertainty about these degrees; that is, what unresolved uncertainty do they maintain when they make their guesses?

Furthermore, people differ! We need random effects to capture individual variation:

```
  // Random effects...
  // how much people vary:
  real<lower=0> sigma_epsilon_guess;
  // each participant's deviation
  vector[N_participant] z_epsilon_guess;
```

This statement of the random effects associated with people's guesses uses a clever trick known as *non-centered parameterization*:

- `sigma_epsilon_guess`: the overall amount of person-to-person variation (i.e., the standard deviation);
- `z_epsilon_guess`: standardized (z-score) deviations for each person.

We'll combine these later to obtain each person's actual adjustment. An alternative approach would be to define and use `vector[N_participant] epsilon_guess` directly, as each person's non-standardized deviation. Why not do this? This separation enforced by the non-centered approach often helps Stan's sampling algorithms converge much faster—a practical consideration that doesn't affect the semantic theory, but which matters for the implementation.

Next, we handle measurement noise:

```
  real<lower=0,upper=1> sigma_e;  // response variability
```

Even if two people agree on an item's degree, their physical slider responses might differ slightly. This parameter captures such noise.

Finally, the boundary responses:

```
  // Censored data...
  // true values for observes 0s:
  array[N_0] real<upper=0> y_0;
  // true values for observed 1s:
  array[N_1] real<lower=1> y_1;
}
```

The issue of boundary responses is subtle, but important. When someone gives a 0 response, their "true" judgment might be −0.1 or −0.5—we don't know because we can't see below 0. These parameters let Stan infer what those true values might have been.

The transformed parameters block: building predictions

Now we combine our basic parameters to build ones which are more directly useful in defining the model. The *transformed parameters* block serves as a bridge between abstract parameters and concrete predictions:

```
transformed parameters {
  vector[N_participant] epsilon_guess;
  vector[N_data] guess;
  vector[N_0] guess_0;
  vector[N_1] guess_1;
```

First, we can convert the z-scores from the parameters block into actual participant adjustments:

```
  // Parameterization of participant deviations:
  epsilon_guess = sigma_epsilon_guess * z_epsilon_guess;
```

Thus if `sigma_epsilon_guess` = 0.2 and participant 3 is such that `z_epsilon_guess[3]` = 1.5, then participant 3 tends to give responses 0.3 units higher than average.

Now we can define the predicted probability of responses:

```
  for (i in 1:N_data) {
    guess[i] = mu_guess[item[i]]
      + epsilon_guess[participant[i]];
  }
```

Let's trace through one prediction to see how this works. Say the item number is 5 and the participant is 3:

- `item[i]` = 5, so we look up `mu_guess[5]` (say it's 0.7);
- `participant[i]` = 3, so we add `epsilon_guess[3]` (say it's 0.1);
- thus `guess[i]` = 0.7 + 0.1 = 0.8.

We can define a similar structure for the boundary responses:

```
  for (i in 1:N_0) {
    guess_0[i] = mu_guess[item_0[i]]
      + epsilon_guess[participant_0[i]];
  }
```

```
  for (i in 1:N_1) {
    guess_1[i] = mu_guess[item_1[i]]
      + epsilon_guess[participant_1[i]];
  }
}
```

The model block

The model block is where we specify our statistical assumptions, both about our prior beliefs and about how the data was generated. This part of the model definition is where PDS contributes the most.

```
model {
  // Priors on random effects
  sigma_epsilon_guess ~ exponential(1);
  z_epsilon_guess ~ std_normal();
```

The priors we state here encode mild assumptions. In turn:

- `exponential(1)`: we expect person-to-person variation to be moderate, not huge;
- `std_normal()`: by definition, z-scores take a standard normal distribution.

Notice that we haven't specified any priors for `mu_guess`; as a result, Stan assigns it an implicit (improper) uniform prior over the real numbers. Since our responses are bounded, moreover, the data will naturally constrain these values.

Now, we define the likelihood, which determines how the data relates to the parameters we've defined:

```
  // Likelihood
  for (i in 1:N_data) {
    y[i] ~ normal(guess[i], sigma_e);
  }
```

This part of the model definition dictates that each response is drawn from a normal distribution centered at our prediction `guess[i]`, with standard deviation `sigma_e`. The `~` symbol means "is distributed as".

For boundary responses, we may use the latent values that we have already defined:

```
  for (i in 1:N_0) {
    y_0[i] ~ normal(guess_0[i], sigma_e);
  }

  for (i in 1:N_1) {
    y_1[i] ~ normal(guess_1[i], sigma_e);
  }
}
```

Remember that we're inferring `y_0` and `y_1` as *parameters*—they are not data. Stan will sample plausible values that are consistent with both the model definition and the number of 0s and 1s observed in the data.

The generated quantities block

Finally, we also wish to compute quantities that help us understand and evaluate the model. These are defined in the generated quantities block:

```
generated quantities {
  vector[N_data] ll; // log-likelihoods

  for (i in 1:N_data) {
    if (y[i] > 0 && y[i] < 1)
      ll[i] = normal_lpdf(y[i] | guess[i], sigma_e);
    else
      ll[i] = negative_infinity();
  }
}
```

The vector `ll` of log-likelihood values tells us how probable each observation is under our model. We'll use these for model comparison: models that assign higher probability to the actual data are, in general, better.[14]

[14] Strictly speaking, what we truly care about is the expected log predictive densities (ELPDs) of a given collection of models fit to some dataset. This quantity is just the sum of the log-probabilities of each data point under a model in which only the other data points have actually been observed. Since this quantity is difficult (or impossible) to compute in principle, it is standard to estimate it by some approximation (we rely on the one provided by the widely-applicable information criterion (WAIC; Vehtari et al., 2023; Watanabe, 2013)).

The complete model

Putting everything together gives us the following full model definition:

```
data {
  int<lower=1> N_item;          // number of items
  int<lower=1> N_participant;   // number of participants
  int<lower=1> N_data;          // number of data points in (0, 1)
  int<lower=1> N_0;             // number of 0s
  int<lower=1> N_1;             // number of 1s
  vector<lower=0, upper=1>[N_data] y; // response in (0, 1)
  array[N_data] int<lower=1, upper=N_item> item;
  array[N_0] int<lower=1, upper=N_item> item_0;
  array[N_1] int<lower=1, upper=N_item> item_1;
  array[N_data] int<lower=1, upper=N_participant> participant;
  array[N_0] int<lower=1, upper=N_participant> participant_0;
  array[N_1] int<lower=1, upper=N_participant> participant_1;
}

parameters {
  vector[N_item] mu_guess;
  real<lower=0> sigma_epsilon_guess;
  vector[N_participant] z_epsilon_guess;
  real<lower=0,upper=1> sigma_e;
  array[N_0] real<upper=0> y_0;
  array[N_1] real<lower=1> y_1;
}

transformed parameters {
  vector[N_participant] epsilon_guess
    = sigma_epsilon_guess * z_epsilon_guess;
  vector[N_data] guess;
  vector[N_0] guess_0;
  vector[N_1] guess_1;

  for (i in 1:N_data) {
    guess[i] = mu_guess[item[i]]
      + epsilon_guess[participant[i]];
  }
  for (i in 1:N_0) {
    guess_0[i] = mu_guess[item_0[i]]
      + epsilon_guess[participant_0[i]];
  }
```

```
  for (i in 1:N_1) {
    guess_1[i] = mu_guess[item_1[i]]
      + epsilon_guess[participant_1[i]];
  }
}

model {
  sigma_epsilon_guess ~ exponential(1);
  z_epsilon_guess ~ std_normal();

  for (i in 1:N_data) {
    y[i] ~ normal(guess[i], sigma_e);
  }
  for (i in 1:N_0) {
    y_0[i] ~ normal(guess_0[i], sigma_e);
  }
  for (i in 1:N_1) {
    y_1[i] ~ normal(guess_1[i], sigma_e);
  }
}

generated quantities {
  vector[N_data] ll; // log-likelihoods
  definition:
  for (i in 1:N_data) {
    if (y[i] >= 0 && y[i] <= 1)
      ll[i] = normal_lpdf(y[i] | guess[i], sigma_e);
    else
      ll[i] = negative_infinity();
  }
}
```

Thus in the end, the model treats each item as being associated with a degree along the relevant adjectival scale. Participants, meanwhile, provide noisy measurements of these degrees. And our censoring approach deals with the fact that there are responses at the boundaries (0 and 1) of the slider scale.

4.2.5 PDS to Stan

Which components of this model are derivable within PDS? To answer this question, we should define our PDS representation of the scale-norming task itself. In PDS we may accomplish this by providing a generic representation

```
s1, q1, discourse1 :: Term

s1         = getSemanticsAdjectives
               [ "Mary", "is", "a", "soccer player" ]

q1         = getSemanticsAdjectives
               [ "how", "tall", "Mary", "is" ]

discourse1 = assert s1 >>> ask q1
```

Figure 20 A Haskell-encoded discourse for the scale-norming task.

of a discourse for each item in the experiment. For example, following the experimental set-up described in Section 4.2.3, we might attempt to model a discourse consisting of the assertion in (63a) and the prompt in (63b).

(63) a. Mary is a soccer player.

b. Guess how tall she is.

This discourse can be modeled by sequencing a representation of an assertion of (63a) with a representation of a question corresponding to (63b), as in Figure 20.

This code models both the assertion and the prompt by relying on a function to obtain probabilistic semantic parses from strings of expressions:

```
getSemanticsAdjectives :: [String] -> Term
```

Without going into the full details of how the relevant terms are extracted, we will describe the important components of this process; in particular the CCG lexical entries that provide the meanings which are assembled compositionally to produce `s1`, `q1`, and thus, ultimately, `discourse1`.

Before diving into these lexical entries, a few small notes. First—as can be seen by inspecting the Haskell representation of the strings which are parsed—we do not explicitly model the semantic contribution of the word *guess*, instead opting to (effectively) identity the meaning of the prompt in (63b) with the meaning of the embedded question that *guess* selects. Second, we don't deal with the issue of anaphora presented by the pronoun *she* inside the prompt, instead replacing it with the name *Mary*.

Working through the discourse

In order to say more about the content of `s1`, `q1`, and `discourse1`, we should work through the lexical entries associated with the individual words featured in Figure 20. Following the strategy adopted in Section 3.2, we'll provide these lexical entries—both in mathematical notation and in Haskell—as separate lettered examples. We start with the lexical entries relevant to `s1`, in (65). It'll be easier to give the lexical entries in this section if we have a few extra Haskell abbreviations for syntactic categories (i.e., besides `np` and `s`):

```
ap, apMdl, d, np, npPred, qDeg :: Cat
ap     = Base "ap"      // adjective phrases
apMdl  = Base "ap_mdl"  // modal adjective phrases
d      = Base "d"       // degrees
npPred = Base "np_pred" // predicate noun phrases
qDeg   = Base "q_deg"   // degree questions
```

Importantly, we're introducing a category for noun phrases that appear in predicate position—these have a different semantic interpretation than regular noun phrases:

(64) $[\![np_{pred.}]\!] = e \to \iota \to t$

Further, the constants featured in the meanings of these lexical entries (as well as those for the next model) will require a type signature:

```
adjSig :: Sig
adjSig = \case
  Left "height"       -> Just (ι :-> e :-> r)
  Left "upd_height"   -> Just ((e :-> r) :-> ι :-> ι)
  Left "soc_pla"      -> Just (ι :-> e :-> t)
  Left "upd_soc_pla"  -> Just ((e :-> t) :-> ι :-> ι)
  Left "@"            -> Just ι
  Left "CG"           -> Just (σ :-> P ι)
  Left "upd_CG"       -> Just (P ι :-> σ :-> σ)
  Left "pop_QUD"      -> Just (σ :-> (ι : -> α :-> t)
                                :* TyCon "popQ" [ι, α, σ])
  Left "push_QUD"     -> Just ((ι :-> α :-> t) :-> σ
                                :-> TyCon "Q" [ι, α, σ])
  Left "ϵ"            -> Just σ
```

```
  Left "m"             -> Just e
  Left "if_then_else" -> Just (t :* α :* α) :-> α
  Left "(≥)"           -> Just (r :-> r :-> r)
  Left "max"           -> Just ((r :-> t) :-> r)
  Left "Pr"            -> Just (P t :-> r)
  Left "Normal"        -> Just (r :* r :-> P r)
  Left "Logit_Normal" -> Just (r :* r :-> P r)
  Left "Truncate"      -> Just (P r :* r :* r -> P r)
  Left "∞"             -> Just r
  Right _              -> Just r
  _                    -> Nothing
```

(65) a. $\overset{Mary}{\boxed{\lambda k.k(\mathsf{m})}} \vdash s/(s\backslash np)$

```
lexAPred :: Lexicon SynSem
lexAPred = \case
  "Mary" -> [ SynSem {
                syn = s // (s \\ np),
                sem = ty adjSig $
                  purePP (lam x (x @@ (SCon "m")))
              } ]
  _      -> []
```

b. $\overset{is}{\boxed{\lambda p.p}} \vdash s\backslash np/np_{pred.}$

```
lexIs :: Lexicon SynSem
lexIs = \case
  "is" -> [ SynSem {
              syn = s \\ np // npPred,
              sem = ty adjSig $
                purePP (lam x x)
            } ]
  _    -> []
```

c. $\overset{a}{\boxed{\lambda p.p}} \vdash np_{pred.}/n$

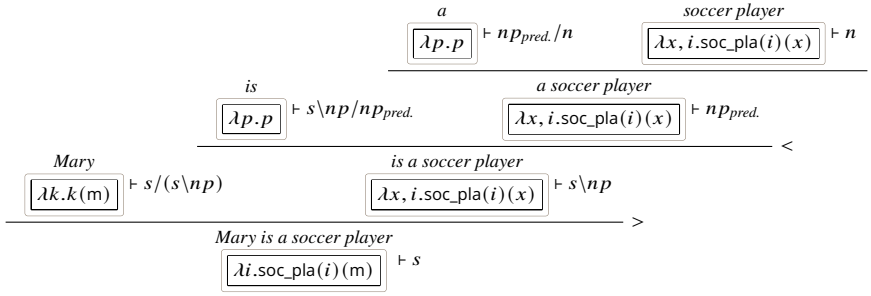


Figure 21 Derivation of *Mary is a soccer player*.

```
lexAPred :: Lexicon SynSem
lexAPred = \case
  "a" -> [ SynSem {
             syn = npPred // n,
             sem = ty adjSig $
               purePP (lam x x)
             } ]
  _   -> []
```

d. *soccer player* $\boxed{\lambda x, i.\text{soc\_pla}(i)(x)} \vdash n$

```
lexAPred :: Lexicon SynSem
lexAPred = \case
  "soccer player"
    -> [ SynSem {
           syn = n,
           sem = ty adjSig $
                   purePP (lam x (lam i (
                     SCon "soc_pla" @@ i @@ x
                   ) ) )
         } ]
  _ -> []
```

A derivation of the sentence in (63a) is provided in Figure 21.

Now for the prompt in (63b), for which we require lexical entries for *how*, *tall*, and another variant of *is* (one which selects adjective phrases instead rather

than noun phrases).

(66) a. $\overset{how}{\boxed{\lambda f, g, d.g(f(d))}} \vdash q_{deg.}/(s/ap)/(ap\backslash d)$

```
lexHow :: Lexicon SynSem
lexHow = \case
  "how" -> [ SynSem {
                syn =
                  qDeg // (s // ap) // (ap \\ d),
                sem = ty adjSig $
                  purePP (lam x (lam y (lam z (
                    y @@ (x @@ z)
                  ))))
              } ]
  _       -> []
```

b. $\overset{tall}{\boxed{\lambda d, x, i.\mathsf{height}(i)(x) \geq d}} \vdash ap\backslash d$

```
lexTall :: Lexicon SynSem
lexTall = \case
  "tall" -> [ SynSem {
                syn = ap \\ d,
                sem = ty adjSig $
                  purePP (lam x (lam y (lam i (
                    SCon "(≥)" @@
                      (SCon "height" @@ i @@ y) @@
                      x
                  ))))
              } ]
  _       -> []
```

c. $\overset{is}{\boxed{\lambda p.p}} \vdash s\backslash np/ap$

```
lexIs :: Lexicon SynSem
lexIs = \case
  "is" -> [ SynSem {
              syn = s \\ np // ap,
              sem = ty adjSig $
                purePP (lam x x)
            } ]
  _    -> []
```

A similar derivation (which we don't provide here)—one which, like the derivation for (63a), features only pure values—can be also be provided for (63b). Given the resulting meanings, the discourse itself can be specified as (67).

(67) $\mathtt{do}\, cg \leftarrow \mathtt{view}_{\mathsf{CG}}$
$\mathtt{set}_{\mathsf{CG}}\left(\begin{array}{l} i \sim cg \\ \mathsf{observe}(\mathsf{soc\_pla}(i)(\mathsf{m})) \\ \boxed{i} \end{array}\right)$
$\mathtt{push}_{\mathsf{QUD}}(\lambda d, i.\mathsf{height}(\mathsf{m}) \geq d)$

In terms of this discourse, we would like to produce a representation of a *response distribution*. As per the discussion in Section 3.2.6, this representation can be obtained from suitable definitions of a prior distribution over states, together with a response function:

```
scaleNormingExample :: Term
scaleNormingExample = respondAdj
                        scaleNormingPrior
                        discourse1
```

Let's unpack this. `respondAdj` implements the function `respond` discussed in Section 3.2, with particular choices made for the likelihood and the answer function. We can define it as follows:

```
respondAdj :: Term -> Term -> Term
respondAdj bg m
  = respond (lam x (Normal x (SCon "sigma"))) bg m

respond :: Term -> Term -> Term -> Term
```

```
respond f bg m = let' s bg m'
  where m' = let' _s' (m @@ s) (
                       let' i (cg (Pi2 _s')) (
                         f @@ max' (lam x (
                           Pi1 (pop_qud (Pi2 _s'))
                             @@ x
                             @@ i
                         ))
                       )
                     )
        s:_s':i:x:_ = map Var $ fresh [bg, m]
        max' pred   = SCon "max" @@ pred
```

The definition of `respond` encodes precisely the definition in (40) of Section 3.2.6, with `SCon "max"` chosen to compute the answer from a degree question meaning. `respondAdj` itself, then, can be rendered more perspicuously in (68).[15]

(68)

$$\begin{aligned}\texttt{respondAdj} &: \mathrm{P}\sigma \to \mathbb{P}^{\sigma}_{Q\iota r\sigma'} \to \mathrm{P}r\\ \texttt{respondAdj}(bg)(m) &= s \sim bg\\ &\quad\ \langle \diamond, s' \rangle \sim m(s)\\ &\quad\ i \sim \mathsf{CG}(s')\\ &\quad\ \mathcal{N}(\mathsf{max}(\lambda d.\pi_1(\mathsf{pop\_QUD}(s'))(d)(i)), \sigma)\end{aligned}$$

Meanwhile, the function `scaleNormingPrior` encodes the prior over states used by this definition of the response function, while `discourse1` is as defined (67). To help reduce clutter, we present the definition of the prior in mathematical notation, rather than in Haskell.

[15]For extra clarity, we present the definition as a single λ-term, rather than as a function from λ-terms to functions from λ-terms to λ-terms (which would more faithfully align with the type of `respondAdj`).

(69)
$$\mathsf{upd\_CG}\left(\begin{array}{l} d_{\textit{soc. pla.}} \sim \mathcal{N}(165,5)\,\mathsf{T}[0,\infty] \\ d_{\textit{not soc. pla.}} \sim \mathcal{N}(160,5)\,\mathsf{T}[0,\infty] \\ p \sim \mathsf{Logit\_Normal})(0,1) \\ \tau_{\textit{soc. pla.}} \sim \mathsf{Bernoulli}(p) \\ \mathsf{upd\_height}(\\ \quad \lambda x.\mathsf{if\_then\_else}(\tau_{\textit{soc. pla.}}, d_{\textit{soc. pla.}}, d_{\textit{not soc. pla.}}))(\\ \quad\quad \mathsf{upd\_soc\_pla}(\lambda x.\tau_{\textit{soc. pla.}})(@) \\ \quad) \end{array}\right)(\epsilon)$$

This prior distribution, of type $\mathrm{P}\sigma$, degenerately returns a single state. This state is gotten by updating a starting state (ϵ) with a common ground two possible prior distributions over Mary's height are defined (understood in centimeters), the choice between which depends on whether or not Mary is a soccer player. Further, a Bernoulli distribution whose parameter is sampled from a logit-normal distribution is defined; this Bernoulli provides the value, T or F, determining whether or not Mary is a soccer player. By inspecting the embedded return statement, we can see that the degree which is chosen, and thus used to set the value of height, depends on precisely this parameter.

δ-rules and semantic computation

If we put everything together, we can see that we obtain the probabilistic program in (70) as `scaleNormingExample`.

(70)
$$\begin{array}{l} s \sim \mathsf{upd\_CG}\left(\begin{array}{l} d_{\textit{soc. pla.}} \sim \mathcal{N}(165,5)\,\mathsf{T}[0,\infty] \\ d_{\textit{not soc. pla.}} \sim \mathcal{N}(160,5)\,\mathsf{T}[0,\infty] \\ p \sim \mathsf{Logit\_Normal})(0,1) \\ \tau_{\textit{soc. pla.}} \sim \mathsf{Bernoulli}(p) \\ \mathsf{upd\_height}(\\ \quad \lambda x.\mathsf{if\_then\_else}(\tau_{\textit{soc. pla.}}, d_{\textit{soc. pla.}}, d_{\textit{not soc. pla.}}))(\\ \quad\quad \mathsf{upd\_soc\_pla}(\lambda x.\tau_{\textit{soc. pla.}})(@) \\ \quad) \end{array}\right)(\epsilon) \\ s' \sim \mathsf{upd\_CG}\left(\begin{array}{l} i \sim \mathsf{CG}(s) \\ \mathsf{observe}(\mathsf{soc\_pla}(i)(\mathsf{m})) \\ i \end{array}\right)(s) \\ s'' \sim \mathsf{push\_QUD}(\lambda d, i.\mathsf{height}(\mathsf{m}) \geq d)(s') \\ i' \sim \mathsf{CG}(s'') \\ \mathcal{N}(\max(\lambda d.\pi_1(\mathsf{pop\_QUD}(s''))(d)(i')), \sigma) \end{array}$$

Crucial now is PDS's application of δ-rules, which act to simplify (70) into a simple, readable λ-term—the kind that is easily translated into Stan code. to simplify these complex λ-terms. As discussed in Section 3.3, δ-rules enable various semantic computations, while preserving semantic equivalence.

Key δ-rules for (70) include, e.g., state and index extraction—rules for height, d_{tall}, etc—and others. Further, β-reduction is also crucial at this point, both as it applies before and after the application of various δ-rules.

Ultimately, these rules transform the complex compositional semantics into simpler forms suitable for compilation. The transformation preserves semantic content while rendering it computationally tractable.

Working through δ-reductions

Having seen the PDS code for degree question prompts, we now trace through how δ-rules transform such complicated λ-terms into forms suitable for Stan compilation. Recall that δ-rules are partial functions from terms to terms that implement semantic computations:

```
type DeltaRule = Term -> Maybe Term
```

First, let's observe what happens to the λ-term in (70) when the function `betaDeltaNormal states` is applied to it. Recall that the definition of `betaDeltaNormal` is provided in Figure 12. We repeat the definition of `states` here from Section 3.3:

```
states :: DeltaRule
states = \case
  LkUp c   (Upd  c' v _) | c' == c -> Just v
  LkUp c   (Upd  c' _ s) | c' /= c -> Just (LkUp c s)
  Upd  c v (Upd  c' _ s) | c' == c -> Just (Upd c v s)
  Pop  c   (Push c' v s) | c' == c -> Just (v & s)
  Pop  c   (Push c' v s) | c' /= c -> Just (v' & s')
    where v', s' :: Term
          v' = Pi1 (Pop c s)
          s' = Push c' v (Pi2 (Pop c s))
  LkUp c   (Push _  _ s)           -> Just (LkUp c s)
  Pop  c   (Upd  c' v s)           -> Just (v' & s')
    where v', s' :: Term
```

```
          v' = Pi1 (Pop c s)
          s' = Upd c' v (Pi2 (Pop c s))
  _                                  -> Nothing
```

Thus the values of constants—those that update the common ground, as well as those that update the QUD—are computed from this rule, yielding the result in (71).

(71) $d_{soc.\ pla.} \sim \mathcal{N}(165, 5)\,\mathsf{T}[0, \infty]$
$d_{not\ soc.\ pla.} \sim \mathcal{N}(160, 5)\,\mathsf{T}[0, \infty]$
$p \sim \mathsf{Logit\_Normal})(0, 1)$
$\tau_{soc.\ pla.} \sim \mathsf{Bernoulli}(p)$
$\mathsf{observe}(\tau_{soc.\ pla.})$
$\mathcal{N}(\max(\lambda d.\mathsf{if\_then\_else}(\tau_{soc.\ pla.}, d_{soc.\ pla.}, d_{not\ soc.\ pla.}) \geq d), \sigma)$

Thus what we are left with is a program that appears quite close to the definition of a probabilistic model.

What we would like to do now is handle the observe statement, but in order to do that, we need to perform a special trick: we need to turn the Bernoulli program that feeds it into a disjunction of programs. To accomplish that, we can use a δ-rule:

```
disjunctions :: DeltaRule
disjunctions = \case
  Let  b (Bern x)     k
    -> Just (Disj x (subst b Tr k) (subst b Fa k))
  Let  v (Disj x m n) k
    -> Just (Disj x (Let v m k) (Let v n k))
  Disj _ m            n          | m == n -> Just m
  Disj _ m            Undefined           -> Just m
  Disj _ Undefined    n                   -> Just n
  _                                       -> Nothing
```

Thus applying `betaDeltaNorming (states <||> disjunctions)` to (70) results in (72).

(72) $d_{soc.\ pla.} \sim \mathcal{N}(165, 5)\,\mathsf{T}[0, \infty]$
$d_{not\ soc.\ pla.} \sim \mathcal{N}(160, 5)\,\mathsf{T}[0, \infty]$
$p \sim \mathsf{Logit\_Normal})(0, 1)$
$\mathsf{disj}(p,$
$\mathsf{observe}(\mathsf{T})$
$\mathcal{N}(\mathsf{max}(\lambda d.\mathsf{if\_then\_else}(\mathsf{T}, d_{soc.\ pla.}, d_{not\ soc.\ pla.}) \geq d), \sigma),$
$\mathsf{observe}(\mathsf{F})$
$\mathcal{N}(\mathsf{max}(\lambda d.\mathsf{if\_then\_else}(\mathsf{F}, d_{soc.\ pla.}, d_{not\ soc.\ pla.}) \geq d), \sigma)$
)

Now we can see that observe statements are more straightforward to deal with; in particular, we can just include the δ-rule `observations` stated in Section 3.3, which eliminates trivial observations, and yields undefined programs from impossible observations:

```
observations :: DeltaRule
observations = \case
  Let _ (Observe Tr) k -> Just k
  Let _ (Observe Fa) _ -> Just Undefined
  _                    -> Nothing
```

Applying `betaDeltaNormal (states <||> disjunctions <||> observations)` can be seen to result in (73).

(73) $d_{soc.\ pla.} \sim \mathcal{N}(165, 5)\,\mathsf{T}[0, \infty]$
$d_{not\ soc.\ pla.} \sim \mathcal{N}(160, 5)\,\mathsf{T}[0, \infty]$
$p \sim \mathsf{Logit\_Normal})(0, 1)$
$\mathcal{N}(\mathsf{max}(\lambda d.\mathsf{if\_then\_else}(\mathsf{T}, d_{soc.\ pla.}, d_{not\ soc.\ pla.}) \geq d), \sigma)$

We are almost there. Now, we need to deal with the *if then else* statement, max, and the fact that we have a vacuous sampling statement relating to the parameter p, which is now not used anywhere. We have three δ-rules that deal with these from Section 3.3, repeated here:

```
ite, maxes, cleanUp :: DeltaRule

ite = \case
  ITE Tr x y -> Just x
  ITE Fa x y -> Just y
  _          -> Nothing
```

```
maxes = \case
   Max (Lam y (GE x (Var y'))) | y' == y -> Just x
   _                                     -> Nothing

cleanUp = \case
  Let v m k
    | sampleOnly m && v `notElem` freeVars k -> Just k
  _                                          -> Nothing
```

Taking these into account results in the final λ-term in (74).

(74) $d_{soc.\ pla.} \sim \mathcal{N}(165, 5)\,\mathsf{T}[0, \infty]$
$\mathcal{N}(d_{soc.\ pla.}, \sigma)$

From λ-terms to Stan parameters

It is fairly straightforward to translate the program in (74) into a simple Stan program. Specifically, PDS outputs the following kernel model:

```
model {
  // FIXED EFFECTS
  w ~ normal(165.0, 5.0) T[0,];

  // LIKELIHOOD
  target += normal_lpdf(y | w, sigma);
}
```

This is the semantic core—it captures the essential degree-based semantics where `w` represents the degree on the height scale. But reality is more complicated: we need random effects, the ability to model censored data, and proper indexing for multiple items and participants. This gap between the kernel model and a full statistical implementation represents ongoing research: how to get from here (PDS output) to here (actual implementation).

A fuller model with analyst augmentations for random effects would look the following one:

```
model {
  // PRIORS (analyst-added)
```

```
  sigma_epsilon_guess ~ exponential(1);
  sigma_e ~ beta(2, 10);

  // FIXED EFFECTS (PDS kernel)
  mu_guess ~ normal(165, 5) T[0,];

  // RANDOM EFFECTS (analyst-added)
  z_epsilon_guess ~ std_normal();

  // LIKELIHOOD (PDS kernel with modifications)
  for (i in 1:N_data) {
    y[i] ~ normal(mu_guess[item[i]]
      + epsilon_guess[participant[i]], sigma_e);
  }
}
```

While including the basic kernel model, this more sophisticated model adds statistical machinery for real data: hierarchical priors, random effects, and indexed parameters—meanwhile, note that we don't deal even here with the censored-data approach discussed above. Thus the kernel captures the core semantic computation–degrees on scales—while the augmentations handle the realities of experimental data.

This scale-norming model establishes how PDS transforms degree questions into parameter inference. Next, we'll see how this extends to modeling the vagueness inherent in gradable adjective judgments.

4.2.6 Modeling vagueness

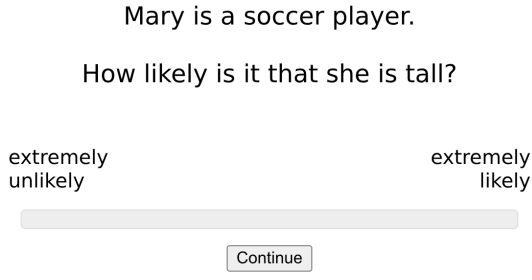
Our next model addresses how speakers reason about the likelihood that gradable adjectives apply to various objects. We'll start by describing an experiment that probes vague inferences, as well as a realistic model of the resulting data. Following our strategy for describing the scale-norming model, we'll build the model up block-by-block, explaining each part. Then, we'll turn to an analysis of a trial of the experiment in PDS, and we'll show which components of the our proposed model correspond to the parts of the resulting PDS kernel model, and which are extensions by the analyst.

participant	adjective	condition	response
1	quiet	high	0.82
1	wide	low	0.34
1	deep	mid	0.77

Figure 22 Example data from the adjectival inference experiment.

4.2.7 Understanding the experimental setup

In this task, which we dub the *adjectival inference* task, we provide experimental participants with prompts like the following one:



Here, we wish to assess the degree to which people believe an adjective applies to some object, given the distribution over degrees on the scale associated with the adjective that represents their uncertainty about where the object falls. Since this task dovetails with the scale-norming task, we employ the same six adjectives, with three possible context sentences for each one—all that has changed here is the prompt.

Before diving into the Stan code, let's consider how we'll represent the adjectival inference data. Three example rows of data are provided in Figure 22, where each row represents a single judgment.

4.2.8 The structure of the Stan program

Let's build up our adjectival inference model block-by-block. Since we introduced Stan's basic architecture in Section 4.2.4, we'll focus here on what's different when we model judgments about likelihood.

4.2.9 The data block

The data block for adjectival inference mostly mirrors that for the scale-norming model, since we'll be encoding distinct sets of parameters for each adjective-condition pair. What's different is that here we also obtain estimates for the mean and standard deviation of the distribution of guesses associated with each item, where we assume that these distributions are normal (and lower-truncated at 0). Indeed, these parameters may be obtained from the scale-norming model—in particular, they can be calculated as the posterior means and standard deviations of `mu_guess`.

```
data {
  int<lower=1> N_item;        // number of items
  int<lower=1> N_participant; // number of participants
  int<lower=1> N_data;        // responses in (0,1)
  int<lower=1> N_0;           // boundary responses at 0
  int<lower=1> N_1;           // boundary responses at 1

  // Response data:
  vector<lower=0, upper=1>[N_data] y;  // slider responses

  // Indexing arrays for responses...
  array[N_data] int<lower=1, upper=N_item> item;
  array[N_0] int<lower=1, upper=N_item> item_0;
  array[N_1] int<lower=1, upper=N_item> item_1;
  array[N_data] int<lower=1, upper=N_participant> participant;
  array[N_0] int<lower=1, upper=N_participant> participant_0;
  array[N_1] int<lower=1, upper=N_participant> participant_1;

  // Guess distribution parameters...
  array[N_item] real<lower=0> mu_guess0;
  array[N_item] real<lower=0> sigma_guess;
}
```

4.2.10 The parameters block

The parameters capture both semantic quantities and statistical variation:

```
parameters {
  // SEMANTIC PARAMETERS
```

```
  // Each item has a degree standard threshold:
  vector<lower=0, upper=1>[N_item] d;

  // PARTICIPANT VARIATION

  // How much participants vary in their guesses:
  real<lower=0> sigma_epsilon_mu_guess;
  // Each participant's standardized deviation:
  vector[N_participant] z_epsilon_mu_guess;

  // RESPONSE NOISE
  real<lower=0, upper=1> sigma_e;

  // CENSORED DATA
  array[N_0] real<upper=0> y_0;  // latent values for 0s
  array[N_1] real<lower=1> y_1;  // latent values for 1s
}
```

The crucial semantic quantity of note is `d`: the standard threshold associated with each item; e.g., how tall soccer players have to be to be considered tall.

4.2.11 The transformed parameters block

This block determines how to compute semantic judgments from the parameters defined above:

```
transformed parameters {
  // Convert standardized participant effects to natural scale:
  vector[N_participant] epsilon_mu_guess =
    sigma_epsilon_mu_guess * z_epsilon_mu_guess;

  // Set up participant-adjusted guess means for each item:
  vector[N_data] mu_guess;

  // Define these means:
  for (i in 0:N_data) {
    mu_guess[i] =
      mu_guess0[item[i]] + epsilon_mu_guess[participant[i]];
  }

  // Do the same for censored data...
```

```
vector[N_0] mu_guess_0;
vector[N_1] mu_guess_1;

for (i in 0:N_0) {
  mu_guess_0[i] =
    mu_guess0[item_0[i]] + epsilon_mu_guess[participant_0[i]]];
}

for (i in 0:N_0) {
  mu_guess_1[i] =
    mu_guess0[item_1[i]] + epsilon_mu_guess[participant_1[i]]];
}

// Finally, compute the predicted responses...
vector<lower=0, upper=1>[N_data] response;
vector<lower=0, upper=1>[N_0] response_0;
vector<lower=0, upper=1>[N_1] response_1;

// For each response in (0,1):
for (i in 1:N_data) {
  // KEY SEMANTIC COMPUTATION:
  // P(adjective applies) = P(degree > threshold)
  // Using normal CDF for smooth threshold crossing
  response[i] =
    (1 - normal_cdf(d[item[i]] |
      mu_guess[i], sigma_guess[item[i]]))
    / (1 - normal_cdf(0 |
      mu_guess[i], sigma_guess[item[i]]));
}

// Repeated for the censored data...
for (i in 1:N_0) {
  // KEY SEMANTIC COMPUTATION:
  // P(adjective applies) = P(degree > threshold)
  // Using normal CDF for smooth threshold crossing
  response_0[i] =
    1 - normal_cdf(d[item_0[i]] |
      mu_guess_0[i], sigma_guess[item_0[i]])
    / (1 - normal_cdf(0 |
      mu_guess_0[i], sigma_guess[item_0[i]]));
}
for (i in 1:N_1) {
```

```
    // KEY SEMANTIC COMPUTATION:
    // P(adjective applies) = P(degree > threshold)
    // Using normal CDF for smooth threshold crossing
    response_1[i] =
      1 - normal_cdf(d[item_1[i]] |
        mu_guess_1[i], sigma_guess[item_1[i]]
      / (1 - normal_cdf(0 |
        mu_guess[i], sigma_guess[item_1[i]])));
  }
}
```

The parameter `response` is the crucial one.

```
  response[i] =
      (1 - normal_cdf(d[item[i]] |
        mu_guess[i], sigma_guess[item[i]]))
      / (1 - normal_cdf(0 |
        mu_guess[i], sigma_guess[item[i]]));
```

Its definition computes the likelihood that the adjective applies, by taking the mass of the guess distribution that resides above the standard threshold as a proportion of this entire distribution—hence, a value in $[0, 1]$. In particular, we take the cumulative distribution of the guess distribution up to the threshold and subtracting it from 1, then re-normalizing the result to account for the fact that no mazz resides below 0, by hypothesis.

4.2.12 The model block

The model block specifies our priors and likelihood:

```
model {
  // PRIORS

  // Participant variation...
  sigma_epsilon_mu_guess ~ exponential(1);
  z_epsilon_mu_guess ~ std_normal();

  // LIKELIHOOD

  // Observed responses:
```

```
  for (i in 1:N_data) {
    y[i] ~ normal(response[i], sigma_e);
  }

  // Censored responses:
  for (i in 1:N_0) {
    y_0[i] ~ normal(response_0[i], sigma_e);
  }
  for (i in 1:N_1) {
    y_1[i] ~ normal(response_1[i], sigma_e);
  }
}
```

The likelihood connects our semantic computation (`response`) to both the observed and the censored data.

4.2.13 The complete model

Here, we give the complete adjectival inference model:

```
data {
  int<lower=1> N_item;        // number of items
  int<lower=1> N_participant; // number of participants
  int<lower=1> N_data;        // responses in (0,1)
  int<lower=1> N_0;           // boundary responses at 0
  int<lower=1> N_1;           // boundary responses at 1

  // Response data:
  vector<lower=0, upper=1>[N_data] y;  // slider responses

  // Indexing arrays for responses...
  array[N_data] int<lower=1, upper=N_item> item;
  array[N_0] int<lower=1, upper=N_item> item_0;
  array[N_1] int<lower=1, upper=N_item> item_1;
  array[N_data] int<lower=1, upper=N_participant> participant;
  array[N_0] int<lower=1, upper=N_participant> participant_0;
  array[N_1] int<lower=1, upper=N_participant> participant_1;

  // Guess distribution parameters...
  array[N_item] real<lower=0> mu_guess0;
  array[N_item] real<lower=0> sigma_guess;
```

```
}

parameters {
  // SEMANTIC PARAMETERS

  // Each item has a degree standard threshold:
  vector<lower=0, upper=1>[N_item] d;

  // PARTICIPANT VARIATION

  // How much participants vary in their guesses:
  real<lower=0> sigma_epsilon_mu_guess;
  // Each participant's standardized deviation:
  vector[N_participant] z_epsilon_mu_guess;

  // RESPONSE NOISE
  real<lower=0, upper=1> sigma_e;

  // CENSORED DATA
  array[N_0] real<upper=0> y_0;  // latent values for 0s
  array[N_1] real<lower=1> y_1;  // latent values for 1s
}

transformed parameters {
  // Convert standardized participant effects to a natural scale:
  vector[N_participant] epsilon_mu_guess =
    sigma_epsilon_mu_guess * z_epsilon_mu_guess;

  // Set up participant-adjusted guess means for each item:
  vector[N_data] mu_guess;

  // Define these means:
  for (i in 0:N_data) {
    mu_guess[i] =
      mu_guess0[item[i]] + epsilon_mu_guess[participant[i]];
  }

  // Do the same for censored data...
  vector[N_0] mu_guess_0;
  vector[N_1] mu_guess_1;

  for (i in 0:N_0) {
```

```
  mu_guess_0[i] =
    mu_guess0[item_0[i]] + epsilon_mu_guess[participant_0[i]]];
}

for (i in 0:N_0) {
  mu_guess_1[i] =
    mu_guess0[item_1[i]] + epsilon_mu_guess[participant_1[i]]];
}

// Finally, compute the predicted responses...
vector<lower=0, upper=1>[N_data] response;
vector<lower=0, upper=1>[N_0] response_0;
vector<lower=0, upper=1>[N_1] response_1;

// For each response in (0,1):
for (i in 1:N_data) {
  // KEY SEMANTIC COMPUTATION:
  // P(adjective applies) = P(degree > threshold)
  // Using normal CDF for smooth threshold crossing
  response[i] =
    (1 - normal_cdf(d[item[i]] |
      mu_guess[i], sigma_guess[item[i]]))
    / (1 - normal_cdf(0 |
      mu_guess[i], sigma_guess[item[i]]));
}

// Repeated for the censored data...
for (i in 1:N_0) {
  // KEY SEMANTIC COMPUTATION:
  // P(adjective applies) = P(degree > threshold)
  // Using normal CDF for smooth threshold crossing
  response_0[i] =
    (1 - normal_cdf(d[item_0[i]] |
      mu_guess_0[i], sigma_guess[item_0[i]]))
    / (1 - normal_cdf(0 |
      mu_guess_0[i], sigma_guess[item_0[i]]));
}
for (i in 1:N_1) {
  // KEY SEMANTIC COMPUTATION:
  // P(adjective applies) = P(degree > threshold)
  // Using normal CDF for smooth threshold crossing
  response_1[i] =
```

```
      (1 - normal_cdf(d[item_1[i]] |
        mu_guess_1[i], sigma_guess[item_1[i]])
      / (1 - normal_cdf(0 |
        mu_guess[i], sigma_guess[item_1[i]])));
  }
}

model {
  // PRIORS

  // Participant variation...
  sigma_epsilon_mu_guess ~ exponential(1);
  z_epsilon_mu_guess ~ std_normal();

  // LIKELIHOOD

  // Observed responses:
  for (i in 1:N_data) {
    y[i] ~ normal(response[i], sigma_e);
  }

  // Censored responses:
  for (i in 1:N_0) {
    y_0[i] ~ normal(response_0[i], sigma_e);
  }
  for (i in 1:N_1) {
    y_1[i] ~ normal(response_1[i], sigma_e);
  }
}

generated quantities {
  vector[N_data] ll; // log-likelihoods
  // definition:
  for (i in 1:N_data) {
    if (y[i] >= 0 && y[i] <= 1)
      ll[i] = normal_lpdf(
                y[i] |
                response[i],
                sigma_e
                );
    else
      ll[i] = negative_infinity();
```

```
s1, q2, discourse2 :: Term

s1         = getSemanticsAdjectives
               [ "Mary", "is", "a", "soccer player" ]

q2         = getSemanticsAdjectives
               [ "how", "likely", "is it that"
               , "Mary", "is", "tall" ]

discourse2 = assert s1 >>> ask q2
```

Figure 23 A Haskell-encoded discourse for the adjectival inference task.

```
  }
}
```

This model treats each item as having a standard threshold, with participants making likelihood judgments by taking into account their uncertainty about where the relevant object lies on the relevant scale.

4.2.14 PDS to Stan

Which components of this model are derivable within PDS? Again, we should define a PDS representation—this time, of the adjectival inference task. Thus we ought to analyze the discourse in (75).

(75) a. Mary is a soccer player.

b. How likely is it that she is tall?

(75) features the same context sentence as the one used to develop the previous model: (75a)/(63a). What's changed is the question prompt, (75b). Following our analysis of the discourse representing trials in the scale-norming experiment, we'll develop a PDS model of the adjectival inference experiment by sequencing an assertion with a question prompt, as shown in Figure 23. We'll work through the semantics of the question prompt, next, with specific attention paid to *likely* and the positive form of *tall*.

4.2.15 Working through the discourse

Recall the lexical entry for *likely* provided in (53) of Section 4.1.

(53)
$$\overset{likely}{\boxed{\begin{array}{l}\texttt{do}\ cg \leftarrow \texttt{view}_{\texttt{CG}} \\ \quad \boxed{\lambda\phi, d, i.\texttt{Pr}\left(\begin{array}{|c|}\hline i' \sim cg \\ \hline p(i') \\ \hline \end{array}\right) \geq d)}\end{array}}} \vdash ap_{mdl}\backslash d/s$$

Here, we will improve on this lexical entry to take into account the semantics of the gradable adjective; in particular, we wish for the probability of *likely*'s prejacent to be calculated with a fixed value in mind for the standard threshold associated with the adjective, rather than, say, by incorporating a distribution over such standard thresholds.

This choice is empirically motivated. The question in (75b), for instance, is odd in a context in which it is known that Mary is 170 centimeters tall—a fact which is unexpected if *likely* computes probabilities on the basis of uncertainty about the adjective's standard threshold. The argument for this claim is as follows. If *likely* computes probabilities based, in part, on uncertainty about the standard threshold associated with the adjective, then it should be possible to produce a set of likelihoods—i.e., a meaning for (75b)—from the distribution over standard thresholds encoding this uncertainty. But if *likely* computes probabilities without taking this distribution into account, then the relevant set of likelihoods is a singleton set if Mary's height is known (i.e., fixed)— it is $\{1\}$ if her height exceeds this degree and $\{0\}$ if it does not—possibly lending some leverage to an account of the observed oddity (which, to be sure, we have not fully spelled out here—it still needs to be said why such singleton-set denotations result in such odd sounding questions).

To account for *likely*'s choosiness with respect to the uncertainty it takes into account when computing probabilities, we can assign it the meaning in (76), also rendered in Haskell.

(76)
$$\overset{likely}{\boxed{\begin{array}{l}\texttt{do}\ cg \leftarrow \texttt{view}_{\texttt{CG}} \\ \quad \boxed{\lambda\phi, d, i'.\texttt{Pr}\left(\begin{array}{|c|}\hline i'' \sim cg \\ \hline p(i_{w_{i''}, c_{i'}}) \\ \hline \end{array}\right) \geq d)}\end{array}}} \vdash ap_{mdl}\backslash d/s$$

```
lexLikely :: Lexicon SynSem
lexLikely = \case
  "likely" ->
    [ SynSem {
        syn = apMdl \\ d // s,
        sem = ty adjSig (
                lam s (purePP (lam p (lam d (lam i (
                  SCon "(≥)" @@ (
                    Pr (let' j (CG s) (
                      Return (p @@
                        (overwrite contextParams i j)
                      ) ) ) ) @@ d) ) ) ) @@ s) )
      } ]
  _       -> []
```

This lexical entry calls a function— `overwrite` —and a list of names of constants— `contextParams` —which are defined as follows:

```
overwrite :: [String] -> Term -> Term -> Term
overwrite []       _ j = j
overwrite (c : cs) i j =
  Upd c (LkUp c i) (overwrite cs i j)

contextParams :: [String]
contextParams = ["d_tall"]
```

As defined, `overwrite` takes a list of names of constants and two indices, and it writes the value that that constant is assigned by the first index onto the second index, effectively erasing any preexisting values that the second index assigns to that constant. This has the result that when the probability operator is applied to the proposition denoted by *likely*'s prejacent, evaluated against the distribution of indices encoded in the common ground, the value of the standard threshold associated with *tall* is held fixed.

In our notation, the overwriting operation is denoted $i_{w_{i''},c_{i'}}$: $w_{i''}$ can be viewed as the "possible world" associated with the index i''—i.e., the constants associated with world knowledge (e.g., height)—while $c_{i'}$ can be called the "context" of the index i'—i.e., all the other constants; the plain i with subscripts—$i_{w_{i''},c_{i'}}$—then denotes a new index whose possible world is gotten from i'', and whose context is gotten from i'. As shown, we may compute such indices

by fixing in advance what constants we take to be associated with possible worlds, and what constants we take to be associated with contexts. Here, we've encoded the hypothesis that d_{tall} is associated contexts; that is, that it is one of the constants whose value *likely* overwrites.

Now for the lexical entry for the positive form of *tall*:

(77) $\overset{tall}{\boxed{\lambda x, i.\mathsf{height}(i)(x) \geq \mathsf{d}_{tall}}} \vdash ap$

```
lexTallPos :: Lexicon SynSem
lexTallPos = \case
  "tall" ->
    [ SynSem {
        syn = Base "ap",
        sem = ty tauAdj (lam s (
                purePP (lam x (lam i (
                  SCon "(≥)" @@
                    (sCon "height" @@ i @@ x) @@
                    (sCon "d_tall" @@ i) ) ) ) @@
                  s) )
      } ]
  _       -> []
```

These meanings yield the result for `discourse2` in (78).

(78) $\mathtt{do}\ cg \leftarrow \mathtt{view}_{\mathsf{CG}}$
$\mathtt{set}_{\mathsf{CG}}\left(\begin{array}{l} i \sim cg \\ \mathsf{observe}(\mathsf{soc\_pla}(i)(\mathsf{m})) \\ \boxed{i} \end{array}\right)$
$cg \leftarrow \mathtt{view}_{\mathsf{CG}}$
$\mathtt{push}_{\mathsf{QUD}}(\lambda d, i'.\mathsf{Pr}\left(\begin{array}{l} i'' \sim cg \\ \boxed{\mathsf{height}(\mathsf{m})(i_{w_{i''},c_{i'}}) \geq \mathsf{d}_{tall}(i_{w_{i''},c_{i'}})} \end{array}\right) \geq d)$

Note the occurences of the derived index $i_{w_{i''},c_{i'}}$ in the denotation of the question prompt in (78). To simplify these occurrences—as well as the set and view statements—we might apply `betaDeltaNormal states` to (78) to obtain (a λ-term β-equivalent to) (79).

(79) $\texttt{do}\, cg \leftarrow \texttt{view}_{\texttt{CG}}$
$$\texttt{set}_{\texttt{CG}}\left(\begin{array}{l} i \sim cg \\ \mathsf{observe}(\mathsf{soc\_pla}(i)(\mathsf{m})) \\ \boxed{i} \end{array}\right)$$
$$\texttt{push}_{\texttt{QUD}}(\lambda d, i'.\mathsf{Pr}\left(\begin{array}{l} i \sim cg \\ \mathsf{observe}(\mathsf{soc\_pla}(i)(\mathsf{m})) \\ \boxed{\mathsf{height}(i) \geq \mathsf{d}_{tall}(i')} \end{array}\right) \geq d)$$

Here, we can see the full effect of making the meaning of *likely* sensitive to the common ground: since the previous assertion (75a) has modified it via an observation, this observation has snuck itself into the denotation of the question prompt! Likelihoods are therefore computed based on the observation that Mary is a soccer player.

As for the scale-norming model, we are after a response distribution, which we can again construct from `respondAdj`:

```
adjInferenceExample :: Term
adjInferenceExample = respondAdj
                        adjInferencePrior
                        discourse2
```

The basic setup is the same, except that we must produce this distribution from `discourse2` and a new prior: `adjInferencePrior`. We provide a definition of this prior distribution in (80).

(80)
$$\mathsf{upd\_CG}\left(\begin{array}{l} d_{soc.\ pla.} \sim \mathcal{N}(165,5)\,\mathsf{T}[0,\infty] \\ d_{not\ soc.\ pla.} \sim \mathcal{N}(160,5)\,\mathsf{T}[0,\infty] \\ \theta_{tall} \sim \mathcal{N}(170,3)\,\mathsf{T}[0,\infty] \\ p \sim \mathsf{Logit\_Normal}(0,1) \\ \tau_{soc.\ pla.} \sim \mathsf{Bernoulli}(p) \\ \boxed{\begin{array}{l} \mathsf{upd\_d}_{tall}(\theta_{tall})(\\ \quad \mathsf{upd\_height}(\\ \quad\quad \lambda x.\mathsf{if\_then\_else}(\tau_{soc.\ pla.}, d_{soc.\ pla.}, d_{not\ soc.\ pla.}))(\\ \quad\quad\quad \mathsf{upd\_soc\_pla}(\lambda x.\tau_{soc.\ pla.})(@) \\ \quad\quad)) \end{array}} \end{array}\right)(\epsilon)$$

This prior distribution over indices is almost exactly like `scaleNormingPrior`; the difference is that we now also sample a new parameter θ_{tall}, which provides the value of the constant d_{tall} featured in the meaning of *tall*.

δ-rules and semantic computation

Putting it all together, we obtain the following definition of `adjInferenceExample` in (81).

(81)
$$
\begin{aligned}
s \sim\ & \mathsf{upd\_CG}\left(\begin{array}{l} d_{soc.\ pla.} \sim \mathcal{N}(165,5)\,\mathsf{T}[0,\infty] \\ d_{not\ soc.\ pla.} \sim \mathcal{N}(160,5)\,\mathsf{T}[0,\infty] \\ \theta_{tall} \sim \mathcal{N}(170,3)\,\mathsf{T}[0,\infty] \\ p \sim \mathsf{Logit\_Normal})(0,1) \\ \tau_{soc.\ pla.} \sim \mathsf{Bernoulli}(p) \\ \mathsf{upd\_d}_{tall}(\theta_{tall})(\\ \quad \mathsf{upd\_height}(\\ \quad\quad \lambda x.\mathsf{if\_then\_else}(\tau_{soc.\ pla.}, d_{soc.\ pla.}, d_{not\ soc.\ pla.}))(\\ \quad\quad\quad \mathsf{upd\_soc\_pla}(\lambda x.\tau_{soc.\ pla.})(@) \\ \quad\quad)) \end{array}\right)(\epsilon) \\
s' \sim\ & \mathsf{upd\_CG}\left(\begin{array}{l} i \sim \mathsf{CG}(s) \\ \mathsf{observe}(\mathsf{soc\_pla}(i)(\mathsf{m})) \\ i \end{array}\right)(s) \\
s'' \sim\ & \mathsf{push\_QUD}(\lambda d, i'.\mathsf{Pr}\left(\begin{array}{l} i' \sim \mathsf{CG}(s') \\ \mathsf{height}(\mathsf{m})(i_{w_{i''},c_{i'}}) \geq \mathsf{d}_{tall}(i_{w_{i''},c_{i'}}) \end{array}\right) \geq d)(s') \\
i' \sim\ & \mathsf{CG}(s'') \\
& \mathcal{N}(\mathsf{max}(\lambda d.\pi_1(\mathsf{pop\_QUD}(s''))(d)(i')), \sigma)
\end{aligned}
$$

Again, we need to apply δ-rules, in order to obtain a straightforward definition of a probability distribution which can be rendered as a Stan model.

Though we won't go through every step involved, one can see that if we layer β-reduction with the collection of δ-rules that was used to reduce `scaleNormingExample`, we obtain the result in (82), for similar reasons.

(82)
$$
\begin{aligned}
& \theta_{tall} \sim \mathcal{N}(170,3)\,\mathsf{T}[0,\infty] \\
& \mathcal{N}(\mathsf{Pr}\left(\begin{array}{l} d_{soc.\ pla.} \sim \mathcal{N}(165,5)\,\mathsf{T}[0,\infty] \\ d_{soc.\ pla.} \geq \theta_{tall} \end{array}\right), \sigma)
\end{aligned}
$$

At this point, we require a δ-rule that can be used to compute the probability which provides the location parameter of the likelihood. Though it is a bit *ad hoc*, we are free to introduce the following sound rule:

```
probabilities :: DeltaRule
probabilities = \case
  Pr (
    Let v (Truncate (Normal x y) a Inf) (
      Return (
        GE (Var v') t
      ) ) ) | v' == v ->
    Just ((1 - NormalCDF x y t) / (1 - NormalCDF x y 0))
  _                   -> Nothing
```

This rule says that computing the probability that values of a normal distribution, truncated at $[0, \infty]$, fall above some threshold (say, θ_{tall}) is a matter of computing 1 minus the cumulative mass of a normal distribution up to that threshold and dividing by 1 minus the cumulative mass of the distribution up to 0 (to re-normalize the result). If we opt to include `probabilities` among our other δ-rules, we obtain (83) instead of (82).

(83) $\theta_{tall} \sim \mathcal{N}(170, 3)\,\mathsf{T}[0, \infty]$
$\mathcal{N}((1 - \mathsf{CDF}_{\mathcal{N}(165,5)}(\theta_{tall}))/(1 - \mathsf{CDF}_{\mathcal{N}(165,5)}(0)), \sigma)$

Thus remarkably, this distribution—easily computed in Stan—is equivalent to the one defined in (81).

From λ-terms to Stan parameters

Again, the resulting PDS kernel model is easily translated into idiomatic Stan code:

```
model {
  // FIXED EFFECTS
  v ~ normal(170.0, 3.0) T[0,];

  // LIKELIHOOD
  target += normal_lpdf(y |
    (1 - normal_cdf(v, 165.0, 5.0))
      / (1 - normal_cdf(0, 165.0, 5.0)),
    sigma);
}
```

As for the scale-norming model, we can develop a fuller model by adding augmentations for random effects:

```
model {
  // PRIORS (analyst-added)
  sigma_epsilon_guess ~ exponential(1);
  sigma_e ~ beta(2, 10);

  // FIXED EFFECTS (PDS kernel)
  theta ~ normal(170.0, 3.0) T[0,];

  // RANDOM EFFECTS (analyst-added)
  z_epsilon_guess ~ std_normal();

  // LIKELIHOOD (PDS kernel with modifications)
  for (i in 1:N_data) {
    target += normal_lpdf(y[i] |
      (1 - normal_cdf(theta[item[i]]
        + epsilon_theta[participant[i]],
        165.0, 5.0))
        / (1 - normal_cdf(0, 165.0, 5.0)),
      sigma_e);
    }
}
```

Indeed, it is worth comparing this output to the full model stated in Section 4.2.13.

4.2.16 Interim conclusion

Here, we've illustrated how PDS may be used to model *unresolved* uncertainty: both the judgments related to world knowledge—guesses about various objects' measurements on different scales—and those related to vagueness—judgments of the likelihood that vague adjectives apply to an object—produce judgments which can vary continuously from trial to trial. In the next section, we turn to a phenomenon which can seen as a candidate for exemplifying *resolved* uncertainty; specifically, the judgments of factivity triggered by certain clause-selecting predicates.

4.3 Factivity inferences and resolved uncertainty

Having explored how PDS handles expected gradience in adjectives, we now turn to the phenomenon of factivity, a domain in which gradience poses a deeper

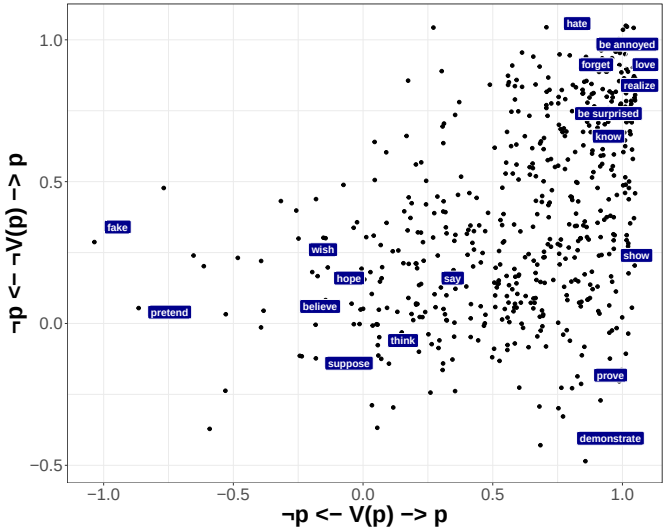


Figure 24 An aggregate measure of factivity, derived from the MegaVeridicality dataset of White and Rawlins 2018. Points are jittered (0.05) to avoid over-plotting. The x-axis corresponds to mean scores for affirmative contexts, the y-axis for negative contexts. Each point represents a predicate. Notably, clusters of predicates appear continuous and gradient, not discrete.

theoretical puzzle. While the gradience in adjective meanings arises from well-understood sources like vagueness and contextual variation, the gradience observed in judgments aimed at measuring factivity is more difficult to explain.

Before diving into any computational models, let's investigate what makes factivity special. The term *factivity* describes the property of certain clause-selecting predicates which are associated with the presupposition that their complement clauses are true, even when the predicate is embedded under various operators, such as negation. Compare these sentences in (84).

(84) a. Jo loved that Mo left.

b. Jo didn't love that Mo left.

c. Did Jo love that Mo left?

d. If Jo loved that Mo left, she'll won't be upset that Bo left.

In all of these cases, there's a strong inference that Mo actually left. We say that this inference *projects* through negation, questions, and conditional sentences.

Contrast *love* with the non-factive predicates in (85).

(85) a. Jo {thought, believed, said} that Mo left.

b. Jo didn't {think, believe, say} that Mo left.

In this case, the negated (85b) doesn't commit us to Mo having left. Even given only these examples, the contrast feels sharp: predicates seem to be either associated with factive presuppositions or not associated with them at all.

Despite such clear judgments, the traditional picture they represent has been challenged by experimental work showing substantial gradience in projection inferences (i.e., inferences, like those exempified by (84), which project). Some predicates (e.g., *love*) are almost always associated with projection, and others (e.g., *think*) rarely trigger it, but many fall somewhere in between. For instance, White and Rawlins 2018 measured veridicality inferences for more than 700 predicates in their MegaVeridicality dataset—visualized in Figure 24.

What explains this gradience? There are two questions that we should disentangle before directly addressing this question. First, does the gradience we observe suggest that there aren't any *classes* (or *sub-classes*) of factive predicates? Second, are the veridicality inferences which are triggered *on particular occasions* by particular expressions that contain particular clause-selecting predicates themselves gradient? Here, we will be concerned only with the latter question, which we believe PDS is uniquely positioned to address.[16] In particular, PDS can address *what kind of uncertainty* accounts for the gradience among inference judgments observed for clause-selecting predicates: resolved uncertainty, or unresolved uncertainty?

To put things in slightly different terms, PDS can help us precisely state two fundamentally different accounts of the gradience observed in data like that represented by Figure 24: the ones in (86).[17]

(86) a. **The Discrete-Factivity Hypothesis**: Factivity is a discrete property of expressions; speakers either interpret a predicate as triggering

[16] Prior work has addressed the first question (Kane, Gantt, & White, 2022). Kane et al. showed that the gradience exhibited by the factivity inference judgments of clause-selecting predicates is largely due to task effects and measurement noise. They show this by applying a clustering model to the inference judgment data—one that accounts for such noise—and which reveals many of the standard sub-classes of factive predicates; in particular, among the emotive factives, e.g., *love* and *hate*.

[17] See Grove and White (2024) for further discussion—as well as several empirical tests—of these two hypotheses.

a factive presupposition on a given occasion of use or they do not. Thus the gradience we observe among projection inference judgments reflects experiment-level measurements of the uncertainty that persists on particular occasions of use.

b. **The Wholly-Gradient Hypothesis**: There is no discrete property of uses of expressions determining whether they are factive or non-factive. Rather, predicates make gradient contributions to inferences about the truth of the clauses they select.

The discrete-factivity hypothesis is ultimately about which interpretation is relevant on particular occasions—it analogizes factivity as a property to lexical ambiguity. Thus different uses of *discover* might involve different lexical entries or different syntactic structures, only some of which are associated with a factive presupposition.

The alternative view represented by the wholly-gradient hypothesis is aligned with Tonhauser et al. (2018)'s Gradient Projection Principle: it treats projection as a continuous phenomenon in which the content of a complement clause projects "to the extent that it is not at-issue."

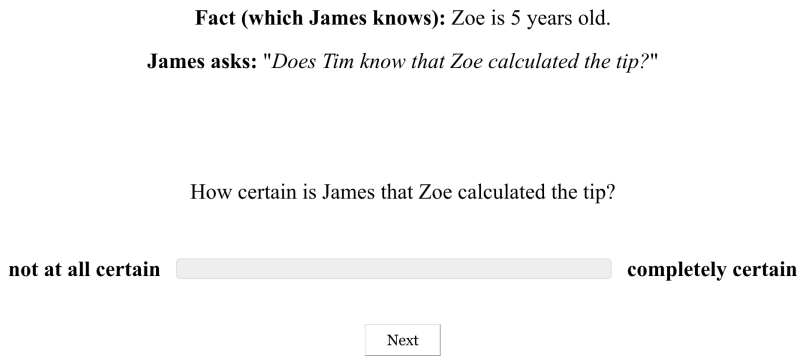
As we'll see, PDS allows us to implement both hypotheses as statistical models so that we may compare these models' empirical predictions. To keep things simple, we focus only on the kernel models that PDS produces, and we don't go into issues regarding parameters added by the researcher (but see Grove and White 2024). Before we show how to do this, let's look at an experimental paradigm that we can use to collect data relevant to these hypotheses.

4.3.1 Understanding the experimental setup

Here, the experimental task provides participants with a prompt following a context which is slightly more complex than the contexts featured in the scale-norming and adjectival inference tasks. This new context includes a statement of a *background fact*, together with a description of a scenario in which a clause-selecting predicate is used. Here is an example trial:

participant	predicate	fact type	clause number	response
1	acknowledge	7	high	1.0
1	say	3	high	0.09
1	discover	6	low	0.25

Figure 25 Example data from the factivity experiment.



The idea here is to probe participants' (un)certainty about the truth of the clause embedded by the relevant predicate, based on two pieces of information: (i) the background fact; and (ii) the choice of clause-selecting predicate featured in the context—*know*, in this case.

Grove and White (2025) originally developed this task by modifying stimuli featured in the experiments reported in Degen and Tonhauser 2021 and Degen and Tonhauser 2022—Grove and White's goal was to assess the extent to which modeling the semantics of the question prompt in inference tasks can improve models of the data collected in those tasks. Here, we describe PDS-generated models for Grove and White's task which could be used to investigate the two hypotheses introduced above, i.e., about whether the projection inferences that clause-selecting predicates like *know* give rise to are gradient or discrete—a question that Grove and White (2024) investigate using the stimuli of Degen and Tonhauser 2021. We choose to model the stimuli of Grove and White (2025), here, rather than those of Degen and Tonhauser (2021)/Grove and White (2024) because the latter use polar questions as prompts, while the former uses *how likely* questions, which are more well-suited to the slider-scale instrument used to answer the prompt.

Example data from this experiment is shown in Figure 25. A total of twenty clause-selecting predicates were tested (following Degen & Tonhauser,

2021, 2022), where each predicate was paired up with one of twenty possible complement clauses. Additionally, each the complement clause of each trial featured one of two possible facts: a "high" fact, and a "low" fact. The "high" fact was originally designed by Degen and Tonhauser (2021) to make the complement clause more likely to be true than the "low" fact; the example trial provided above, for instance, pairs *Zoe calculated the tip* with its corresponding "low" fact—the corresponding "high" fact, meanwhile, is *Zoe is a math major*. The columns representing the data encode the following information

- participant: which person made this judgment (participant 1, 2, etc.)
- predicate: which clause-selecting predicate is being tested
- clause numner: which of the 20 possible clauses is being used
- fact type: whether the background fact is "high" or "low"
- response: the participant's slider response (in [0, 1])

We use the next two subsections to describe models—generated in PDS—that encode the Discrete-Factivity Hypothesis and the Wholly-Gradient Hypothesis, respectively. As we show, these models reflect distinct choices about how to analyze the lexical semantics of the clause-selecting predicate *know*. We start with the Discrete-Factivity Hypothesis.

4.3.2 Discrete factivity in PDS

For both kinds of models, we need a type signature that can be used to type constants:

```
factSig :: Sig
factSig = \case
  Left "tau_know"     -> Just (ι :-> e :-> r)
  Left "upd_tau_know" -> Just ((e :-> r) :-> ι :-> ι)
  Left "five_yo"      -> Just (ι :-> e :-> t)
  Left "upd_five_yo"  -> Just ((e :-> t) :-> ι :-> ι)
  Left "calc"         -> Just (ι :-> e :-> e :-> t)
  Left "upd_calc"     -> Just ((e :-> e :-> t) :-> ι :-> ι)
  Left "epi"          -> Just (ι :-> e :-> (ι :-> t) :-> t)
  Left "upd_epi"      -> Just ((e :-> (ι :-> t))
                               :-> ι :-> ι)
  Left "@"            -> Just ι
  Left "CG"           -> Just (σ :-> P ι)
  Left "upd_CG"       -> Just (P ι :-> σ :-> σ)
```

```
  Left "pop_QUD"       -> Just (σ :-> (ι : -> α :-> t)
                                  :* TyCon "popQ" [ι, α, σ])
  Left "push_QUD"      -> Just ((ι :-> α :-> t) :-> σ
                                  :-> TyCon "Q" [ι, α, σ])
  Left "ϵ"             -> Just σ
  Left "t"             -> Just e
  Left "tip"           -> Just e
  Left "j"             -> Just e
  Left "z"             -> Just e
  Left "&"             -> Just (t :-> t -> t)
  Left "if_then_else" -> Just (t :* α :* α) :-> α
  Left "(≥)"           -> Just (r :-> r :-> r)
  Left "max"           -> Just ((r :-> t) :-> r)
  Left "Pr"            -> Just (P t :-> r)
  Left "Normal"        -> Just (r :* r :-> P r)
  Left "Logit_Normal" -> Just (r :* r :-> P r)
  Left "Truncate"      -> Just (P r :* r :* r -> P r)
  Left "∞"             -> Just r
  Right _              -> Just r
  _                    -> Nothing
```

Compared to the type signature used to model the two previous tasks, this one has constants for the new names (*Zoe*, *Tim*, and *James*), predicates (*5 years old*), and verbs (*calculate*); but crucially, it also types a constant `"epi"` which we use to state the meaning of the verb *know*, as well as the logical constant `"&"`, which we will use to help encode *know*'s veridicality entailment.

Our goal in developing both models will be to model the discourse represented by (87).

(87) a. Zoe is 5 years old.

b. Tim knows Zoe calculated the tip.

c. How certain is James that Zoe calculated the tip?

Thus we make a couple of simplifying choices. First, we model the labeled fact, which states that Zoe is 5 years old, as an assertion—more generally, we leave out the metalinguistic annotations for utterances that say how the discourse is structured. And second, rather than providing a semantics for clause-selecting predicates that causes their presuppositions to project out of questions, we treat these presuppositions as veridicality entailments and—correspondingly—the

question that James asks as another assertion. Making this move allows us to simplify the semantics for such predicates considerably, since we can avoid having to provide a compositional semantics of presupposition projection.

With these choices in mind, let's provide lexical entries for the various names (we treat *the tip* as a name for our purposes), as well as the verb *calculated* and the adjective phrase *5 years old*.

(88) a. $\overset{Zoe}{\boxed{\lambda k.k(\mathsf{m})}} \vdash s/(s\backslash np)$

```
lexZoe :: Lexicon SynSem
lexZoe = \case
  "Zoe" -> [ SynSem {
                syn = np,
                sem = ty factSig $
                  purePP (SCon "z")
              } ]
  _     -> []
```

b. $\overset{James}{\boxed{\lambda k.k(\mathsf{j})}} \vdash s/(s\backslash np)$

```
lexJames :: Lexicon SynSem
lexJames = \case
  "James" -> [ SynSem {
                 syn = s // (s \\ np),
                 sem = ty factSig $
                   purePP (lam x (x @@ (SCon "j")))
                 } ]
  _       -> []
```

c. $\overset{Tim}{\boxed{\lambda k.k(\mathsf{t})}} \vdash s/(s\backslash np)$

```
lexTim :: Lexicon SynSem
lexTim = \case
  "Tim" -> [ SynSem {
                 syn = np,
                 sem = ty factSig $
                   purePP (SCon "t")
                 } ]
  _     -> []
```

d. *the tip*
$\boxed{\text{tip}} \vdash np$

```
lexTip :: Lexicon SynSem
lexTip = \case
  "the tip" -> [ SynSem {
                   syn = np,
                   sem = ty factSig $
                     purePP (SCon "tip")
                  } ]
  _         -> []
```

e. *5 years old*
$\boxed{\lambda x, i.\text{five\_yo}(i)(x)} \vdash ap$

```
lex5yo :: Lexicon SynSem
lex5yo = \case
  "5 years old"
    -> [ SynSem {
           syn = ap,
           sem = ty factSig $
                   purePP (lam x (lam i (
                     SCon "five_yo" @@ i @@ x
                   ) ) )
         } ]
  _ -> []
```

f. *calculated*
$\boxed{\lambda x, y, i.\text{cal}(i)(x)(y)} \vdash s\backslash np/np$

```
lexCalculated :: Lexicon SynSem
lexCalculated = \case
  "calculated"
    -> [ SynSem {
           syn = s \\ np // np,
           sem = ty factSig $
                   purePP (lam x (lam i (
                     SCon "calc" @@ i @@ x
                   ) ) )
         } ]
  _ -> []
```

Three lexical entries remain. First, we need to provide a semantics for the modal adjective *certain* that captures the degrees of likelihood that we take the question prompt in (87c) to be querying. Second, we need a lexical entry for *know* that captures its potential for being factive (or, under the running implementation, veridical). Providing a semantics for *certain* can be considered pretty straightforward, if we model its contribution after that of *likely* in our previous models. That is, we can give it the lexical entry in (89).

(89) a. *certain*

$$\boxed{\begin{array}{l}\mathsf{do}\ cg \leftarrow \mathtt{view}_{\mathsf{CG}} \\ \quad \boxed{\lambda\phi, d, x, i'.\mathsf{Pr}\left(\begin{array}{l} i'' \sim cg \\ \boxed{p(i_{w_{i''}, c_{i'}})} \end{array}\right) \geq d)}\end{array}} \vdash ap\backslash d/s$$

b.

```
lexCertain :: Lexicon SynSem
lexCertain = \case
  "certain" ->
    [ SynSem {
        syn = ap \\ d // s,
        sem =
          ty factSig (
            lam s (
              purePP (
                lam p (lam d (lam x (lam i (
                  SCon "(≥)" @@ (
                    Pr (let' j (CG s) (
                      Return (p @@
                        (overwrite
                          contextParams i j)
                      ) ) ) @@ d) ) ) ) ) @@ s) )
    } ]
  _       -> []
```

Thus *certain* determines its subject's certainty by calculating a likelihood with respect to the common ground (indeed, the subject argument is simply thrown out, in this case). Note that this analysis of *certain* could be improved, not least in order to take into account the distinction between the degree scales that *likely* and *certain* employ (see, e.g., Klecha, 2012, for discussion).[18]

We also now require a new lexical entry for *how* that can combine with *certain*. This lexical entry, like the ones provided in (55) of Section 4.1 and (66a) of Section 4.2, just involves function chasing: it produces a degree abstraction, while reconstructing the meaning of the adjective into the "extraction site".

[18] Grove and White (2025), in particular, attempt to account for the difference in degree scales between *likely* and *certain*, finding that doing this improves models of the data collected by using *certain* in the question prompt.

(90) a. *how*

$$\boxed{\lambda f, g, \phi, d.g(f(\phi)(d))} \vdash q_{deg.}/s/(s/ap)/(ap\backslash d/s)$$

b.

```
lexHowFact :: Lexicon SynSem
lexHowFact = \case
"how" -> [ SynSem {
               syn =
                 qDeg // s // (s // ap) //
                   (ap \\ d // s),
               sem = ty adjSig $
                       purePP (lam x (lam y (
                         lam z (lam w (
                           y @@ (x @@ z @@ w)
                         ) ) ) ) )
             } ]
  _       -> []
```

Finally, the lexical entry we propose for the clause-selecting predicate *know* is given in (91); we take this lexical entry to reflect the Fundamental Discreteness Hypothesis (86a).

(91) a. *know*

$$\boxed{\begin{array}{l}\mathsf{do}\ b \leftarrow \mathtt{view}_{\tau_{know}} \\ \quad \boxed{\begin{array}{l}\lambda\phi, x, i.\mathsf{if\_then\_else}(\\ \quad b, \\ \quad \mathsf{epi}(i)(x)(\phi) \wedge \phi(i), \\ \quad \mathsf{epi}(i)(x)(\phi) \\ )\end{array}}\end{array}} \vdash ap\backslash d/s$$

b.
```
lexKnowDF :: Lexicon SynSem
lexKnowDF = \case
  "know" ->
    [ SynSem {
        syn = s \\ np // s,
        sem =
          ty factSig (
            lam s (
              purePP (lam p (lam x (lam i (
                ITE (TauKnow s)
                  (SCon "&" @@
                    ((SCon "epi" @@ i @@ x @@ p)
                      & p @@ i))
                  (SCon "epi" @@ i @@ x @@ p)
              ) ) ) ) ) )
      } ]
  _     -> []
```

(91) regards *know* as fundamentally semantically ambiguous. Specifically, a parameter *b*—determining whether *know* should be understood as contributing a veridicality entailment or not—is read from the state, and the entailment is encoded depending on whether *b* is T or F, via an *if then else* statement. If *b* is T, *know* means (a) that its agent is in the relevant epistemic relation with the prejacent, and (b) that the prejacent is true at the index of evaluation; meanwhile, if *b* is F, *know* contributes only the first entailment.

One can observe the contribution of this lexical entry by inspecting the derivation of (87b) in Figure 26. The meaning provided for the whole sentence ultimately does two things: first, it reads a boolean (T or F) parameter from the state; and second, it returns a proposition that says that Tim is in the relevant epistemic relation with the proposition that Zoe calculated the tip. Further, it contributions the additional entailment that Zoe indeed calculated the tip, but only if the boolean parameter is T.

The entire discourse in (87) can be seen to be the one in (92).

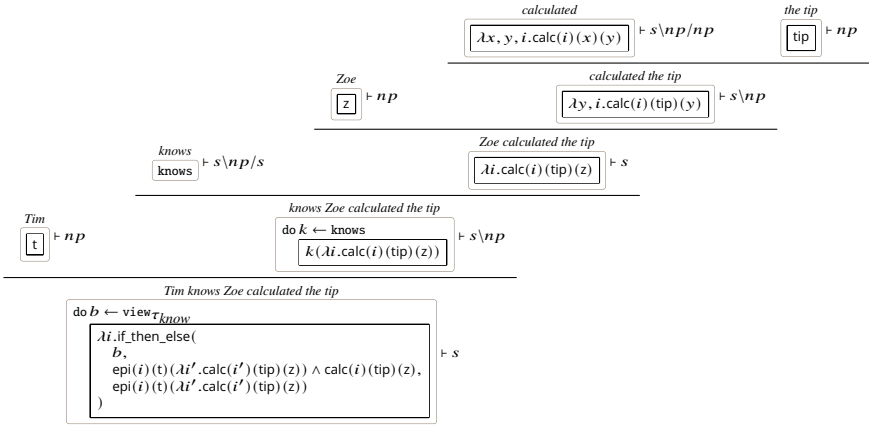


Figure 26 Derivation of *Tim knows Zoe calculated the tip*.

(92)
$$\begin{aligned}
&\mathtt{do}\ cg \leftarrow \mathtt{view}_{\mathsf{CG}}\\
&\mathtt{set}_{\mathsf{CG}}\left(\begin{array}{l} i \sim cg \\ \mathsf{observe}(\mathsf{five_yo}(i)(\mathsf{z})) \\ \boxed{i} \end{array}\right)\\
&b \leftarrow \mathtt{view}_{\tau_{know}}\\
&cg \leftarrow \mathtt{view}_{\mathsf{CG}}\\
&\mathtt{set}_{\mathsf{CG}}\left(\begin{array}{l} i \sim cg \\ \mathsf{observe}\left(\begin{array}{l} \mathsf{if_then_else}(\\ \quad b, \\ \quad \mathsf{epi}(i)(\mathsf{t})(\lambda i'.\mathsf{calc}(i')(\mathsf{tip})(\mathsf{z})) \wedge \mathsf{calc}(i)(\mathsf{tip})(\mathsf{z}), \\ \quad \mathsf{epi}(i)(\mathsf{t})(\lambda i'.\mathsf{calc}(i')(\mathsf{tip})(\mathsf{z})) \\ ) \end{array}\right) \\ \boxed{i} \end{array}\right)\\
&cg \leftarrow \mathtt{view}_{\mathsf{CG}}\\
&\mathtt{push}_{\mathsf{QUD}}(\lambda d, i'.\mathsf{Pr}\left(\begin{array}{l} i'' \sim cg \\ \boxed{\mathsf{calc}(i'')(\mathsf{tip})(\mathsf{z})} \end{array}\right) \geq d)
\end{aligned}$$

It features two updates to the common ground: one of these updates contributes the proposition that Zoey is five years old, and the other, the proposition that Tim knows she calculated the tip. Depending on whether the parameter b that *know* reads from the state is T or F, the second update will constrain the common ground to only include indices at which which Zoey calculated the tip.

Following these updates, the question prompt makes its contribution. Specif-

ically, it contributes a question that denotes a set of (minimum) degrees of likelihood that Zoey calculated the tip.

We can provide a response model for the discourse in (87), as in our previous examples, by defining a prior and a response function. For the response function, we'll employ the one presented in (50) of Section 3.2.6.

(50)

$$
\begin{aligned}
& \texttt{respond}^{\lambda x.\mathcal{N}(x,\sigma)\,\mathsf{T}[0,1]}_{\mathsf{max}}(bg)(m) \\
= \;& s \sim bg \\
& \langle \diamond, s' \rangle \sim m(s) \\
& i \sim \mathsf{CG}(s') \\
& \mathcal{N}(\mathsf{max}(\lambda d.\pi_1(\mathsf{pop\_QUD}(s'))(d)(i)), \sigma)
\end{aligned}
$$

This response function features a truncated normal distribution as its likelihood—truncated to between 0 and 1, in order to account for the slider scale endpoints. We use the slightly more sophisticated truncated likelihood to model the current dataset—in contrast to the vanilla normal used in the previous section (modified in the implementation to account for censored data)—because it more directly represents the strategy of Grove and White (2025) (who follow Grove and White (2024)).

The prior over discourse states we'll use for the discrete-factivity model is the one in (93)

(93)
$$
\begin{aligned}
&p \sim \mathsf{Logit\_Normal}(0,1)\\
&q \sim \mathsf{Logit\_Normal}(0,1)\\
&r \sim \mathsf{Logit\_Normal}(-2,1)\\
&t \sim \mathsf{Logit\_Normal}(2,1)\\
&u \sim \mathsf{Logit\_Normal}(0,1)\\
&b \sim \mathsf{Bernoulli}(u)\\
&\mathsf{upd\_CG}\left(\begin{array}{l}
c \sim \mathsf{Bernoulli}(p)\\
d \sim \mathsf{Bernoulli}(q)\\
e \sim \mathsf{Bernoulli}(r)\\
f \sim \mathsf{Bernoulli}(t)\\
\mathsf{upd\_five\_yo}(\lambda x.c)(\\
\quad \mathsf{upd\_epi}(\lambda \phi, x.d)(\\
\quad\quad \mathsf{upd\_calc}(\\
\quad\quad\quad \lambda x, y.\mathsf{if\_then\_else}(c,e,f)\\
\quad\quad)(@)\\
\quad))
\end{array}\right)(\mathsf{upd}\_\tau_{know}(b)(\epsilon))
\end{aligned}
$$

This prior distribution over states encodes a probability distribution over values for the parameter τ_{know} determining whether or not the content of the clause selected by *know* "projects" (i.e., whether or not it is entailed), as well as a probability distribution over values for the common ground. The common ground itself encodes a probability distribution over indices; it thus sets values, probabilistically, for the parameters five_yo, epi, and calc and writes them into the index.

The value of the parameter CG, moreover, is itself determined probabilistically; the Bernoulli distributions that characterize the common ground are parameterized by values sampled from logit-normal distributions. By inspecting how intensional constants are valued, one can see the probability distributions over their possible values are not independent. Specifically, whether or not Zoe calculated the tip is taken to depend, by an *if then else* statement, on whether or not she is five years old: if she is five years old (c), then her probability of having calculated the tip tends to decrease—it is r, which parameterizes the Bernoulli distribution from which e is sampled—compared to when she is not five years old—in which case, this probability is t, which parameterizes the Bernoulli distribution from which f is sampled. We have designed this prior with such common-sense knowledge in mind, but such choices are precisely the kind of thing which can be learned from experimental data (see Degen and Tonhauser 2021 and Grove and White 2024 for discussion of precisely these kinds of examples).

δ-rules and semantic computation

If we put it all together, the full model of the response distribution we get is β-equivalent to (94).

(94)
$$
\begin{aligned}
&p \sim \mathsf{Logit\_Normal}(0,1)\\
&q \sim \mathsf{Logit\_Normal}(0,1)\\
&r \sim \mathsf{Logit\_Normal}(-2,1)\\
&t \sim \mathsf{Logit\_Normal}(2,1)\\
&u \sim \mathsf{Logit\_Normal}(0,1)\\
&b \sim \mathsf{Bernoulli}(u)\\
&s \sim \mathsf{upd\_CG}\left(\begin{array}{l} c \sim \mathsf{Bernoulli}(p)\\ d \sim \mathsf{Bernoulli}(q)\\ e \sim \mathsf{Bernoulli}(r)\\ f \sim \mathsf{Bernoulli}(t)\\ \mathsf{upd\_five\_yo}(\lambda x.c)(\\ \quad \mathsf{upd\_epi}(\lambda \phi, x.d)(\\ \quad\quad \mathsf{upd\_calc}(\\ \quad\quad\quad \lambda x, y.\mathsf{if\_then\_else}(c,e,f)\\ \quad\quad)(@)\\ \quad))\end{array}\right)(\mathsf{upd\_}\tau_{know}(b)(\epsilon))\\
&s' \sim \mathsf{upd\_CG}\left(\begin{array}{l} i \sim \mathsf{CG}(s)\\ \mathsf{observe}(\mathsf{five\_yo}(i)(\mathsf{z}))\\ i\end{array}\right)(s)\\
&b' \leftarrow \tau_{know}(s')\\
&s'' \sim \mathsf{upd\_CG}\left(\begin{array}{l} i \sim \mathsf{CG}(s')\\ \mathsf{observe}\left(\begin{array}{l}\mathsf{if\_then\_else}(\\ \quad b',\\ \quad \mathsf{epi}(i)(\mathsf{t})(\lambda i'.\mathsf{calc}(i')(\mathsf{tip})(\mathsf{z})) \wedge\\ \quad\quad \mathsf{calc}(i)(\mathsf{tip})(\mathsf{z}),\\ \quad \mathsf{epi}(i)(\mathsf{t})(\lambda i'.\mathsf{calc}(i')(\mathsf{tip})(\mathsf{z}))\\ )\end{array}\right)\\ i\end{array}\right)(s')\\
&i \sim \mathsf{CG}(s'')\\
&\mathcal{N}(\max(\lambda d, i.\Pr\left(\begin{array}{l} i' \sim \mathsf{CG}(s'')\\ \mathsf{calc}(i')(\mathsf{tip})(\mathsf{z})\end{array}\right) \geq d), \sigma)\,\mathsf{T}[0,1]
\end{aligned}
$$

To get this probabilistic program into a manageable form, we need to apply

δ-rules. To a get a clearer picture of what the program is accomplishing, let's see what we get from applying the rules for managing states and indices, along with the rule that removes vacuous bindings and the one for taking maxima of sets of degrees. If we apply `betaDeltaNormal (states <|> cleanUp <|> maxes)` to (94), we get (95).

(95)
$$
\begin{aligned}
& p \sim \mathsf{Logit\_Normal}(0,1)\\
& q \sim \mathsf{Logit\_Normal}(0,1)\\
& r \sim \mathsf{Logit\_Normal}(-2,1)\\
& t \sim \mathsf{Logit\_Normal}(2,1)\\
& u \sim \mathsf{Logit\_Normal}(0,1)\\
& b \sim \mathsf{Bernoulli}(u)\\
& \mathcal{N}\left(\mathsf{Pr}\left(\begin{array}{l}
c \sim \mathsf{Bernoulli}(p)\\
d \sim \mathsf{Bernoulli}(q)\\
e \sim \mathsf{Bernoulli}(r)\\
f \sim \mathsf{Bernoulli}(t)\\
\mathsf{observe}(c)\\
\mathsf{observe}(\mathsf{if\_then\_else}(b, d \wedge \mathsf{if\_then\_else}(c,e,f), d))\\
\boxed{\mathsf{if\_then\_else}(c,e,f)}
\end{array}\right), \sigma\right)\mathsf{T}[0,1]
\end{aligned}
$$

This result shows us that the truncated normal likelihood of the response distribution is centered at a particular probability: the probability that the Bernoulli variable e is T, or that the Bernoulli variable f is T, where the choice depends—by an *if then else* statement—on whether the Bernoulli variable c is T or F. What are e and f? e is whether or not Zoe calculated the tip, assuming she is five years old, and f is whether or not Zoe calculated the tip, assuming she is not five years old. Thus c encodes whether or not Zoe is five.

Of course, we would ultimately like to end up with e, not f: crucially, c—Zoe's five-year-old status—has been observed! Thus we need to handle both observations and *if then else* statements. The former can be handled by incorporating both the `observations` and the `disjunctions` rules (recall that we need the latter to marginalize out Bernoulli distributions).

If then else statements may be handled by the following δ-rule, repeated from Section 3.3.

```
ite :: DeltaRule
ite = \case
  ITE Tr x y -> Just x
  ITE Fa x y -> Just y
  _          -> Nothing
```

Finally, we need one more δ-rule; specifically, to handle the fact that we have used a *conjunction* inside the second observe statement in (95) (and in (94)). To treat this case, we can call on the δ-rule `logical`, repeated here from Section 3.3.

```
logical :: DeltaRule
logical = \case
  And p  Tr -> Just p
  And Tr p  -> Just p
  And Fa _  -> Just Fa
  And _  Fa -> Just Fa
  Or  p  Fa -> Just p
  Or  Fa p  -> Just p
  Or  Tr _  -> Just Tr
  Or  _  Tr -> Just Tr
  _         -> Nothing
```

Note that the patterns featured in this rule which are important for the current example are `And Tr p` and `And Fa p`, since these will allow the Bernoulli distribution from which the variable d is sampled to ultimately be marginalized out.

Finally, if we apply

```
betaDeltaNormal ( states       <||>
                  cleanUp      <||>
                  maxes        <||>
                  observations <||>
                  disjunctions <||>
                  ite          <||>
                  logical )
```

to (94), we end up with (96).

(96) $r \sim \mathsf{Logit\_Normal}(-2, 1)$
$u \sim \mathsf{Logit\_Normal}(0, 1)$
$\mathsf{disj}(u, \mathcal{N}(\mathsf{Pr}(\boxed{\mathsf{T}}), \sigma)\,\mathsf{T}[0,1], \mathcal{N}(\mathsf{Pr}(\mathsf{disj}(r, \boxed{\mathsf{T}}, \boxed{\mathsf{F}})), \sigma)\,\mathsf{T}[0,1])$

We can see, in (96), a more recognizable probability distribution beginning to take shape—a mixture of two truncated normal distributions. All that's left is to compute the two probabilities that parameterize these distributions, respectively. To do this, we can use our `probabilities` δ-rule, introduced in Section 3.3:

```
probabilities :: DeltaRule
probabilities = \case
  Pr (Return Tr)  -> Just 1
  Pr (Return Fa)  -> Just 0
  Pr (Bern x)     -> Just x
  Pr (Disj x t u) -> Just (x * Pr t + (1 - x) * Pr u)
  _               -> Nothing
```

Crucially, this δ-rule computes that the probability of T is 1, that the probability of F is 0, and that the probability of a disjunction of programs with weight x is just x multiplied the probability of the first program, added to $1-x$ multiplied by the probability of the second program. Since in the case at hand, the probability of the relevant second program will evaluate to 0, and should thus be removed from the sum, it'll be useful to also have our `arithemtic` δ-rule handy (see Section 3.3). Putting it all together, i.e., applying

```
betaDeltaNormal ( states        <||>
                  cleanUp       <||>
                  maxes         <||>
                  observations  <||>
                  disjunctions  <||>
                  ite           <||>
                  logical       <||>
                  probabilities <||>
                  arithmetic )
```

thus results in (97).

(97) $r \sim \mathsf{Logit\_Normal}(-2, 1)$
$u \sim \mathsf{Logit\_Normal}(0, 1)$
$\mathsf{disj}(u, \mathcal{N}(1, \sigma)\,\mathsf{T}[0, 1], \mathcal{N}(r, \sigma)\,\mathsf{T}[0, 1])$

From λ-terms to Stan parameters

The PDS kernel represented by (97) can be straightforwardly translated into Stan:

```
model {
  // FIXED EFFECTS
```

```
  w ~ logit_normal(-2.0, 1.0);
  v ~ logit_normal(0.0, 1.0);

  // LIKELIHOOD
  target +=
    log_mix(v,
      truncated_normal_lpdf(y | 1.0, sigma, 0.0, 1.0),
      truncated_normal_lpdf(y | w, sigma, 0.0, 1.0));
}
```

By comparing the two representations—λ-term and Stan code—one can see that the sampling statements are left intact, while the returned program disjunction is translated into a `log_mix` statement (one which computes a log-probability and adds it to the running total). Stan's encoding of a mixture of two distributions captures the discrete branching associated with probability `v`: either the response is near 1, i.e., if τ_{know} is set to T and *know* is thus assigned a factive interpretation; or, if τ_{know} is set to F and the response thus depends only on world knowledge `w`. Remarkably, this model, which encodes these important aspects of the meaning we've assigned to clause-selecting *know*, is obtained automatically once we compose our lexical entries into sentences, slot them into discourses and then response functions, and then apply δ-rules to the result: (94) and (97) describe exactly the same probability distribution.

4.3.3 Wholly gradient factivity in PDS

The model which encodes the Wholly-Gradient Hypothesis (86b) requires that we adjust two aspects of the model defined above. First, we need to update the lexical entry for *know*, in order to render its factivity parameter *gradient*; and second, we should accommodate the resulting change in the prior distribution over the states.

A new lexical entry for *know* is provided in (98).

(98) a. *know*

$$\boxed{\begin{array}{l}\lambda\phi, x, i.\mathsf{if\_then\_else}(\\ \quad \tau_{know}(i),\\ \quad \mathsf{epi}(i)(x)(\phi) \wedge \phi(i),\\ \quad \mathsf{epi}(i)(x)(\phi)\\ )\end{array}} \vdash ap\backslash d/s$$

b.

```
lexKnowWG :: Lexicon SynSem
lexKnowWG = \case
  "know" ->
    [ SynSem {
        syn = s \\ np // s,
        sem =
          ty factSig (
            purePP (lam p (lam x (lam i (
              ITE (TauKnow i)
                (SCon "&" @@
                  ((SCon "epi" @@ i @@ x @@ p)
                    & ϕ @@ i) )
                (SCon "epi" @@ i @@ x @@ p)
            ) ) ) ) ) )
      } ]
  _     -> []
```

According to (98), whether or not *know* receives a factive interpretation no longer depends on the state: it now depends on an index of the common ground. Thus reflecting the Wholly-Gradient Hypothesis, this lexical entry regards uncertainty about factivity as on a par with the uncertainty which is fundamentally associated with world knowledge—it is just the kind of uncertainty that can be probed with questions containing predicates such as *likely* and *certain*.

To adjust the prior distribution over states, we need only move the sampling statement associated with the factivity parameter into the definition of the common ground, where it may update indices. Doing this yields (99).

(99)
$$
\begin{aligned}
&p \sim \text{Logit\_Normal}(0,1)\\
&q \sim \text{Logit\_Normal}(0,1)\\
&r \sim \text{Logit\_Normal}(-2,1)\\
&t \sim \text{Logit\_Normal}(2,1)\\
&u \sim \text{Logit\_Normal}(0,1)\\
&\text{upd\_CG}\left(\begin{array}{l}
b \sim \text{Bernoulli}(u)\\
c \sim \text{Bernoulli}(p)\\
d \sim \text{Bernoulli}(q)\\
e \sim \text{Bernoulli}(r)\\
f \sim \text{Bernoulli}(t)\\
\text{upd\_five\_yo}(\lambda x.c)(\\
\quad \text{upd\_epi}(\lambda \phi, x.d)(\\
\quad\quad \text{upd\_calc}(\\
\quad\quad\quad \lambda x, y.\text{if\_then\_else}(c,e,f)\\
\quad\quad)(\text{upd\_}\tau_{know}(b)(@))\\
\quad))
\end{array}\right)(\epsilon)
\end{aligned}
$$

By using the new lexical entry for *know* together with the new definition of the prior, we obtain the response distribution defined in (100).

(100)
$$
\begin{aligned}
& p \sim \mathsf{Logit\_Normal}(0,1)\\
& q \sim \mathsf{Logit\_Normal}(0,1)\\
& r \sim \mathsf{Logit\_Normal}(-2,1)\\
& t \sim \mathsf{Logit\_Normal}(2,1)\\
& u \sim \mathsf{Logit\_Normal}(0,1)\\
& s \sim \mathsf{upd\_CG}\left(\begin{array}{l} b \sim \mathsf{Bernoulli}(u)\\ c \sim \mathsf{Bernoulli}(p)\\ d \sim \mathsf{Bernoulli}(q)\\ e \sim \mathsf{Bernoulli}(r)\\ f \sim \mathsf{Bernoulli}(t)\\ \mathsf{upd\_five\_yo}(\lambda x.c)(\\ \quad \mathsf{upd\_epi}(\lambda\phi, x.d)(\\ \quad\quad \mathsf{upd\_calc}(\\ \quad\quad\quad \lambda x, y.\mathsf{if\_then\_else}(c,e,f)\\ \quad\quad)\mathsf{upd}\_\tau_{know}(b)(@)\\ \quad))\end{array}\right)(\mathsf{upd}\_\tau_{know}(b)(\epsilon))\\
& s' \sim \mathsf{upd\_CG}\left(\begin{array}{l} i \sim \mathsf{CG}(s)\\ \mathsf{observe}(\mathsf{five\_yo}(i)(\mathsf{z}))\\ i\end{array}\right)(s)\\
& s'' \sim \mathsf{upd\_CG}\left(\begin{array}{l} i \sim \mathsf{CG}(s')\\ \mathsf{observe}\left(\begin{array}{l}\mathsf{if\_then\_else}(\\ \quad \tau_{know}(i'),\\ \quad \mathsf{epi}(i)(\mathsf{t})(\lambda i'.\mathsf{calc}(i')(\mathsf{tip})(\mathsf{z})) \wedge\\ \quad\quad \mathsf{calc}(i)(\mathsf{tip})(\mathsf{z}),\\ \quad \mathsf{epi}(i)(\mathsf{t})(\lambda i'.\mathsf{calc}(i')(\mathsf{tip})(\mathsf{z}))\\ )\end{array}\right)\\ i\end{array}\right)(s')\\
& i \sim \mathsf{CG}(s'')\\
& \mathcal{N}(\mathsf{max}(\lambda d, i.\mathsf{Pr}\left(\begin{array}{l} i' \sim \mathsf{CG}(s'')\\ \mathsf{calc}(i')(\mathsf{tip})(\mathsf{z})\end{array}\right) \geq d), \sigma)\,\mathsf{T}[0,1]
\end{aligned}
$$

Finally, if we apply the very set of δ-rules to (100) which we used to simplify (94), we end up with the result in (101).

(101)
$$
\begin{aligned}
& r \sim \mathsf{Logit\_Normal}(-2,1)\\
& u \sim \mathsf{Logit\_Normal}(0,1)\\
& \mathcal{N}(u + (1-u) * r, \sigma)\,\mathsf{T}[0,1]
\end{aligned}
$$

From λ-terms to Stan parameters

This PDS kernel can be translated into Stan as follows:

```
model {
  // FIXED EFFECTS
  w ~ logit_normal(-2.0, 1.0);
  v ~ logit_normal(0.0, 1.0);

  // LIKELIHOOD
  target += truncated_normal_lpdf( y |
    v + (1.0 - v) * w,
    sigma,
    0.0,
    1.0
  );
}
```

As we see, there is no longer any mixture of probability distributions, due to the fact that (101) itself doesn't contain a program disjunction. Instead, we obtain a continuous function of the two parameters `v` and `w` —one which takes their fuzzy-logic disjunction! This contrasts with the model corresponding to the Discrete-Factivity Hypothesis, which employed discrete branching. This version of the model makes the gradient contribution of factivity clear: *know*, here, provides a multiplicative boost to world knowledge rather than overriding it entirely.

5 Conclusion

We have introduced Probabilistic Dynamic Semantics (PDS) as a framework that merges the strengths of Dynamic Montague Semantics (DMS) with probabilistic reasoning. PDS maintains the compositional and modular characteristics that make modern DMS powerful, while introducing probabilistic elements that allow for more nuanced predictions about semantic inference. By retaining the core structure of traditional dynamic semantic representations and augmenting them with representations of probabilistic programs, PDS offers a dual-level analysis: one level which preserves the interpretative richness of DMS and another enabling precise, quantitative assessment in terms of experimental data. This approach not only allows one to seamlessly incorporate probabilistic reasoning into an existing semantic theory, but also facilitates the explicit modeling

of the way linguistic expressions are interpreted in context, considering both the range of possible meanings and the likelihood that those meanings are inferred by language comprehenders.

Our exposition has been primarily formal, laying out the groundwork for future (as well as ongoing) empirical investigations. The ability to develop, test, and refine semantic analyses using experimentally derived datasets is integral to the advancement of semantic theory—we hope that PDS can support this goal.

References

Asher, N. (1993). *Reference to Abstract Objects in Discourse* (Vol. 50; G. Chierchia, P. Jacobson, & F. J. Pelletier, Eds.). Dordrecht: Springer Netherlands. Retrieved 2024-09-13, from http://link.springer.com/10.1007/978-94-011-1715-9 doi: 10.1007/978-94-011-1715-9

Atkey, R. (2009, July). Parameterised notions of computation. *Journal of Functional Programming*, *19*(3-4), 335–376. Retrieved 2020-09-16, from https://doi.org/10.1017/S095679680900728X doi: 10.1017/S095679680900728X

Barker, C. (2002, February). The Dynamics of Vagueness. *Linguistics and Philosophy*, *25*(1), 1–36. Retrieved 2021-11-29, from https://doi.org/10.1023/A:1014346114955 doi: 10.1023/A:1014346114955

Barker, C., & Shan, C.-c. (2014). *Continuations and natural language* (Vol. 53). Oxford studies in theoretical linguistics.

Beaver, D. I. (1999). Presupposition Accomodation: A Plea for Common Sense. In L. S. Moss, J. Ginzburg, & R. de Maarten (Eds.), *Logic, language, and computation* (Vol. 2, pp. 21–44). Stanford: CSLI Publications.

Beaver, D. I. (2001). *Presupposition and Assertion in Dynamic Semantics*. Stanford: CSLI Publications. Retrieved from https://semanticsarchive.net/Archive/jU1MDVmZ

Benton, P. N., Bierman, G. M., & Paiva, V. C. V. D. (1998, March). Computational types from a logical perspective. *Journal of Functional Programming*, *8*(2), 177–193. Retrieved 2020-09-16, from https://www.cambridge.org/core/journals/journal-of-functional-programming/article/computational-types-from-a-logical-perspective/37B1EAE149C3EE88BE5A90EF9B56FD4F (Publisher: Cambridge University Press) doi: 10.1017/S0956796898002998

Bergen, L., Levy, R., & Goodman, N. (2016, May). Pragmatic reasoning through

semantic inference. *Semantics and Pragmatics*, *9*, ACCESS–ACCESS. Retrieved 2023-07-26, from https://semprag.org/index.php/sp/article/view/sp.9.20 doi: 10.3765/sp.9.20

Bernardy, J.-P., Blanck, R., Chatzikyriakidis, S., Lappin, S., & Maskharashvili, A. (2019, September). Predicates as Boxes in Bayesian Semantics for Natural Language. In *Proceedings of the 22nd Nordic Conference on Computational Linguistics* (pp. 333–337). Turku, Finland: Linköping University Electronic Press. Retrieved 2021-03-01, from https://www.aclweb.org/anthology/W19-6137

Bernardy, J.-P., Blanck, R., Chatzikyriakidis, S., & Maskharashvili, A. (2022). Bayesian Natural Language Semantics and Pragmatics. In J.-P. Bernardy, R. Blanck, S. Chatzikyriakidis, S. Lappin, & A. Maskharashvili (Eds.), *Probabilistic Approaches to Linguistic Theory.* CSLI Publications.

Bumford, D., & Charlow, S. (2022, November). *Dynamic semantics with static types.* LingBuzz. Retrieved 2023-11-27, from https://ling.auf.net/lingbuzz/006884 (LingBuzz Published In: Submitted)

Bumford, D., & Charlow, S. (2025, March). *Effect-driven interpretation: Functors for natural language composition.* LingBuzz. Retrieved 2025-07-06, from https://ling.auf.net/lingbuzz/008912 (LingBuzz Published In: Submitted to Cambridge University Press Elements in Semantics)

Bürkner, P.-C. (2017, August). brms: An R Package for Bayesian Multilevel Models Using Stan. *Journal of Statistical Software*, *80*, 1–28. Retrieved 2025-01-08, from https://doi.org/10.18637/jss.v080.i01 doi: 10.18637/jss.v080.i01

Charlow, S. (2014). *On the semantics of exceptional scope* (PhD Thesis, New York University, New York). Retrieved from https://semanticsarchive.net/Archive/2JmMWRjY

Charlow, S. (2019, November). Where is the destructive update problem? *Semantics and Pragmatics*, *12*, 10:1–24. Retrieved 2024-09-21, from https://semprag.org/index.php/sp/article/view/sp.12.10 doi: 10.3765/sp.12.10

Charlow, S. (2020a). The scope of alternatives: indefiniteness and islands. *Linguistics and Philosophy*, *43*(4), 427–472. doi: 10.1007/s10988-019-09278-3

Charlow, S. (2020b). *Static and dynamic exceptional scope.* Retrieved from https://ling.auf.net/lingbuzz/004650 (Publisher: Rutgers University Published: LingBuzz)

Degen, J., & Tonhauser, J. (2021, September). Prior Beliefs Modulate Projection. *Open Mind*, *5*, 59–70. Retrieved 2023-04-25, from https://doi.org/

10.1162/opmi_a_00042 doi: 10.1162/opmi_a_00042

Degen, J., & Tonhauser, J. (2022, September). Are there factive predicates? An empirical investigation. *Language*, *98*(3), 552–591. Retrieved 2022-11-28, from https://muse.jhu.edu/article/864635 (Publisher: Linguistic Society of America) doi: 10.1353/lan.0.0271

de Groote, P. (2006, August). Towards a Montagovian Account of Dynamics. *Semantics and Linguistic Theory*, *16*(0), 1–16. Retrieved 2020-10-12, from https://journals.linguisticsociety.org/proceedings/index.php/SALT/article/view/2952 (Number: 0) doi: 10.3765/salt.v16i0.2952

Dekker, P. (1993). *Transsentential Meditations; Ups and downs in dynamic semantics* (doctoral, University of Amsterdam). Retrieved 2024-09-13, from https://eprints.illc.uva.nl/id/eprint/1958/

Elliott, P. D. (2022, October). A flexible scope theory of intensionality. *Linguistics and Philosophy*. Retrieved 2022-10-22, from https://doi.org/10.1007/s10988-022-09367-w doi: 10.1007/s10988-022-09367-w

Erk, K. (2022, January). The Probabilistic Turn in Semantics and Pragmatics. *Annual Review of Linguistics*, *8*(Volume 8, 2022), 101–121. Retrieved 2024-09-13, from https://www.annualreviews.org/content/journals/10.1146/annurev-linguistics-031120-015515 (Publisher: Annual Reviews) doi: 10.1146/annurev-linguistics-031120-015515

Erk, K., & Herbelot, A. (2023, November). How to Marry a Star: Probabilistic Constraints for Meaning in Context. *Journal of Semantics*, *40*(4), 549–583. Retrieved 2024-09-13, from https://doi.org/10.1093/jos/ffad016 doi: 10.1093/jos/ffad016

Farkas, D. F., & Bruce, K. B. (2010, February). On Reacting to Assertions and Polar Questions. *Journal of Semantics*, *27*(1), 81–118. Retrieved 2024-04-28, from https://doi.org/10.1093/jos/ffp010 doi: 10.1093/jos/ffp010

Frank, M. C., & Goodman, N. D. (2012, May). Predicting Pragmatic Reasoning in Language Games. *Science*, *336*(6084), 998–998. Retrieved 2022-06-20, from https://www.science.org/doi/10.1126/science.1218633 (Publisher: American Association for the Advancement of Science) doi: 10.1126/science.1218633

Ginzburg, J. (1996). Dynamics and the semantics of dialogue. In J. Seligman & D. Westerståhl (Eds.), *Logic, Language, and Computation* (Vol. 1, pp. 221–237). Stanford: CSLI Publications.

Giorgolo, G., & Asudeh, A. (2012). ⟨M, η, $\star$⟩ Monads for Conventional

Implicatures. In A. A. Guevara, A. Chernilovskaya, & R. Nouwen (Eds.), *Sinn und Bedeutung* (Vol. 16, pp. 265–278). Retrieved from http://mitwpl.mit.edu/open/sub16/Giorgolo.pdf

Giorgolo, G., & Asudeh, A. (2014). One Semiring to Rule Them All. In *CogSci 2014 Proceedings.* Retrieved from https://cogsci.mindmodeling.org/2014/papers/031/

Giorgolo, G., & Unger, C. (2009). Coreference without Discourse Referents. In B. Plank, E. T. K. Sang, & T. Van de Cruys (Eds.), *The 19th Meeting of Computational Linguistics in the Netherlands* (pp. 69–81).

Goodman, N. D., & Lassiter, D. (2015). Probabilistic Semantics and Pragmatics Uncertainty in Language and Thought. In S. Lappin & C. Fox (Eds.), *The Handbook of Contemporary Semantic Theory* (pp. 655–686). John Wiley & Sons, Ltd. Retrieved 2021-04-20, from http://onlinelibrary.wiley.com/doi/abs/10.1002/9781118882139.ch21 (Section: 21 _eprint: https://onlinelibrary.wiley.com/doi/pdf/10.1002/9781118882139.ch21) doi: 10.1002/9781118882139.ch21

Goodman, N. D., Mansinghka, V. K., Roy, D., Bonawitz, K., & Tenenbaum, J. B. (2008, July). Church: a language for generative models. In *Proceedings of the Twenty-Fourth Conference on Uncertainty in Artificial Intelligence* (pp. 220–229). Arlington, Virginia, USA: AUAI Press.

Goodman, N. D., & Stuhlmüller, A. (2014). *The Design and Implementation of Probabilistic Programming Languages.* Retrieved 2024-11-15, from http://dippl.org

Goodman, N. D., Tenenbaum, J. B., Buchsbaum, D., Hartshorne, J., Hawkins, R., O'Donnell, T., & Tessler, M. H. (2016). *Probabilistic Models of Cognition (2nd ed.).* Retrieved 2024-06-30, from http://probmods.org/

Groenendijk, J., & Stokhof, M. (1984). *Studies on the semantics of questions and the pragmatics of answers* (Doctoral dissertation, University of Amsterdam, Amsterdam). Retrieved from https://stokhof.org/wp-content/uploads/2020/09/groenendijk-stokhof_ssqpa.pdf

Groenendijk, J., & Stokhof, M. (1990). Dynamic Montague Grammar. In L. Kálmán & L. Pólos (Eds.), *Proceedings of the Second Symposium on Logic and Language* (pp. 3–48). Budapest: Eötvös Loránd Press.

Groenendijk, J. A., & Stokhof, M. B. J. (1991). Dynamic Predicate Logic. *Linguistics and Philosophy*, *14*(1), 39–100. Retrieved from https://doi.org/10.1007/BF00628304

Grove, J. (2019). *Scope-taking and presupposition satisfaction* (PhD Thesis, University of Chicago, Chicago). Retrieved from https://semanticsarchive.net/Archive/TRmOTkzM

Grove, J., & Bernardy, J.-P. (2023). Probabilistic Compositional Semantics, Purely. In K. Yada, Y. Takama, K. Mineshima, & K. Satoh (Eds.), *New Frontiers in Artificial Intelligence* (pp. 242–256). Cham: Springer Nature Switzerland. doi: 10.1007/978-3-031-36190-6_17

Grove, J., & White, A. S. (2024, March). *Factivity, presupposition projection, and the role of discrete knowlege in gradient inference judgments.* LingBuzz. Retrieved 2024-04-25, from https://ling.auf.net/lingbuzz/007450 (LingBuzz Published In: submitted)

Grove, J., & White, A. S. (2025, January). Modeling the prompt in inference judgment tasks. *Experiments in Linguistic Meaning*, *3*, 176–187. Retrieved 2025-04-02, from https://journals.linguisticsociety.org/proceedings/index.php/ELM/article/view/5857 doi: 10.3765/elm.3.5857

Hamblin, C. L. (1973). Questions in Montague English. *Foundations of Language*, *10*(1), 41–53. Retrieved 2021-04-23, from http://www.jstor.org/stable/25000703 (Publisher: Springer)

Hausser, R., & Zaefferer, D. (1978). Questions and Answers in a Context-Dependent Montague Grammar. In F. Guenthner & S. J. Schmidt (Eds.), *Formal Semantics and Pragmatics for Natural Languages* (pp. 339–358). Dordrecht: Springer Netherlands. Retrieved 2024-04-19, from https://doi.org/10.1007/978-94-009-9775-2_12 doi: 10.1007/978-94-009-9775-2_12

Hausser, R. R. (1983). The Syntax and Semantics of English Mood. In F. Kiefer (Ed.), *Questions and Answers* (pp. 97–158). Dordrecht: Springer Netherlands. Retrieved 2024-04-19, from https://doi.org/10.1007/978-94-009-7016-8_6 doi: 10.1007/978-94-009-7016-8_6

Heim, I. (1982). *The Semantics of Definite and Indefinite Noun Phrases* (PhD Thesis, University of Massachussetts, Amherst). Retrieved from https://semanticsarchive.net/Archive/Tk0ZmYyY/

Heim, I. (1983). File Change Semantics and the Familiarity Theory of Definiteness. In R. Bäuerle, C. Schwarze, & A. v. Stechow (Eds.), *Meaning, Use, and Interpretation of Language* (pp. 164–189). De Gruyter. Retrieved 2024-09-13, from https://www.degruyter.com/document/doi/10.1515/9783110852820.164/html?lang=en doi: 10.1515/9783110852820.164

Joshi, A. K. (1985). Tree adjoining grammars: How much context-sensitivity is required to provide reasonable structural descriptions? In A. M. Zwicky, D. R. Dowty, & L. Karttunen (Eds.), *Natural Language Parsing: Psychological, Computational, and Theoretical Perspectives* (pp. 206–250). Cambridge: Cam-

bridge University Press. Retrieved 2024-10-02, from https://www.cambridge.org/core/books/natural-language-parsing/tree-adjoining-grammars-how-much-contextsensitivity-is-required-to-provide-reasonable-structural-descriptions/81BFD6DAC6B0CB24A3042A06E964F2E1 doi: 10.1017/CBO9780511597855.007

Kamp, H. (1981). A Theory of Truth and Semantic Representation. In J. A. Groenendijk, T. M. V. Janssen, & M. B. J. Stokhof (Eds.), *Formal Methods in the Study of Language* (pp. 277–322). Amsterdam: Mathematisch Centrum.

Kamp, H., & Reyle, U. (1993). *From Discourse to Logic: Introduction to Modeltheoretic Semantics of Natural Language, Formal Logic and Discourse Representation Theory* (Vol. 42). Dordrecht: Springer.

Kane, B., Gantt, W., & White, A. S. (2022, January). Intensional Gaps: Relating veridicality, factivity, doxasticity, bouleticity, and neg-raising. *Semantics and Linguistic Theory*, *31*(0), 570–605. Retrieved 2023-05-11, from https://journals.linguisticsociety.org/proceedings/index.php/SALT/article/view/31.029 doi: 10.3765/salt.v31i0.5137

Karttunen, L. (1976, December). Discourse Referents. In *Notes from the Linguistic Underground* (pp. 363–385). Brill. Retrieved 2024-09-13, from https://brill.com/display/book/edcoll/9789004368859/BP000021.xml (Section: Notes from the Linguistic Underground) doi: 10.1163/9789004368859_021

Karttunen, L. (1977). Syntax and Semantics of Questions. *Linguistics and Philosophy*, *1*(1), 3–44. Retrieved 2024-06-17, from https://www.jstor.org/stable/25000027 (Publisher: Springer)

Kennedy, C. (2007, February). Vagueness and grammar: the semantics of relative and absolute gradable adjectives. *Linguistics and Philosophy*, *30*(1), 1–45. Retrieved 2022-10-26, from https://doi.org/10.1007/s10988-006-9008-0 doi: 10.1007/s10988-006-9008-0

Kennedy, C., & McNally, L. (2005). Scale Structure, Degree Modification, and the Semantics of Gradable Predicates. *Language*, *81*(2), 345–381. Retrieved 2023-07-27, from https://muse.jhu.edu/pub/24/article/184167 (Publisher: Linguistic Society of America) doi: 10.1353/lan.2005.0071

Klecha, P. (2012). Positive and Conditional Semantics for Gradable Modals. *Proceedings of Sinn und Bedeutung*, *16*(2), 363–376. Retrieved 2023-12-14, from https://ojs.ub.uni-konstanz.de/sub/index.php/sub/article/view/433 (Number: 2)

Kobele, G. (2006). *Generating Copies: An investigation into structural identity in language and grammar* (Unpublished doctoral dissertation). UCLA, Los Angeles.

Lassiter, D. (2011). Vagueness as Probabilistic Linguistic Knowledge. In R. Nouwen, R. van Rooij, U. Sauerland, & H.-C. Schmitz (Eds.), *Vagueness in Communication* (pp. 127–150). Berlin, Heidelberg: Springer. doi: 10.1007/978-3-642-18446-8_8

Lassiter, D. (2017). *Graded Modality: Qualitative and Quantitative Perspectives*. Oxford, New York: Oxford University Press.

Lassiter, D., & Goodman, N. D. (2017, October). Adjectival vagueness in a Bayesian model of interpretation. *Synthese*, *194*(10), 3801–3836. Retrieved 2021-02-11, from https://doi.org/10.1007/s11229-015-0786-1 doi: 10.1007/s11229-015-0786-1

Liang, S., Hudak, P., & Jones, M. (1995). Monad transformers and modular interpreters. In *POPL '95 Proceedings of the 22nd ACM SIGPLAN-SIGACT symposium on Principles of programming languages* (pp. 333–343). New York. Retrieved from https://doi.org/10.1145/199448.199528 (event-place: San Francisco)

Link, G. (1983). The Logical Analysis of Plurals and Mass Terms: A Lattice-theoretical Approach. In R. Bäuerle, C. Schwarze, & A. v. Stechow (Eds.), *Meaning, Use, and Interpretation of Language* (pp. 303–329). De Gruyter. Retrieved 2025-09-02, from https://www.degruyterbrill.com/document/doi/10.1515/9783110852820.302/html?lang=en&srsltid=AfmBOoox_QoerB82cweUfyLXGxbs3Q_issFNXmH9n2Y4yd1y1AGTbsQh

Muskens, R. (1996, April). Combining Montague semantics and discourse representation. *Linguistics and Philosophy*, *19*(2), 143–186. Retrieved 2023-12-03, from https://doi.org/10.1007/BF00635836 doi: 10.1007/BF00635836

Phillips, C., Gaston, P., Huang, N., & Muller, H. (2021). Theories All the Way Down: Remarks on "Theoretical" and "Experimental" Linguistics. In G. Goodall (Ed.), *The Cambridge Handbook of Experimental Syntax* (pp. 587–616). Cambridge: Cambridge University Press. Retrieved 2024-09-17, from https://www.cambridge.org/core/books/cambridge-handbook-of-experimental-syntax/theories-all-the-way-down/2745E21345BE3F05D3BC430FF75A798A doi: 10.1017/9781108569620.023

Roberts, C. (2012, December). Information Structure: Towards an integrated formal theory of pragmatics. *Semantics and Pragmatics*, *5*, 6:1–69. Retrieved 2023-07-26, from https://semprag.org/index.php/sp/

article/view/sp.5.6 doi: 10.3765/sp.5.6

Shan, C.-c. (2002, May). Monads for natural language semantics. *arXiv:cs/0205026*. Retrieved 2020-10-06, from http://arxiv.org/abs/cs/0205026 (arXiv: cs/0205026)

Simons, M. (2007). Observations on embedding verbs, evidentiality, and presupposition. *Lingua*, *117*(6), 1034–1056. doi: 10.1016/j.lingua.2006.05.006

Simons, M., Beaver, D., Roberts, C., & Tonhauser, J. (2017). The Best Question: Explaining the Projection Behavior of Factives. *Discourse Processes*, *54*(3), 187–206.

Simons, M., Tonhauser, J., Beaver, D., & Roberts, C. (2010). What projects and why. In Nan Li & David Lutz (Eds.), *Semantics and Linguistic Theory* (Vol. 20, pp. 309–327). University of British Columbia and Simon Fraser University: Linguistic Society of America. doi: 10.3765/salt.v20i0.2584

Stabler, E. (1997). Derivational minimalism. In C. Retoré (Ed.), *Logical Aspects of Computational Linguistics* (pp. 68–95). Berlin, Heidelberg: Springer. doi: 10.1007/BFb0052152

Stalnaker, R. (1978). Assertion. In P. Cole (Ed.), *Pragmatics* (Vol. 9, pp. 315–332). New York: Academic Press.

{Stan Development Team}. (2024). *Stan Modeling Language Users Guide and Reference Manual, 2.36* (Tech. Rep.). Retrieved from https://mc-stan.org

Steedman, M. (1996). *Surface Structure and Interpretation*. Cambridge: MIT Press.

Steedman, M. (2000). *The Syntactic Process*. Cambridge: MIT Press.

Swanson, E. (2016, April). The Application of Constraint Semantics to the Language of Subjective Uncertainty. *Journal of Philosophical Logic*, *45*(2), 121–146. Retrieved 2025-09-26, from https://doi.org/10.1007/s10992-015-9367-5 doi: 10.1007/s10992-015-9367-5

Tonhauser, J., Beaver, D. I., & Degen, J. (2018, August). How Projective is Projective Content? Gradience in Projectivity and At-issueness. *Journal of Semantics*, *35*(3), 495–542. doi: 10.1093/jos/ffy007

van Benthem, J., Gerbrandy, J., & Kooi, B. (2009, October). Dynamic Update with Probabilities. *Studia Logica*, *93*(1), 67–96. Retrieved 2024-09-17, from https://doi.org/10.1007/s11225-009-9209-y doi: 10.1007/s11225-009-9209-y

Vehtari, A., Gabry, J., Magnusson, M., Yao, Y., Bürkner, P.-C., Paananen, T., ... Lindgren, L. (2023, March). *loo: Efficient Leave-One-Out Cross-Validation and WAIC for Bayesian Models.* Retrieved 2023-06-12, from https://cran.r-project.org/web/packages/loo/index.html

Vijay-Shanker, K., & Weir, D. J. (1994, November). The equivalence of four extensions of context-free grammars. *Mathematical systems theory*, *27*(6), 511–546. Retrieved 2025-06-17, from https://doi.org/10.1007/BF01191624 doi: 10.1007/BF01191624

Watanabe, S. (2013). A Widely Applicable Bayesian Information Criterion. *Journal of Machine Learning Research*, *14*(27), 867–897. Retrieved from http://jmlr.org/papers/v14/watanabe13a.html

White, A. S., & Rawlins, K. (2018). The role of veridicality and factivity in clause selection. In S. Hucklebridge & M. Nelson (Eds.), *NELS 48: Proceedings of the Forty-Eighth Annual Meeting of the North East Linguistic Society* (Vol. 48, pp. 221–234). University of Iceland: GLSA (Graduate Linguistics Student Association), Department of Linguistics, University of Massachusetts.

Xiang, Y. (2021, June). A hybrid categorial approach to question composition. *Linguistics and Philosophy*, *44*(3), 587–647. Retrieved 2024-09-26, from https://doi.org/10.1007/s10988-020-09294-8 doi: 10.1007/s10988-020-09294-8

Yalcin, S. (2010). Probability Operators. *Philosophy Compass*, *5*(11), 916–937. Retrieved 2024-07-09, from https://onlinelibrary.wiley.com/doi/abs/10.1111/j.1747-9991.2010.00360.x (_eprint: https://compass.onlinelibrary.wiley.com/doi/pdf/10.1111/j.1747-9991.2010.00360.x) doi: 10.1111/j.1747-9991.2010.00360.x

Zeevat, H. (2013). Implicit Probabilities in Update Semantics. In *The dynamic, inquisitive, and visionary life of ϕ, ?ϕ, and $\diamond\phi$: A festschrift for Jeroen Groenendijk, Martin Stokhof, and Frank Veltman* (pp. 319–324). Amsterdam: ILLC.